Overview of C Language

studytonight.com/c/overview-of-c.php

Welcome to the tutorial series for C language. This is the best C language tutorial set on the Internet, that will help you learn the C language.

C language was developed by **Dennis Ritchie in 1972 at Bell Laboratories**. In 1989, the C language was standardized, where C language features were defined, also known as the **1989 ANSI standard** for C, and that is the reason, you will see C language also called **C89**, because of the 1989 ANSI standard.

The current latest version of the C language is **C99**, as some new features were added to the C language in **1999**.

You must be thinking, 1999 was more than 20 years ago, but the **C language is still very relevant** and **widely used** although there are many other new languages like C++, Java, Python, C#, etc. in the market.

C Language is a **middle-level**, **structured programming language**, that **needs a compiler** for running the programs written in C language. Do not worry about these terms, we will cover all of them in detail in this tutorial.

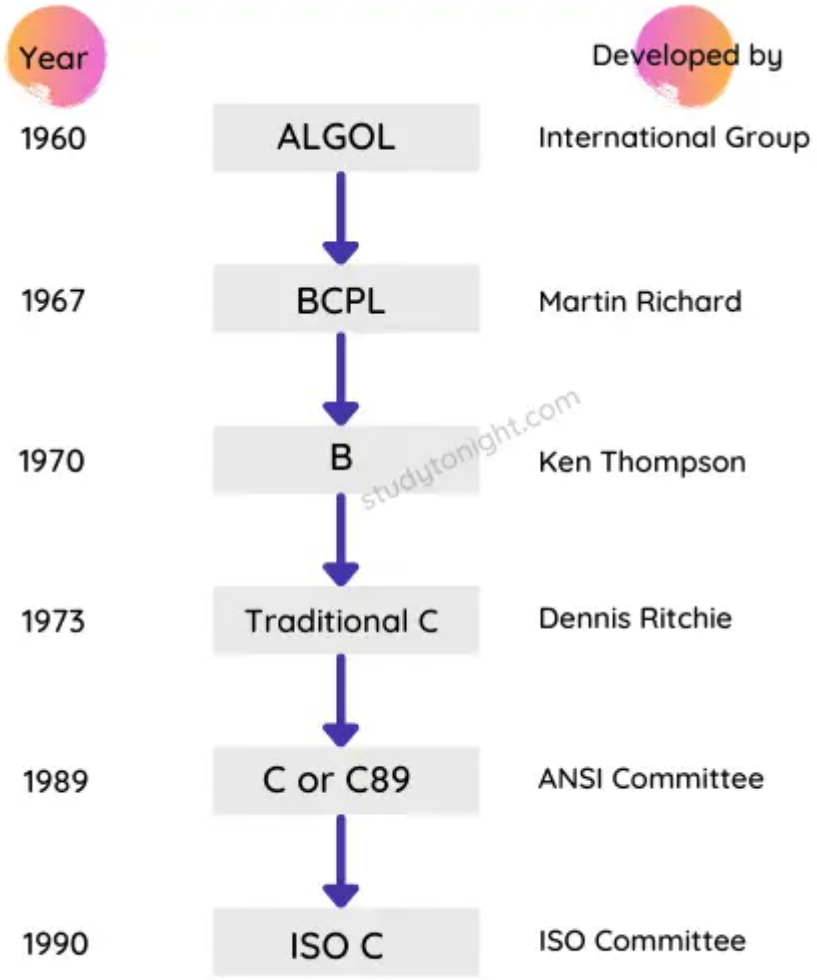
History of C language

C language has evolved from three different structured languages **ALGOL, BCPL, and B Language**. It uses many concepts from these languages while introduced many new concepts such as datatypes, struct, pointer, etc.

BCPL was developed by **Martin Richards**, based on which the **B language** was created by **Ken Thompson**. And then the B language was the language using which the C language was created.

In 1989, the language was formalized by **American National Standard Institute**(ANSI).

In 1990, a version of the C language was approved by the **International Standard Organisation**(ISO), and that version of C is also referred to as **C89**.



After the C language was accepted worldwide and programmers around the world started using it, soon, to improve C language further, the work on the development of C++ language started.

The **idea behind** creating C language was to create an **easy language** that requires a **simple compiler** (to translate the code from English to binary (0's and 1's that computer understands)) and to enable programmers to **write complex programs** in a language which is close to English because earlier programmers had to write Machine instructions which were very difficult to remember.

C language **compiler** converts the readable C language code into machine instructions.

Why is C Language so popular?

C language is a very good language to introduce yourself to the programming world, as it is a simple, and easy-to-learn language that is capable of doing wonders.

Programs that are written in C language **take very little time to execute** and almost execute at the **speed of assembly language instructions**. (Assembly level instructions are nothing but direct commands to communicate with computer's hardware)

Initially, C language was mainly used for writing system-level programs, like designing Operating Systems, because in the eighties (1980-89) the fight to make a stable, worldwide accepted operating system was going on.

But C language can be used to develop other applications as well, like **Text Editors, Compilers, Network Drivers**, many traditional PoS(Point of Sale) software like **Restaurant Billing systems**, etc.

Latest Version of C

The current latest version of the C language is **C11**, which was **introduced in 2018**. It is supported by all the standard C language compilers, but many compilers still do not support this.

Many new features have been introduced in this version, and an overall attempt to improve the compatibility of the C language with the C++ language has been made. We will learn about the **C11** edition once we are done with learning the C language towards the end of this tutorial series.

The **most widely used version** of the C language is still **C89** and **C99**.

What will we learn in this tutorial series?

In this tutorial series, we will cover all the concepts of the C programming language with code examples, simple explanations, and understanding the real-world use of the various concepts as we learn them.

Apart from the tutorials, we provide you rich set of **C programs** and **C Interview MCQs** so that you can test your knowledge parallelly.

You can also try writing C programs and run them in our **online C Compiler**, just log in and start using it.

Frequently Asked Questions

Here are some frequently asked questions, that beginners have when they start learning the C language.

1. What is the C language?

It is a computer programming language developed in the early 1970s by Ken Thompson and Dennis Ritchie at Bell Laboratories.

It is a programming language that was developed to do system programming for the operating system UNIX, but later it was used for developing software and programs for daily usage. It is still being used to develop operating systems and for creating system-level programs, by big tech companies (FAANG companies).

2. What are the features of the C language?

C is one of the most popular programming languages. It has many features:

- **Modularity:** It allows the reusability of modules. It has header files with readymade functions that you can use.
- **Middle-Level language:** It combines both the advantages of low-level and high-level languages. (C arrays, C Pointers, etc.)
- **General-purpose programming language:** It can be used to implement any kind of application, just like Python. (**Learn Python**)
- **Portability:** C program can be compiled or executed in any operating system (Unix, DOS, Windows). You can compile the program in Windows and then run it on a Unix machine, it will run without any issue.
- **Powerful programming language:** It is a very efficient and powerful programming language. It is best used for data structures and designing system software.
- **C is a case-sensitive language**.

We have listed down all the features of the C language in the next tutorial - Features of C Language

3. What is the C language used for?

It is amongst one of the most portable languages and is mainly used for creating system applications that form a major part of Windows, UNIX, and Linux operating systems.

C is a **general-purpose** programming language. It is used for enterprise applications, games, graphics, and applications requiring calculations, or simple software, etc.

4. Where is C used today?

C is used for developing operating systems and Kernel-level programs.

It is used for the production of Compiler. We can develop desktop as well as mobile phone operating systems with the help of C, for example Android.

Conclusion

This tutorial was all about an overview of the C language and its History. C was developed by Dennis Ritchie in 1972 at AT and T's bell laboratories. In the next tutorial, we will learn about C language features in detail.

Features of C language

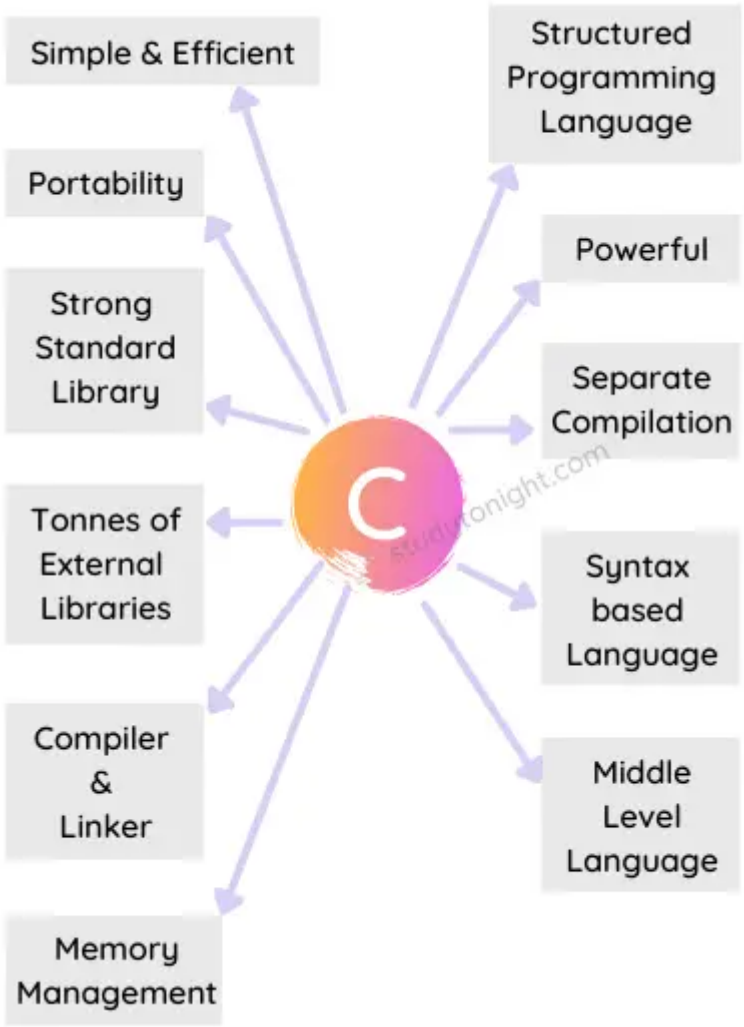
studytonight.com/c/features-of-c.php

Here we have listed all the major features of C language to give you more reasons to learn C language.

The C programming language is a **feature-rich** programming language. It has all the required features that a developer (beginner or expert) would want a programming language to have.

The C language was created from **Basic Cambridge Programming Language(BCPL)** and **B Language**. But it is many times faster than BASIC and also has many more features which we will learn in this tutorial.

Here is a simple chart to list down the **main features of the C language**.



Let's talk about all these features one by one.

1. Simple and Efficient

The C Language is a simple language that is easy to learn even for a beginner and is super efficient to use both in terms of the time of development and time of execution. Yes, if you want to create software using C language you can do so in quite less time depending upon the size of the software that you want to code.

2. Portability

C Language program runs the same way everywhere. It means if you have written a simple C program like a program to find sum of N numbers in C, on your Windows OS laptop or computer, and then compiled it and run it, you can then take the compiled code and run it on any other operating system or machine, like, Linux or macOS, etc., your C program will always return the same result.

If you have software written in the C language for Unix OS, and you now want to run it on Windows OS, you can easily adapt the software for Windows OS, and that is the power of the C language.

Hence, we can say that the C language is portable.

3. Structured Programming language

C language is a structured programming language because we can create functions in the C language. Using functions we can separate a particular operation from the main program and then use it again and again.

A structured language is not just about having the ability to create functions, but it supports **loops**, **conditional statements**, etc. All of this we will cover in detail in the upcoming tutorials.

4. Powerful

C language is a very powerful programming language. It has a broad range of features like support for many **data types, operators, keywords, etc.**, allows structuring of code using **functions**, **loops**, **decision-making statements**, then there are complex data-structures like **structures**, **arrays**, etc., and **pointers**, which makes C quite resourceful and powerful, etc.

Using the C language you can **easily read, write and create files**. This may sound like a basic feature today, but in the early 1990s, this was a game-changer.

5. Rich Standard Library

C supports various inbuilt functions and libraries that create development fast. The **standard library support** for the C language is superb and you will see that a lot of keywords or ready-made operations that you will use while writing code in C

language are already pre-defined. We just have to use them, without worrying about how they work.

These libraries are called **Header files** in C language.

6. Libraries Support

There is not just the standard library for C language but also a **large ecosystem of 3rd party libraries**, which are created by developers/coders around the world, and anyone can use them.

Even you can write your own C language library and distribute it online through your Github profile. You never know, people around the world might need your solution.

7. Separate Compilation

C language code is compiled and then it is run. We will learn how the C language code is compiled in another tutorial. For now, just assume that the code is compiled. Obviously, the compilation of code will take some time. A small piece of code will compile faster while a large code will take time to get compiled.

In C language you can **break your code and put it in multiple source code files**. C language will compile the files separately and then link them together for execution.

This makes **compilation fast**.

Another plus point of this is, **multiple programmers/developers** can work on different code files while working on a single project.

8. Middle-Level Language

The C programming language brings together the best of both worlds.

A **Low-level language** is generally fast, powerful but hard to understand and write code in. Whereas a **High-level Language** is easy for us to understand and write code in, it is also highly portable, but it is generally slow and is unable to directly talk to the system hardware.

Hence, the C programming language is said to be a **Middle-level** programming Language, allows manipulation of bits, bytes, and addresses, hence providing low-level access to the computer systems, while being easy to use, portable, and supporting all other features of a High-level language, etc.

9. Syntax Based Language

Like most High-level languages, for example, Java, C++, C#, the **C language has a syntax**, there are **proper rules for writing the code**, and the C language strictly follows it.

If you write anything that is not allowed, you will get a compile-time error, which happens when the compiler is unable to compile your code because of some incorrect code syntax.

10. Format Free Language

The C language is a format-free language. There are no line numbers needed in the C language code, or we can say that the line number holds no significance. There is no need to place statements on a specified location on a line.

11. Compiled Language

The C language uses a Compiler to compile the code into **object code**, which is nothing but **machine code** that the computer understands. Hence to run a C language program we have to install a C language compiler first.

12. Case sensitive Language

In C, the uppercase and lowercase characters are different. That means `if` is not the same as `IF` in C language.

Frequently Asked Questions(FAQ)

Here are some frequently asked questions, that beginners have when they learn about features of the C language.

1. What are the main features of the C language?

The main features of the C language are, it is **simple(easy to learn) yet powerful**, **Portable**, **Mid-level language**, which is **structured** and needs a **Compiler** for execution. The C language also has a **rich standard library** with the support of **extensive 3rd party libraries**.

2. What is the main reason behind the popularity of the C language?

The main reason behind the popularity of the C language is that it has a rich feature set, making it a suitable language for a wide range of use cases. You can create an operating system using C language or any other simple software. The C language is a general-purpose programming language.

3. Why is C considered a Powerful Programming Language?

It is considered a powerful language because it provides a broad range of features like support for many **data types, operators, keywords, etc.**, allows structuring of code using **functions**, **loops**, **decision-making statements**, then there are complex data-structures like **structures**, **arrays**, etc., and **pointers**, which makes C quite resourceful and powerful, etc.

4. Is C language Portable?

Yes, the C language code is **portable**. If you have software written in the C language for Unix OS, and you now want to run it on Windows OS, you can easily adapt the software for Windows OS, and that is the power of the C language.

5. What are the advantages of C Language?

Given below are some of the advantages of the C language:

a) C is a Portable language.

b) It provides rich support of built-in functions.

c) It is a powerful and efficient language.

d) C language is a structured language, with features of both High-level language and Low-level language.

Conclusion

In this tutorial, we covered the features of the C language, explaining every feature with an example so that you understand what each feature means. Do not just learn the C language features but try to understand the meaning of those features.

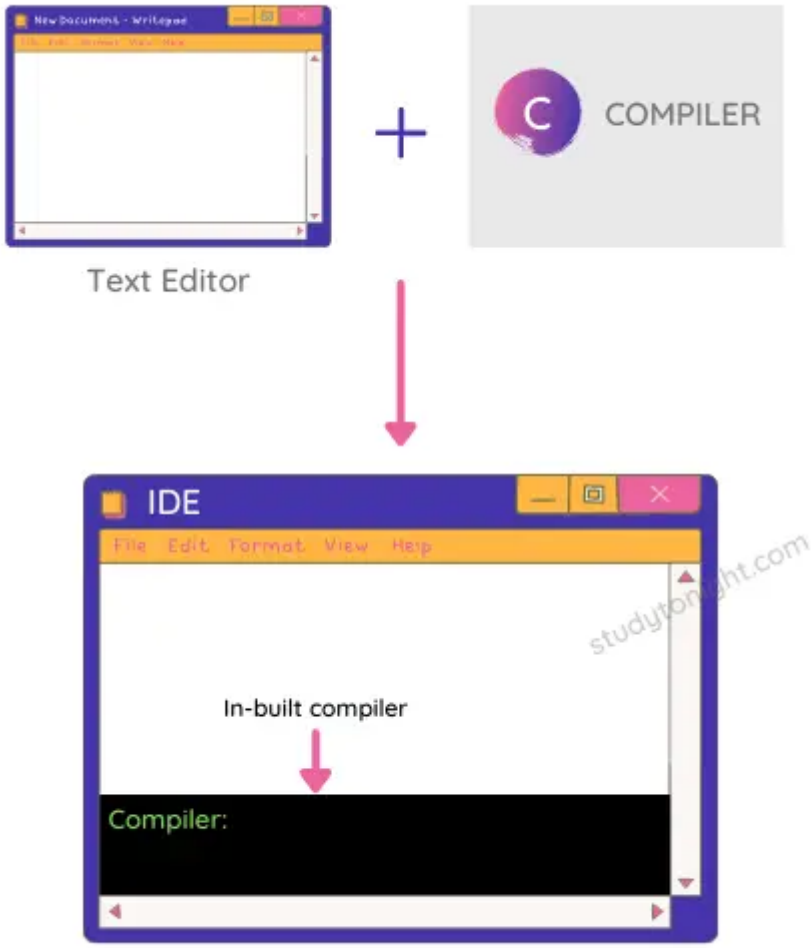
Installing C Compiler & IDE (Turbo C++)

studytonight.com/c/installing-c-compiler-ide.php

What is an IDE?

An IDE or **Integrated Development environment** is a software, which has a code editor, along with the compiler of the programming language too, hence you do not need to install anything else.

An IDE has all the required features to manage your source code, like File handling, Edit options, etc. while it also has the support for different programming language environments too, for which it has in-built compilers/interpreters installed.



Many IDEs allow installing 3rd party plugins to further enhance the features of the IDE.

Turbo C is an IDE used for writing code in the C language. And you can also, compile and run your code using Turbo C itself. **Turbo C++ is the upgraded version** of Turbo C, which can also be used for C language programming.

The C++ compiler supports the C language syntax as the C++ language is built on top of the C language.

There are other IDEs too that you can use for C programming, like:

1. Dev-C++ (Download from here: Download Dev-C++)
2. Eclipse (Download Eclipse IDE)

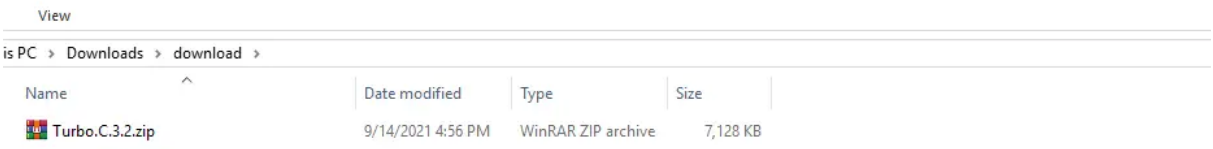
If you are a beginner, we recommend using Turbo C or Turbo C++.

Installing Turbo C/C++ for Windows 7/8/10

Let's see how we can install Turbo C for writing C programs.

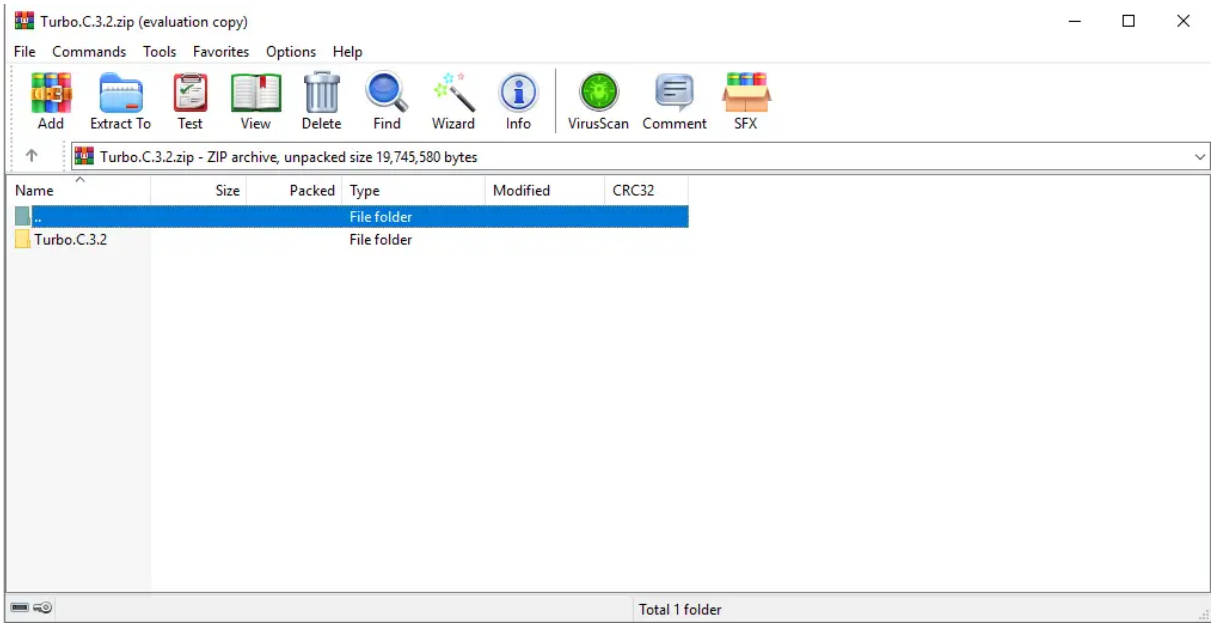
Step 1. Start by downloading the IDE installer

You can download the Zip file from this link: Download Turbo C



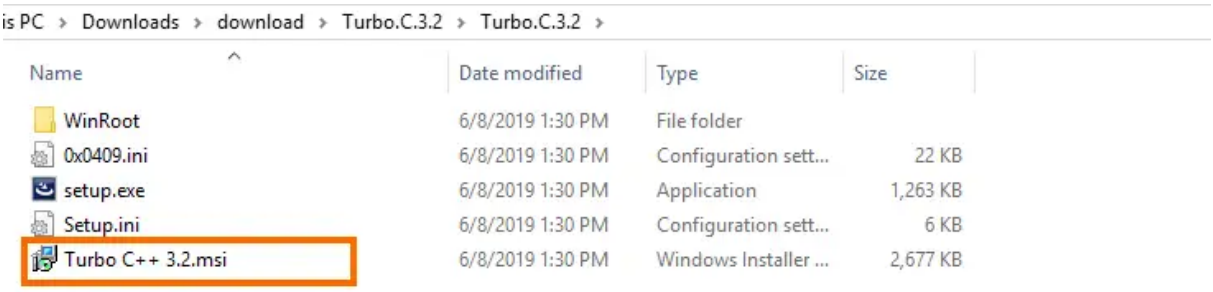
Step 2. Unzip the file to see the Installer file

Next use WinRar or any other software to see what is there in the zip file you downloaded.

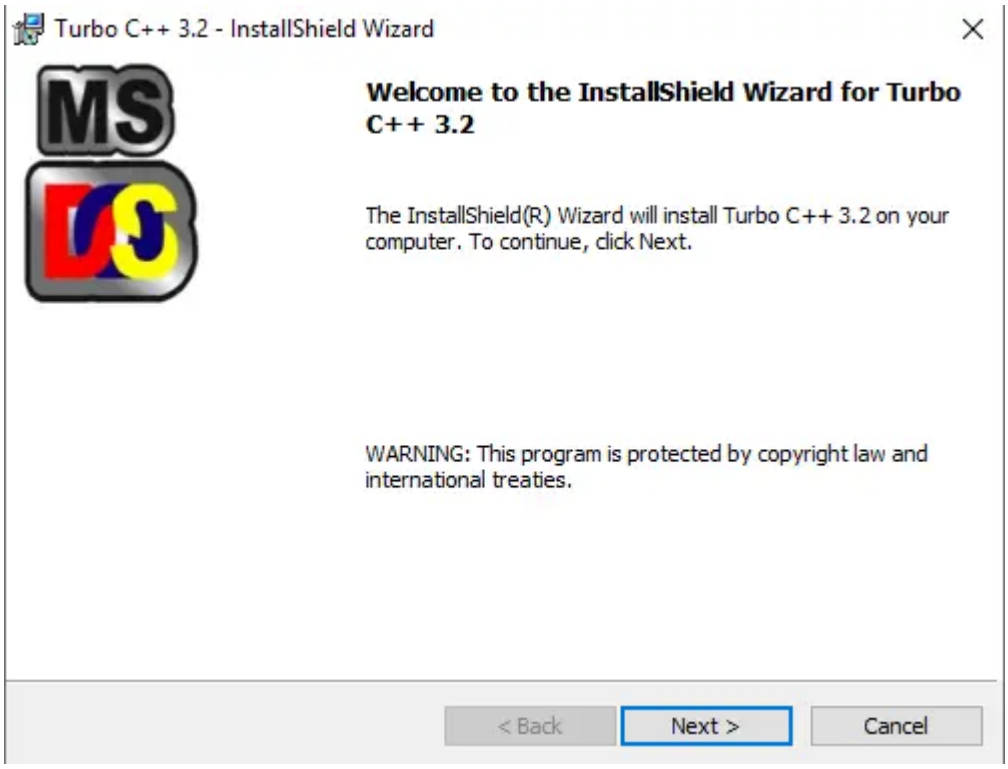


Step 3. Start the Installation

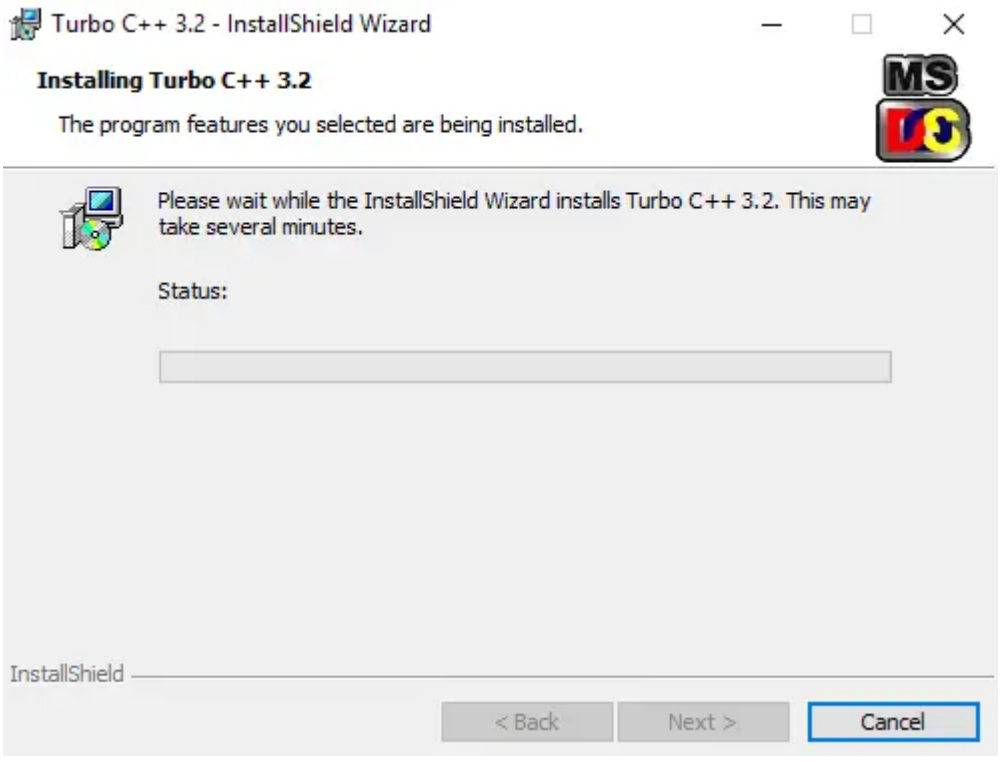
If you are a Windows OS user, click on the **.msi** file for starting the installation.



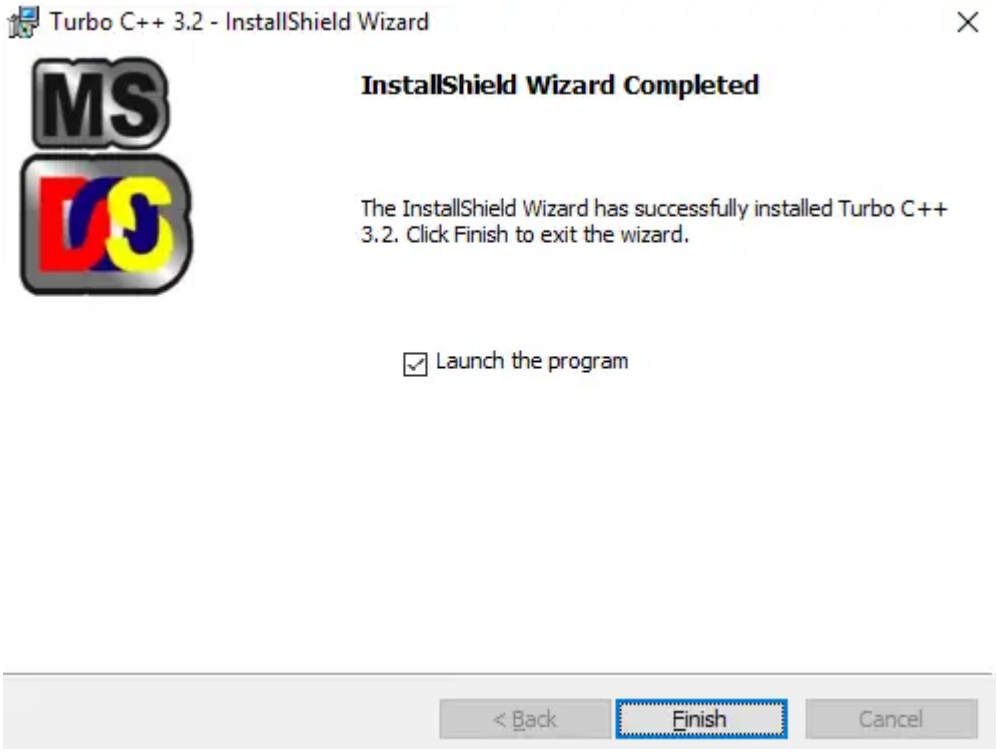
You will see the InstallShield Wizard for Turbo C/C++, click on the **Next** button.



And the installation should start. Now sit back and wait for the installation to get complete.

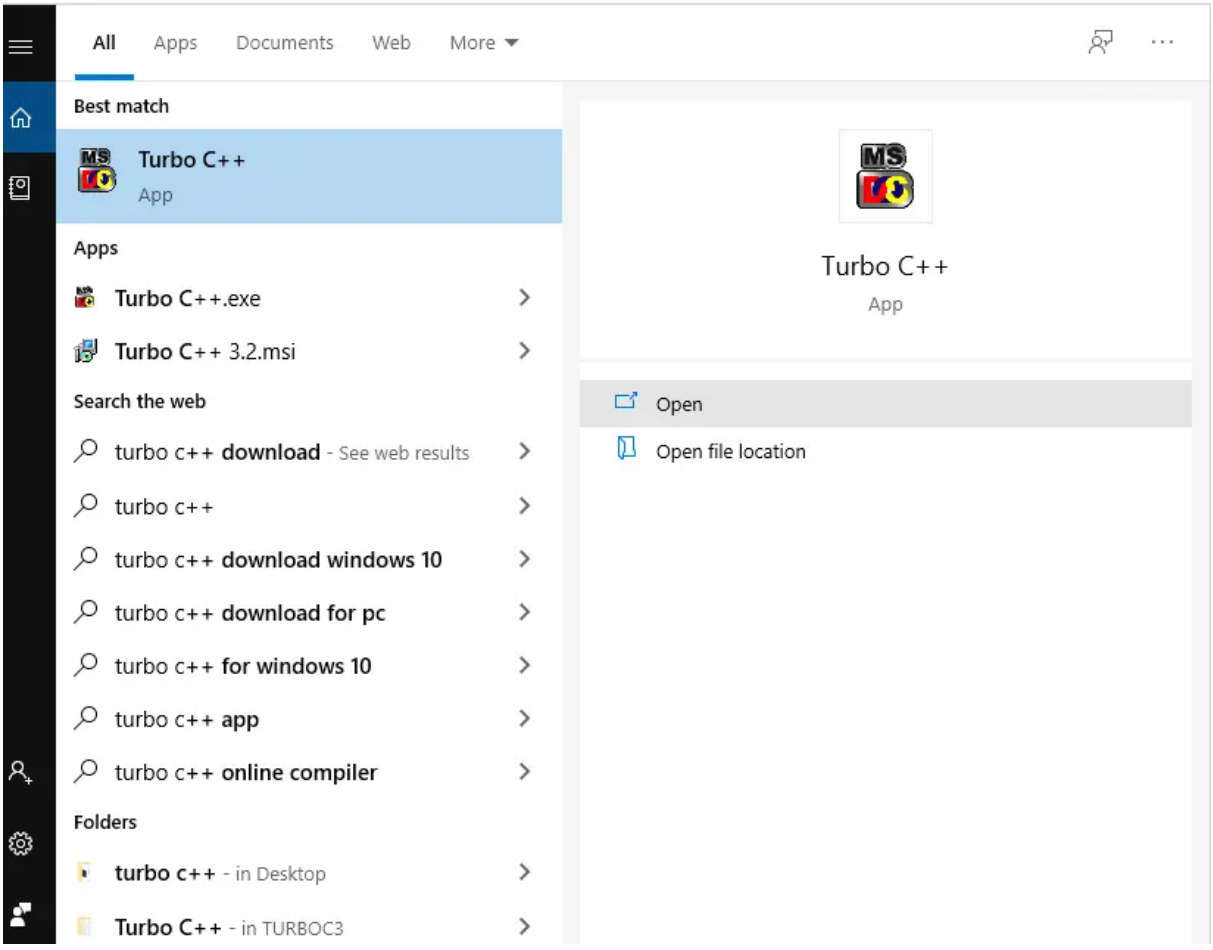


Once the installation is complete, you will see the following window, check on the **Launch the program** checkbox(if not already checked) and you click on the **Finish** to start the Turbo C/C++ IDE, to start writing the C language code.

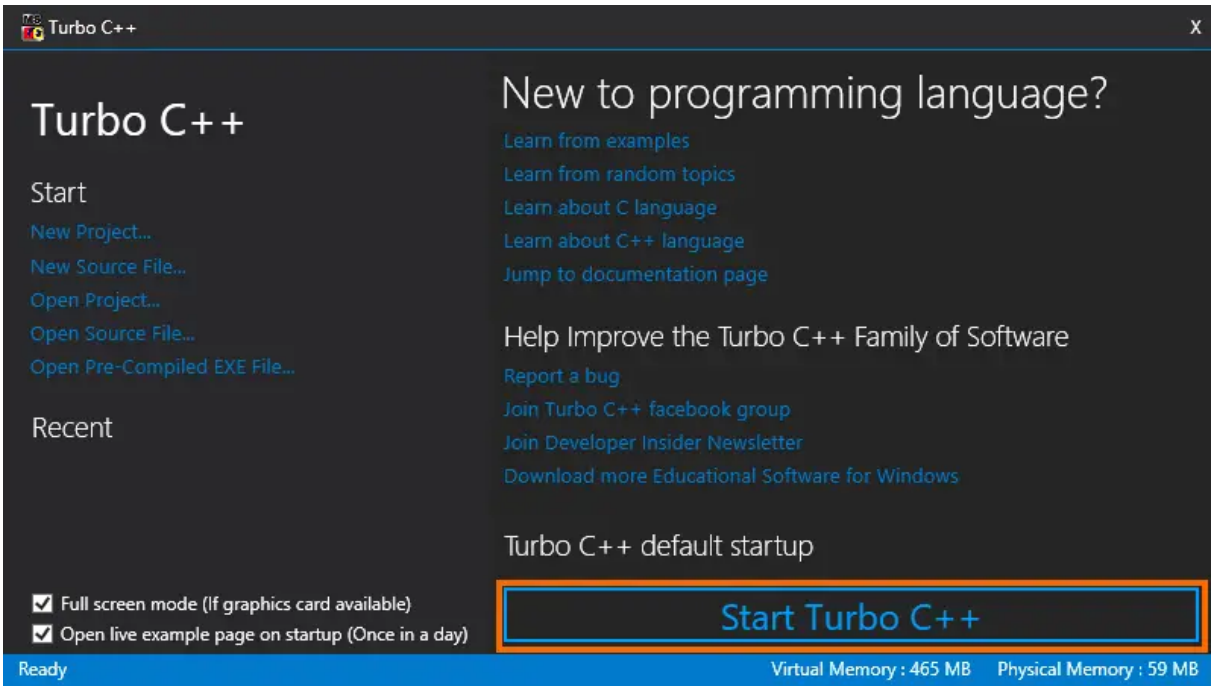


Step 4. Start Turbo C/C++ IDE

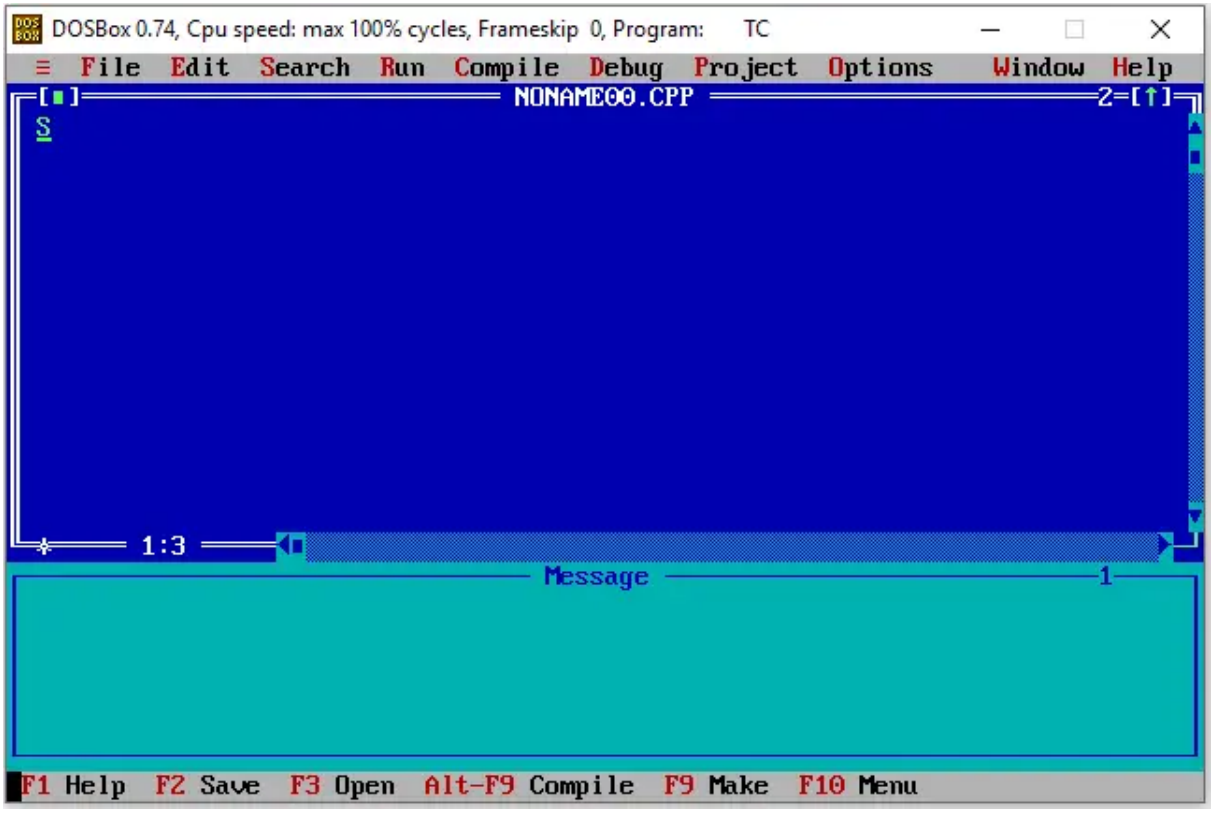
To start the IDE, you can either **create a shortcut on your desktop** to easily start the IDE every time. Or open the **Start** menu, and **search for Turbo C** and you will the application there.



Click on the **Turbo C++ app option** and you will see the following Window. Then click on the **Start Turbo C++** button to launch the IDE application.



And this is how it will look. **Bluuueee!**

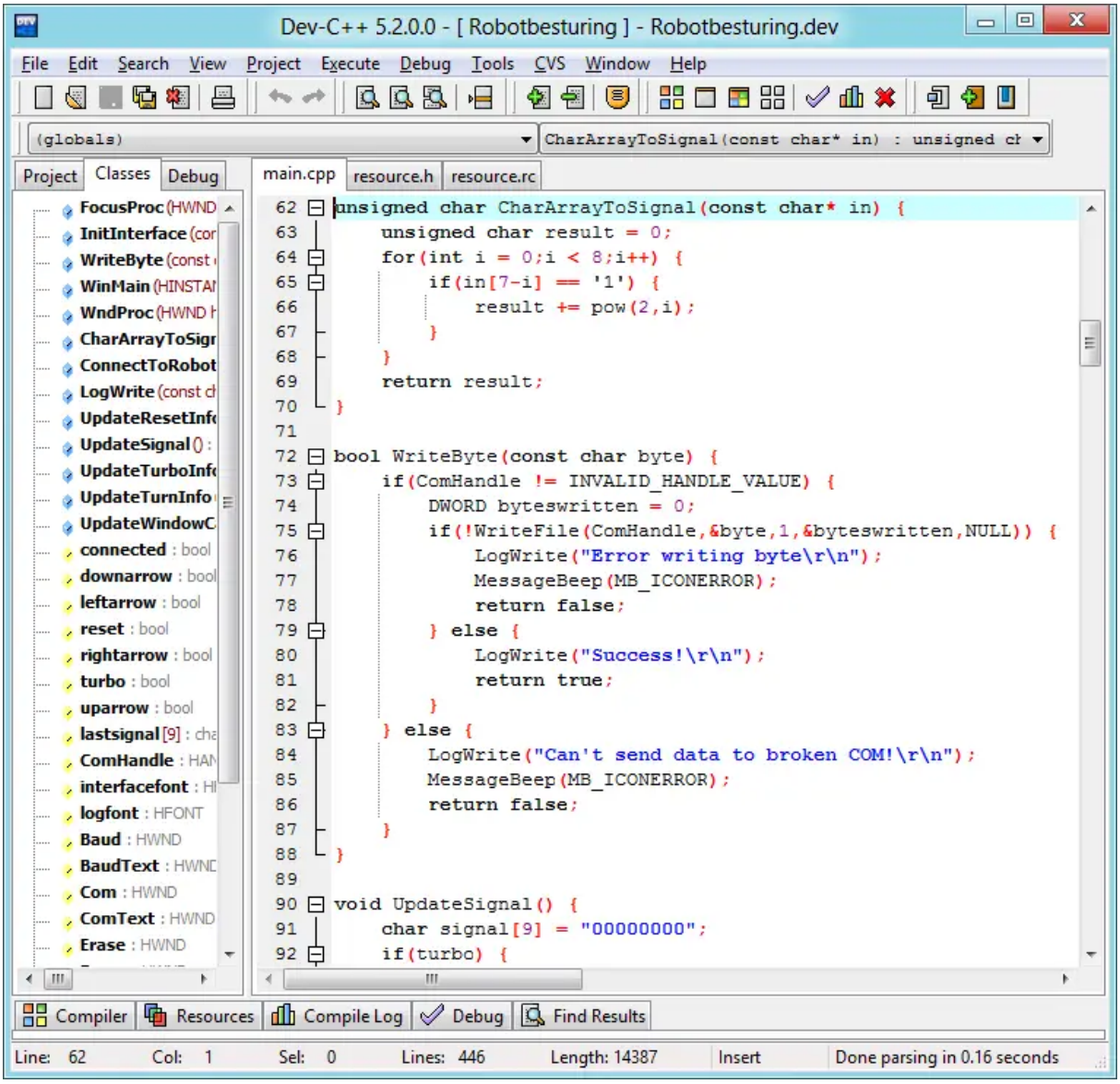


How to Compile and Run C Program in Linux

Other IDEs for C Programming

You can use the **Dev-C++** IDE or the **Eclipse IDE** for C programming. We have shared the download link for both in the introduction of IDE.

If you want a more modern C/C++ IDE for writing C programs then you should try Dev-C++ as it is not as complex as the Eclipse IDE, while has many features which the basic Turbo C/C++ IDE lacks.



Conclusion:

In this tutorial, we learned what is an IDE or Integrated development environment, which IDEs can be used for C programming and how to install the most popular C language IDE which is Turbo C/C++ step by step.

First C Program and its Structure

studytonight.com/c/first-c-program.php

In this tutorial, we will learn to create the first C program and then will understand its structure. First of all, let's have a look at how to write a **simple** and **most basic Hello World program** in C language. Let's get started.

Here is the program for printing "Hello World" in C language.

```
#include <stdio.h>

int main()
{
    printf("Hello World");
    return 0;
}
```

Hello, World

To run the above code on your local machine, you will have to install a C language compiler on your Computer/Laptop. We will learn how to do that in the next tutorial - Compile and Run C Code.

If you do not want to install the C compiler on your computer, don't worry. You can use our **Online Compiler** to run C programs and Practice. Click on the **Run Program button** above to open the compiler.

Understanding Structure of the C Program

Given below are some of the different parts of a C Program:

- Pre-processor
- Header file
- main() function
- Variables in C
- Statements & expressions in C

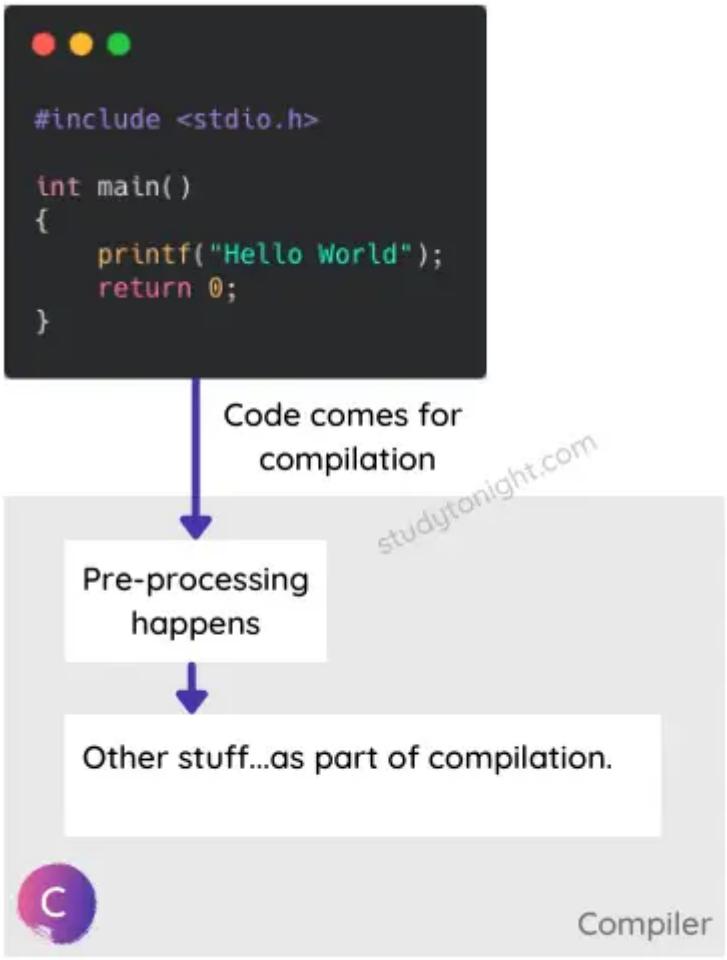
All these are essential parts of a C language program. Don't worry about all this, we will learn about everything one by one and will clear all your confusion.

Let's start with a basic introduction of various code statements that we used in the above Hello World program.

1. Pre-processor

The `#include` is the first statement of any C program. It is known as a **pre-processor**. The task of a pre-processor is to initialize the environment of the program, i.e. to link the program with the header files required.

As its name suggests, this line of code is **responsible for doing pre-processing**, before the actual code (logic) is executed.



So, when we say `#include<stdio.h>` , it is to inform the compiler to include the **stdio.h** header file which is the **standard I/O library** into the program before executing the program.

The **standard I/O library** lets you read input from the keyboard(i.e standard in) and then write the output to the console screen (i.e. standard out) and it is an extremely useful library.

By **Console screen,** we mean **CMD** or command prompt in case of Windows OS and **Terminal** in case you use Linux/Ubuntu/macOS.

Similarly, we can include any number of header files.

The `#include` is not the only pre-processor. Whenever you see any piece of code starting with a `#` symbol, that means it's a pre-processor in the C language. We will learn about pre-processors in detail later.

2. Header file

A **Header file** is a set or **collection of built-in(readymade) functions**, which we can directly use in our program.

Header files contain **definitions of the functions** which can be used in any C program by using pre-processor `#include` statement along with the name of the header file.

There are some standard header files that come along with default C installation, like **stdio.h** header file. There are many other such files, we will learn about them later.

With time, you will have a clear picture of what header files are, as of now consider them as a readymade collection of functions that comes packaged with the C language and you can use them without worrying about how they work, all you have to do is include the header file in your program.



To use any of the standard library functions, the appropriate header file must be included. This is done at the beginning of the C source code.

For example, to use the `printf()` function in a C program, which is used to **display anything** on the console screen, the line `#include <stdio.h>` is required, because the header file **stdio.h** contains the `printf()` function definition.

All header files will have **.h** extension.

3. The `main()` Function

The `main()` function is a function that **must be there in every C program**.

Everything inside this function in a C program will be executed, hence the actual logic or the code is always written inside the `main()` function.

As the name suggests, this is the main(*of prime importance or center of attraction*) function.

```
#include <stdio.h>

int main()
{
    printf("Hello World");
    return 0;
}
```

In the Hello World code example above, there was `int` written before the `main()` function, remember? Well, that is the **return type** of the `main()` function. we will discuss it in detail later.

The curly braces `{ }` just after the `main()` function enclose the **body** of the `main()` function.

We will learn what functions in C language are in upcoming tutorials.

4. The `printf()` Function

The `printf()` is a function that is used to print(show) anything on the console as output. This function is defined in the **stdio.h** header file, which we have included in our C program.

We will cover how to take input and show output in one of the next tutorials.

5. Return Statement

A `return` statement is used to return a response to the caller function. It is generally the last statement of any C language function. Do not worry about this too, we will cover this statement when we learn about functions in the C language.

6. Semicolon

It is important to note that every statement in C should end with a **semicolon(;)**. If you miss adding any semicolon, the compiler will give an error.

How to write a C program on my Computer?

All the C programs can be written and edited in normal text editors like **Notepad** or **Notepad++** and must be saved with a file name and **.c** extension. For example, **helloworld.c** can be the file name for the Hello World program.

If you do not add the extension **.c** then the compiler will not recognize it as a C language program file.

Frequently Asked Questions (FAQ)

1. Name different preprocessors in C programming Language?

Different preprocessors in the C language are `#include`, `#if`, `#define`, `#ifdef`, `#undef`, etc. and many more. A pre-processor is a statement that is executed before the main code for the C program is executed.

2. What do you understand by the main() function in C language?

The `main()` function in the C language acts as the entry point of any program or we can say that the execution of the code starts from here. The source code which is inside the `main()` function gets executed.

3. How comments can be added to a C program?

In order to add the single line comment, you can use slash `//` followed by the comment and for multiline comments, you can use the following: /* **comment here** */.

4. What is stdio.h in C program?

The **stdio** stands for **standard input and output** and **.h** is the extension of the file indicating that it is a **header file**.

The main use of this header file is, that it helps to get the input from the user(Keyboard) and return the output result text to the monitor(screen). There are readymade functions in this header file that we can use in our program, for example, `printf()` and `scanf()` functions are defined in the **stdio.h** header file.

Conclusion

In this tutorial, we saw a simple Hello World program in the C language and covered its structure. In the next tutorial, we will set up the C compiler on our local computer/laptop so that you can run this program on your computer too.

Compile and Run C Program

studytonight.com/c/compile-and-run-c-program.php

To compile and run a C language program, you need a C compiler. A **compiler** is a software that is used to compile and execute programs. To set up a C language compiler in your Computer/laptop, there are two ways:

1. Download a full-fledged IDE like Turbo C++ or Microsoft Visual C++ or DevC++, which comes along with a C language compiler.

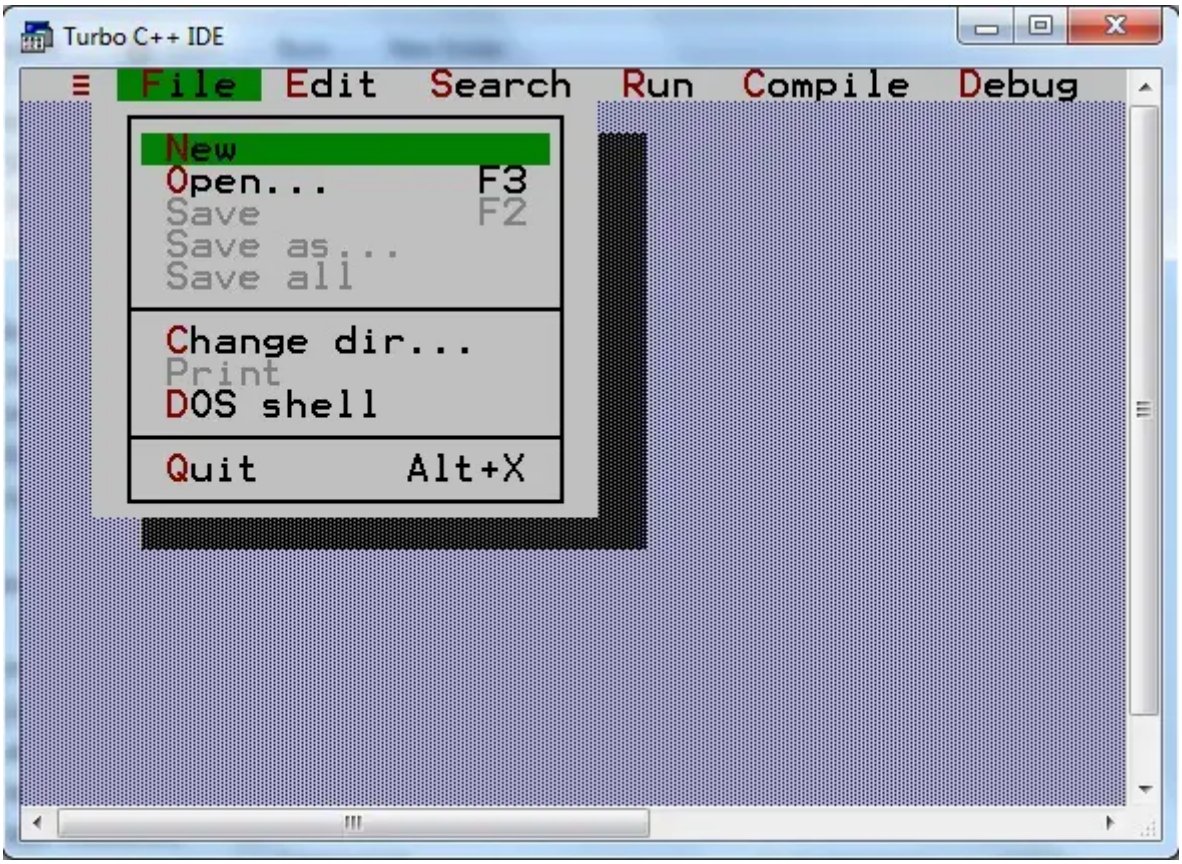
2. Or, you can use any text editor to edit the program files and download the C compiler separately and then run the C program using the command line.

If you haven't already installed an IDE for the C language - Follow this step-by-step guide to **Install Turbo C++ for C Language**

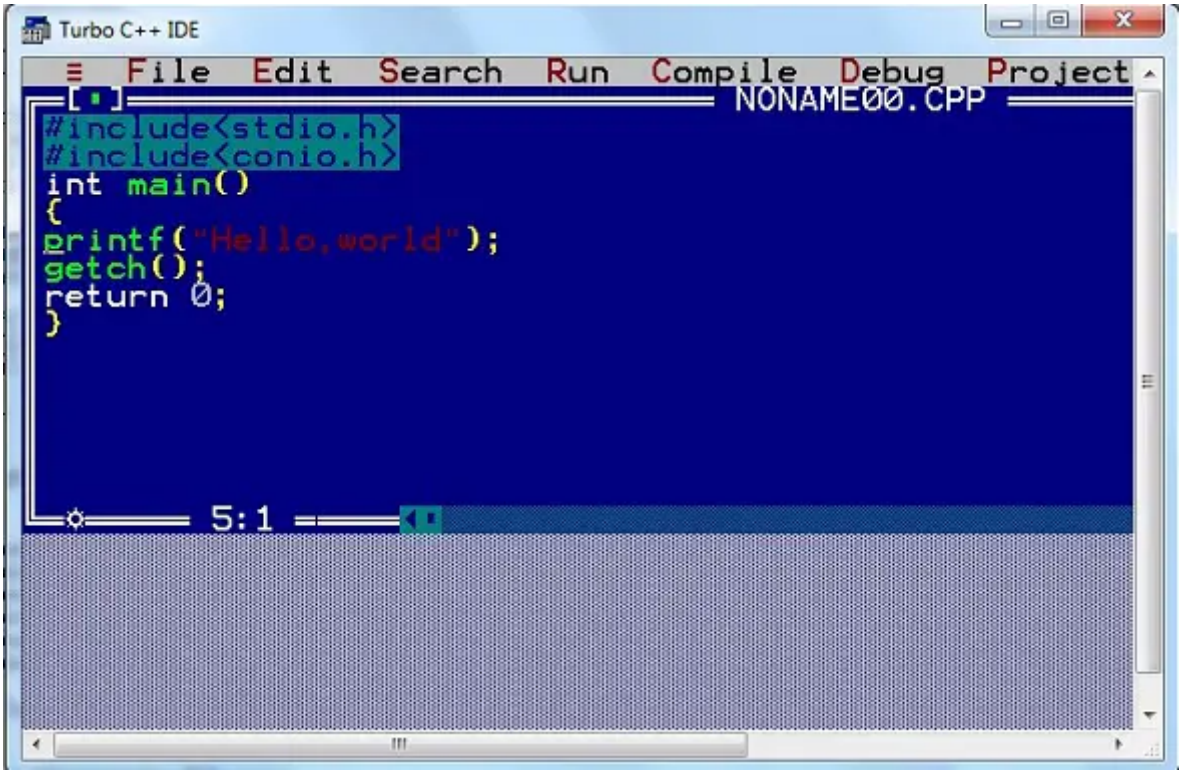
Using an IDE - Turbo C

We will recommend you to use **Turbo C or Turbo C++** IDE, which is the oldest IDE for C programming. It is freely available over the internet and is good for a beginner.

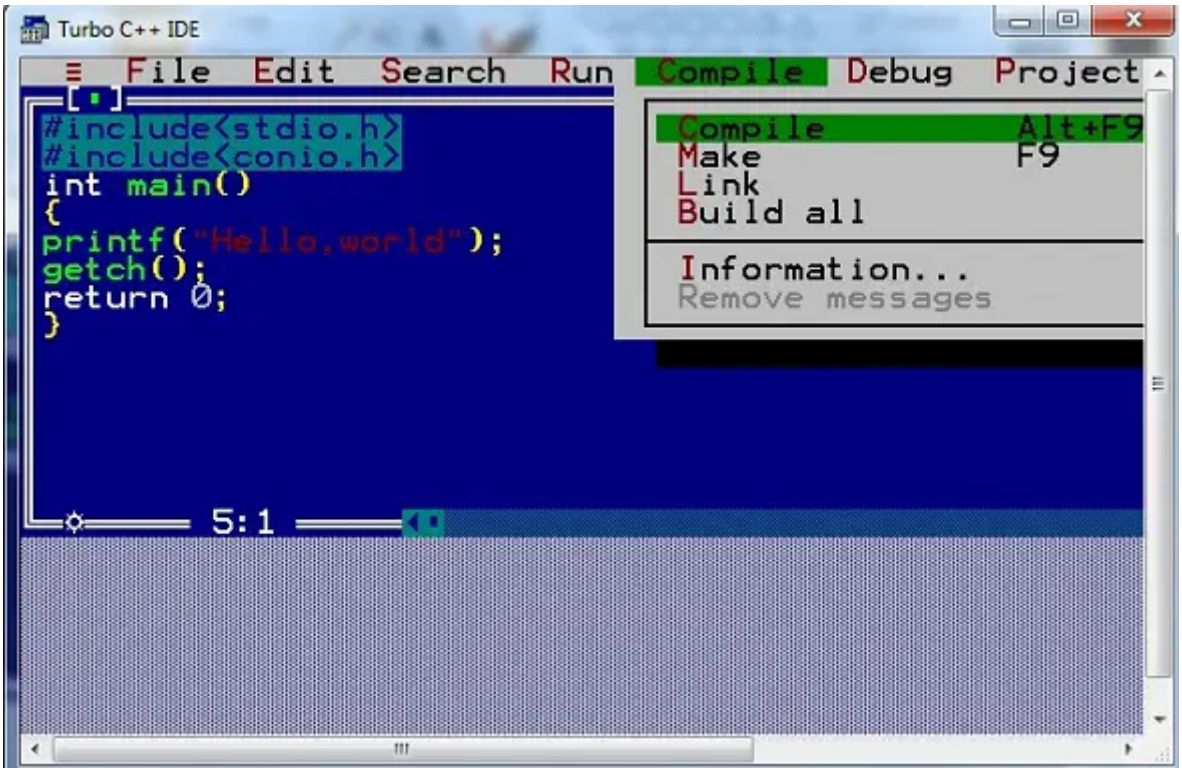
Step 1: Open turbo C IDE(Integrated Development Environment), click on **File** and then click on **New**



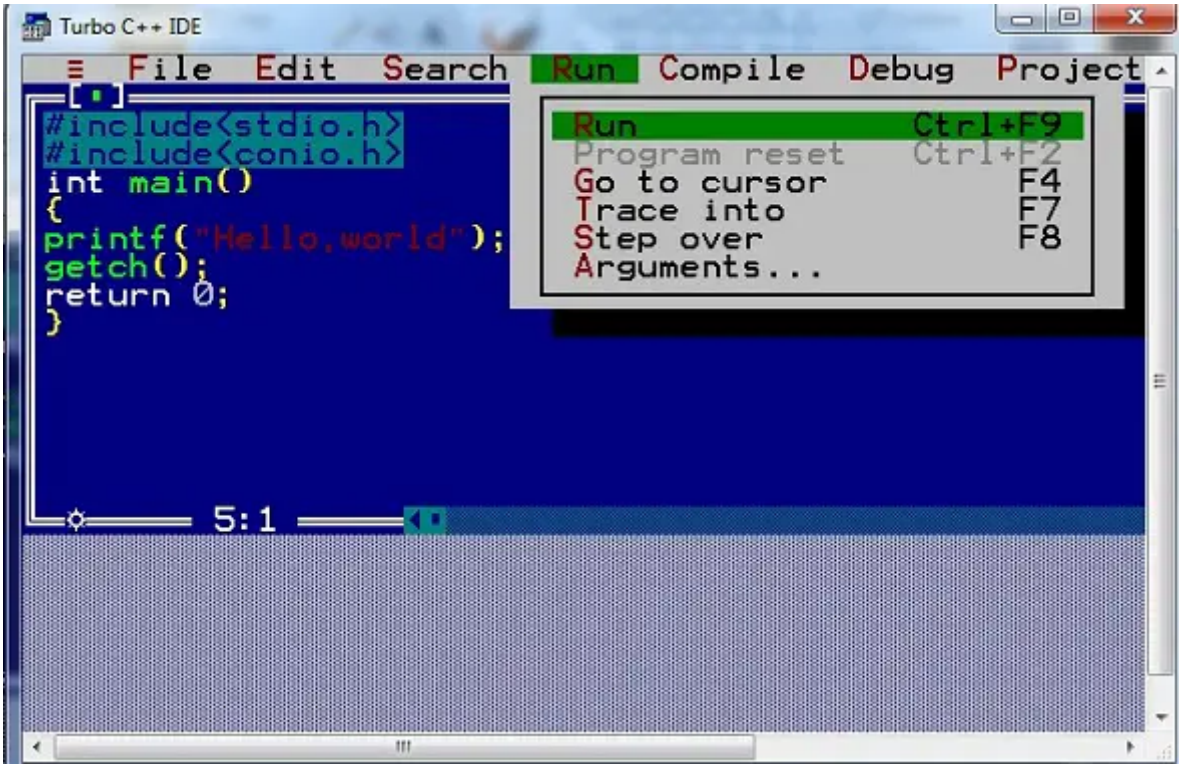
Step 2: Write a Hello World program that we created in the previous article - **C Hello World program**.



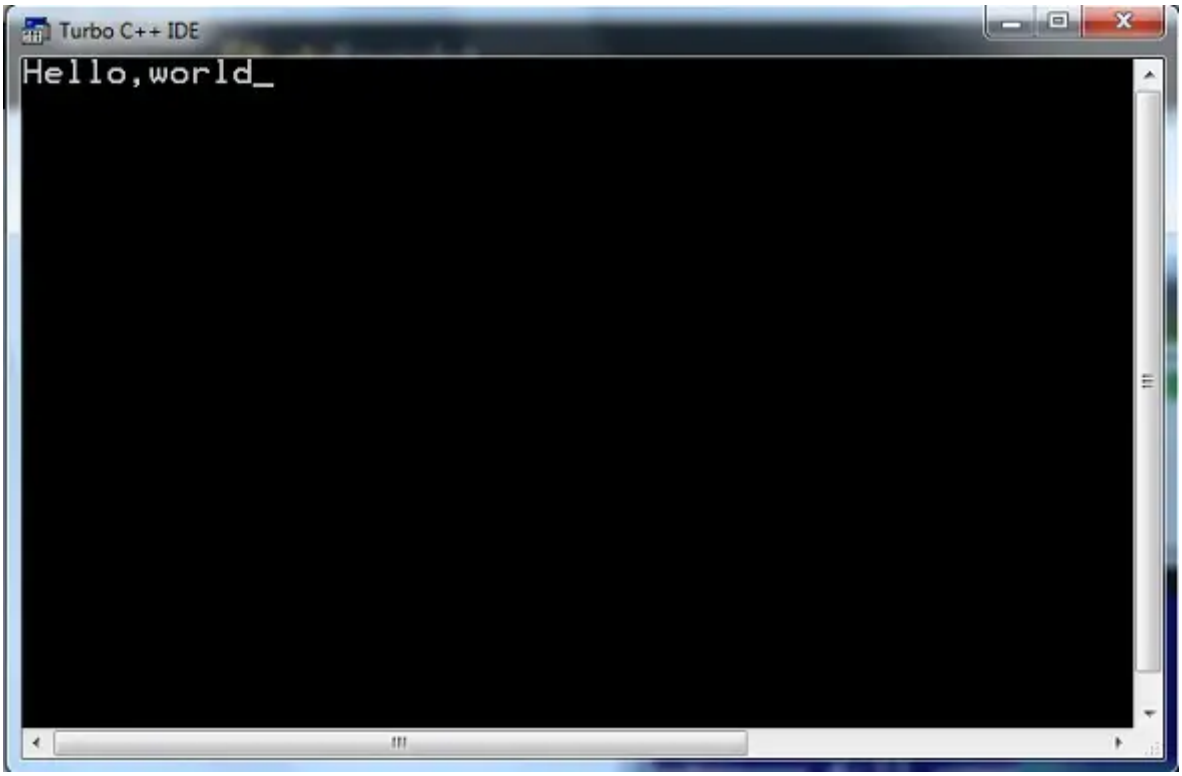
Step 3: Click on **Compile** menu and then on **Compile** option, or press the keys press **Alt + F9** to compile the code.



Step 4: Click on **Run** or press **Ctrl + F9** to run the code. Yes, C programs are first compiled to generate the object code and then that object code is Run.



Step 5: Output is here.



Run C Program Without using any IDE

If you do not wish to set up an IDE and prefer the old school way, then download the C compiler which is called `gcc` from the GCC website https://gcc.gnu.org/install/

Once you have downloaded and installed the `gcc` compiler, all you have to do is, **open any text editor**, copy and paste the C program code for C Hello World Program, and save it with the name the **helloworld.c** like any other file you save with a name.

Now, Open **Command prompt or Terminal**(if you use Ubuntu or Mac OS), and go to the directory where you have saved the **helloworld.c** program file.

Type the command `gcc hello.c` to compile the code. This will compile the code, and if there are no errors then it will produce an output file with name **a.out**(default name)

Now, to run the program, type in `./a.out` and you will see **Hello, World** displayed on your screen.

```
$ gcc hello.c
$ ./a.out
```

Hello, World

Difference between Compile and Run in C?

You must be thinking about why it is a 2 step process, first, we compile the code and then we run the code. We did the same thing with Turbo C and the same is with the command line or Terminal too.

Well, the **compilation** is the process where the compiler checks whether the **program is correct syntax-wise**, and there are no errors in the syntax, and if the code is fine converts the C language source code into machine understandable object code.

When we **run a compiled program**, it is just the already compiled code that is run.

This difference is clear when we run a C program using the command line. When you compile the code, a **.out** file is generated, which is then Run to execute the program.

Once a **.out** file is generated, and then you make any **changes to your program** in the source code file, you will have to **again compile the code**, otherwise, the **.out file will have the old source code** and will keep running the old program itself.

Frequently Asked Questions(FAQ)

1. What do you understand by the process of compilation?

It is a process where the compiler checks whether the program is syntactically correct or not. If there is any mistake found in the syntax then it will throw the errors. The error thrown at the time of compilation is called a **Compile-time error**.

2. Name the keys used in Turbo C to compile the code.

ALT + F9 is used in turbo C in order to compile the code.

3. How is a C program run?

During the compilation and execution of the C program, the compiler generates output files with the same name as that of the **C program** file but with different extensions. The .c extension file is called the source file that keeps the code of the **program**. Now, when we compile the file, then the **C** compiler looks for errors.

4. Where can I write and run the C program?

You can use an IDE to write and run the C program by following these steps:

- Step 1: Open turbo **C** IDE(Integrated Development Environment), click on **File** and then click on **New**.
- Step 2: **Write** the C program code.
- Step 3: Click on **Compile** or press **Alt + F9** to **compile** the code.

- Step 4: Click on **Run** or press **Ctrl + F9** to **run** the code.
- Step 5: And turbo C will open the console to show you the output of the program.

Conclusion

This tutorial was all about how to run and compile any C program using an IDE name Turbo C . We have also covered how it can be done without using any IDE.

C Source Code Compilation Process

studytonight.com/c/c-compilation-process.php

The C source code compilation process is a multi-step process, which involves **pre-processing**, **compiling of code**, **linking of libraries**, etc. In this tutorial, we will learn how the C code is compiled into object code.

The process of converting the source code written in any **programming language**(generally, mid-level or high-level language) into **machine-level language** that is understandable by the computer is known as **Compilation**. The software that is used for this conversion is known as a **Compiler**.

In C language, through compilation, the C language **source code** is converted into **object code**.


C language Source code to object code

The compiler takes the **input** which is the **source code** and provides the **output** in the form of **object code**. The complete process of compilation in the C language is further divided into four phases:

1. **Pre-processing**,
2. **Compiling**,
3. **Assembling**, and
4. **Linking**

The compiler checks the source code for any **syntax** or **structural errors** and after checking, if the source code is found error-free, it then generates the object code that has an extension **.obj** (for Windows) or **.o** (for Linux).

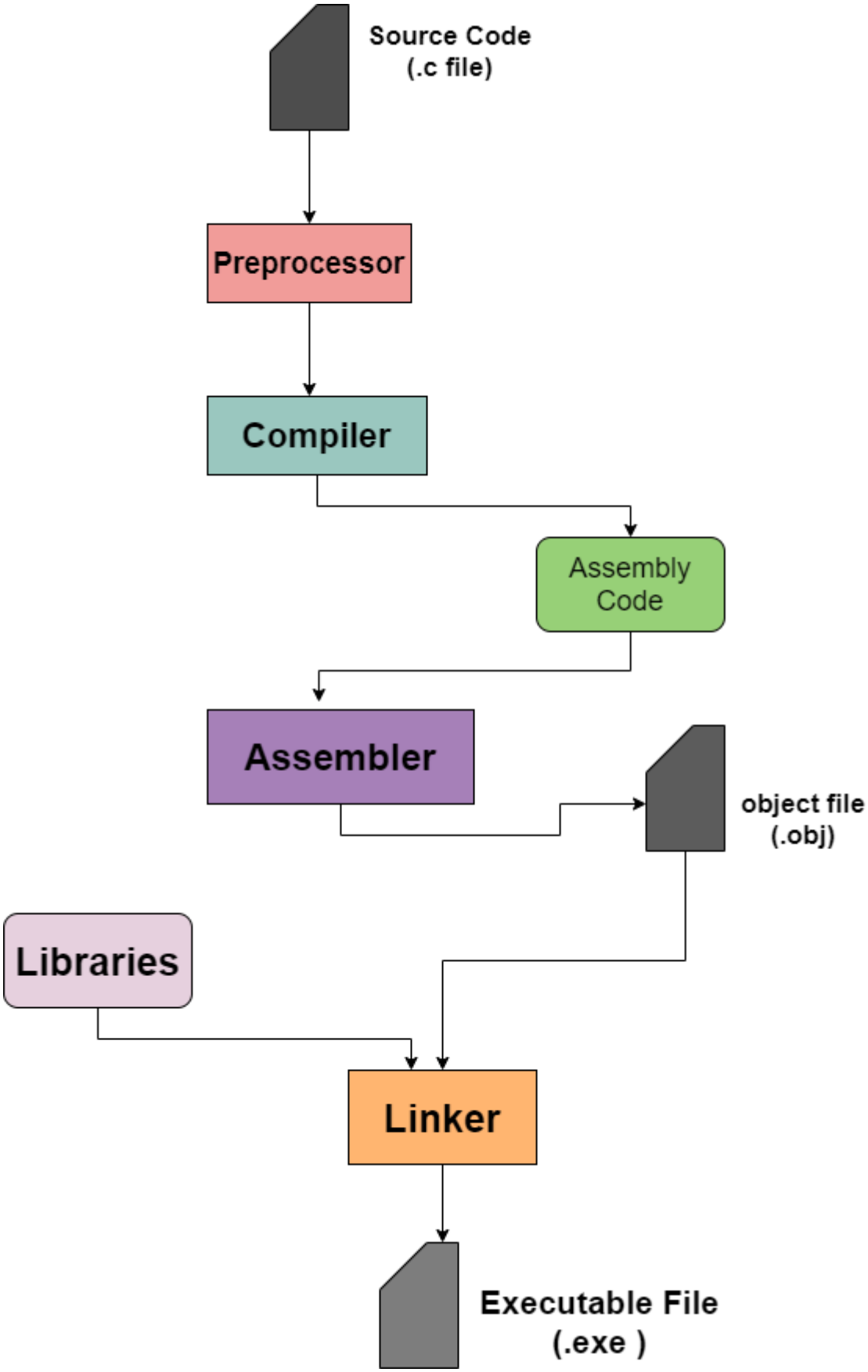
Let us now take a look at the different stages of the compilation process in the C language.

Compilation Process

Like mentioned above, the different stages of the compilation process are as follows:

- Pre-processing
- Compiling
- Assembling
- Linking

In the flow chart below we have explained how the compilation process work and what are the different stages of compiling the C language source code.



Let's discuss all these stages of the C language source code compilation in the order they are performed.

Step 0: Pre-processing of the source file

In this phase, pre-processing of the source file is done. The **Pre-processor** is a program that accepts the C source code file and then it performs the following tasks:

- It will remove the comments from the source code.
- It will perform the **Macro expansion** if any Macro is used (Do not worry about Macros, we will learn about them later)
- It will perform the expansion of the included header files.

Step 1: Preprocessor

It is a program that processes the source program before passing them on to the compiler. At this step the pre-processors used in any C program are handled and the source code is made ready for compilation.

- Each preprocessing statement must start with `#`, where `#` is called the **preprocessor directive**.
- Each preprocessing directive is a single-line code statement.
- The word after `#` is called the **preprocessor command**.

Some of the preprocessor directives are as follows:

1. #include

To include a particular header using the name of the header file into the C language program code.

2. #define

This is used to define a MACRO in the C language.

3. #error

This preprocessor command is used to print the error message.

Just like the above three, there are many other preprocessors, we will cover them in detail in a separate tutorial.

Hence, the preprocessor expands the source code(adds the required information) and then this expanded source code is passed on to the compiler.

It gives the (**.i**) extension to the **source code file** which is initially with (**.c**) extension.

Step 2: Compiler

The expanded code by the preprocessor is then passed on to the compiler. As we know a compiler is a program that converts the **high-level language**(or **mid-level language**) code to the **assembly code**, which is then converted into the machine code, which the machine can understand.

Therefore the preprocessed code given by the preprocessor to the compiler is then converted into assembly code by the compiler, which is then passed on to the **Assembler**.

The source file which got the (**.i**) extension in the previous step gets converted into (**.s**) extension by the compiler.

Step 3: Assembler

The assembler converts the assembly code that it gets from the compiler into the **object code**. The extension of the file in this step becomes (**.obj**).

Don't think that Assembler is a separate program generating the object code. The Assembler is part of the compilation process of the C language source code.

When in laymen language, we say, the C code is compiled, it means the complete compilation process, covering all these steps, is done.

Step 4: Linker

A linker is a tool that is used to **link all the parts of a program together** in order of execution. The code after this stage becomes **Executable Machine code**.

There might be some situations when our program refers to the functions that are defined in some other files. Or, if the code for some program is too big, we can break it into two files, which will be compiled separately and then linked using the Linker.

In the C language compilation process, the Linker plays a very important role.

If your C program includes a **header file**, and you are using some function defined in that header file, then the Linker will **link the required object code** for the function in the library, to the **object code of your program** and package them together.

Similarly, if your program code is too big and you break it into two files, then both the source code files will be converted into object code separately and then the Linker will link them and make the code ready for execution. This is also called **Separate Compilation**.

Frequently Asked Questions

Here are some frequently asked questions related to the C language compilation process.

Q1. What do you understand by the term Compiler?

It is a computer program that is used to translate the source code that is in a high-level programming language or a mid-level programming language into machine code, which can be executed and the machine can understand what to do.

Q2. What is a preprocessor?

The preprocessor is a directive used to instruct the compiler to preprocess the source code before the start of the actual compilation. There are some parts of the C language program that can be specified using the preprocessor, which is nothing but statements to be executed at the beginning of compilation.

Q3. What is the other name of the .c file?

The C language code files which have extension **.c** are also called the **source code file**.

Q4. What is the role of Linker?

The linker is used to link all the parts of the program together in order for the execution.

If your C program includes a **header file**, and you are using some function defined in that header file, then the Linker will **link the required object code** for the function in the library, to the **object code of your program** and package them together.

Similarly, if your program code is too big and you break it into two files, then both the source code files will be converted into object code separately and then the Linker will link them and make the code ready for execution. This is also called **Separate Compilation**.

Q5. What are the steps of compilation in the C Language?

Following are the steps of compilation in the C language:

1. Preprocessor (preprocessing)
2. Compiler (compiling)
3. Assembler (assembly)
4. Linker (linking)

Conclusion

In this tutorial, we learned how the C language code that we will be writing in the upcoming tutorials is compiled. What are the various steps followed in compilation and what happens in those steps?

If you are left confused after this, do not worry, just read it and as we move on with the tutorial, you will understand everything.

C Language Basic Syntax Rules

studytonight.com/c/c-syntax.php

The C language syntax specifies the rules for writing the code in the C language. In simple words, these rules inform how to form statements in a C language program - How should the line of code start, how it should end, where to use double quotes, where to use curly brackets, where to use parenthesis, etc.

What is Syntax?

Any language, be it English or Hindi, or Spanish, has a **grammar** that defines the **rules for using the language**, for example *how to form a sentence*, what different *words mean*, etc.

In a normal spoken language or a computer programming language, syntax means how to arrange words, characters, special characters, to make a meaningful statement, or expression, etc.

If someone says there is a **syntax error** in the program, means you have not written the program correctly, you might have missed some semicolon or some other general mistake in typing the code for the program.

Having a **syntax error** doesn't mean your code's logic is incorrect, it means you have written it incorrectly. Once the **syntax is correct**, then only the **code is compiled and then run**.

The C Tokens

The smallest individual unit in the C program is known as **C Token**. Tokens are either **keywords or identifiers**, **constants**, **variables**, or any other symbol which has some meaning in C language. The C program can also be called a **collection of various tokens**.

Hence the syntax for C language defines how to use these tokens together while writing the C language code.

Let's take an example,

```
#include <stdio.h>
int main()
{
    printf("Hello,World");
    return 0;
}
```

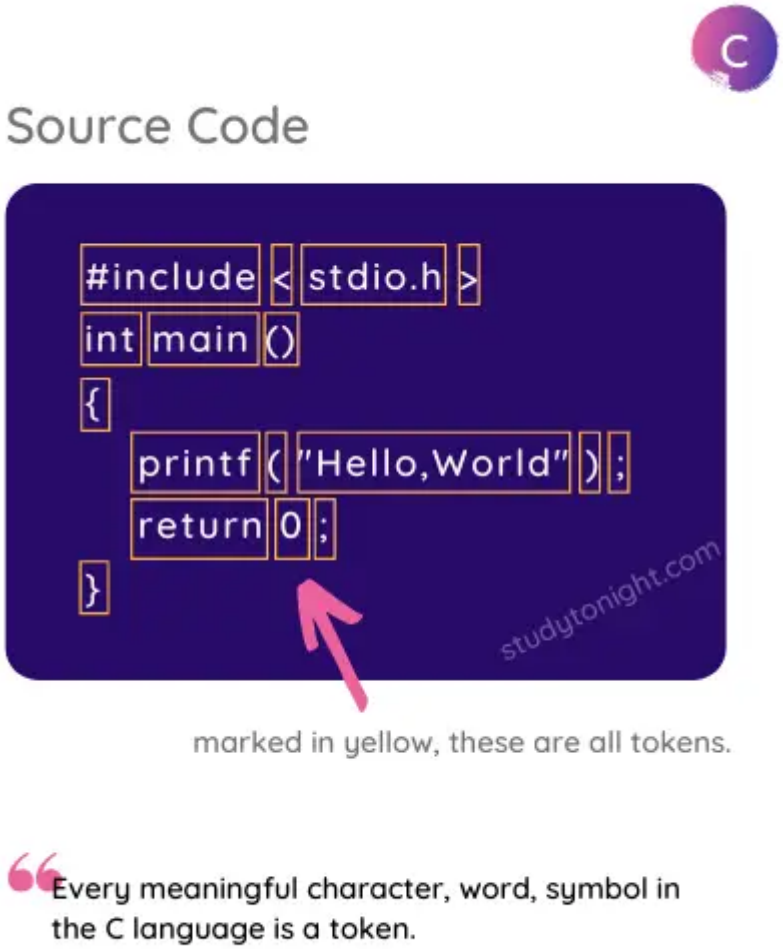
In the above code, if we take the `printf` statement:

```
printf("Hello,World");
```

Then the tokens in this statement are: `printf` , `(` , `"Hello,World"` , `)` and `;` .

So the C tokens are basically the building blocks of a C program.

The picture below shows token in the entire Hello World Program.



Now let's see some important syntax rules, which you must remember always while writing code in the C language.

C Program to Reverse Characters of String using Pointer | Pointer in C

Statement ends with Semicolon (`;`)

A semicolon `;` is used to mark the end of a statement and the beginning of another statement in the C language.

The absence of a semicolon at the end of any statement will mislead the compiler to think that this statement is not yet finished and it will add the next consecutive statement after it, which may lead to a **compilation(syntax) error**.

```
#include <stdio.h>
int main()
{
    printf("Hello,World")
    return 0;
}
```

error: expected ';' before 'return'
return 0;
^
command terminated with exit code 1

In the above program, we have omitted the semicolon from the `printf("...")` statement, hence the compiler will think that starting from `printf` up untill the semicolon after `return 0` statement is a single statement and this will lead to compilation error.

Adding Comments to Code

Comments are plain simple text in a C program that is not compiled in the compilation process.

We can write comments in a program, to explain various parts of the program. Although writing comments is not compulsory, but it is recommended to make your program more descriptive, and easier for others to understand.

Adding comments makes the code more readable.

There are **two ways** in which we can write comments.

1. Using `//` : This is used to write a **single-line comment**.
2. Using `/* */` : Anything enclosed within `/*` and `*/` , will treated as **multi-line comments**.

You can even comment out code using these two techniques and that statement or part of code will not get compiled and executed.

Example of comments in C language:

Here is a simple program to show how to use comments:

```
/*
    This is my first program.
    I am very excited!
*/

#include <stdio.h>
int main()
{
    // Printing Hello World
    printf("Hello,World");
    // printf("Useless piece of code.");
    return 0;
}
```

As you can see in the above code, we have added comments, and have also, commented a `printf()` statement, which will not get executed.

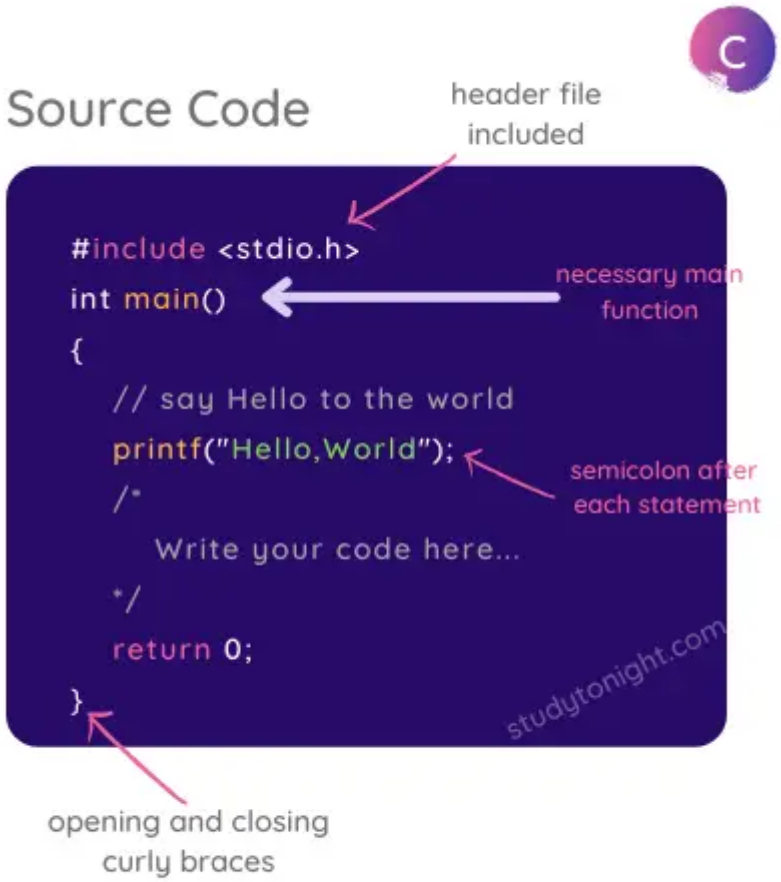
More Syntax rules for the C Language

This is just the beginning, as we will learn more concepts in the C language, we will learn the syntax for using all of them along with examples.

Everything has a way of writing and use when it comes to writing code, and that way is its syntax.

- C is a **case-sensitive language** so all C instructions must be written in lower case letters. **main** is not the same as **MAIN**.
- All C statements must end with a **semicolon**.
- **Whitespace** is used in C to add blank space and tabs.
- You do not have to worry about the indentation of the code.
- When we write a function, its body is enclosed in **curly braces**, like for the `main()` function. We will learn this in detail when we cover functions.

The picture below shows you the structure of the C program.



Frequently Asked Questions (FAQ)

Here are some frequently asked questions related to the C language syntax.

Q1. What do you understand by tokens in C language?

The smallest individual unit in the C program is known as **C Token**. Tokens are either **keywords or identifiers**, **constants**, **variables**, or any other symbol which has some meaning in C language. The C program can also be called a **collection of various tokens**.

Q2. What will happen if we forgot to use a semicolon at the end of the statement in C?

If we forgot to put the **semicolon at the end** of any **statement** in the C language then it will lead to a syntax error. In that case, an error message is issued by the compiler that **a semicolon is expected**.

Q3. What is a Compilation Error?

A compilation error, or compile-time error, is the error returned by the compiler if the syntax of the C program is incorrect. When you try to compile a program with some syntax mistake, then the compiler will give an error, which is called a **Compile-time error**.

Q4. How to add comments in C?

We can add single-line comments using the `//` at the starting of the comment text, and multi-line comment by enclosing the text within `/*` and `*/` in the C program.

Q5. Can a program run without the `main()` function in C?

In the C language, the `main()` function defines the starting point of execution. If a C program doesn't have a `main()` function then no code statement will be executed in that program. In most cases the compiler gives an error, if it cannot find the `main()` function.

Conclusion

This tutorial was all about basic syntax rules in the C language. We have covered the importance of semicolons, tokens in C, and how to write a single line and multiline comments in C.

C Keywords and Identifiers

studytonight.com/c/keywords-and-identifier.php

Keywords and Identifiers in the C language are the building block of any program. **Keywords are predefined**, which means the C language has a list of words that are Keywords, while an **Identifier is user-defined**, which means you while writing the C language program can specify identifiers. Let's see what these two are.

What are Keywords?

Every programming language has some reserved words that are used internally and have some meaning for the language, such words are called **Reserved Keywords** or just **Keywords**.

Taking an example from real-world, when someone says, **Do you have my Laptop?** You immediately picture a Laptop computer, what if the person asking for his/her Laptop, has a **dog named Laptop**(and is asking about the Dog), you won't understand until and unless you know that he/she has a dog named Laptop. This can lead to confusion, right.

Hence, the words that any programming language like the C language uses internally for some specific purpose, are not allowed to be used other than that purpose, otherwise, the language compiler will get confused.

In the last tutorial, where we learned **C language syntax**, we learned that everything in the C program is a **Token**.

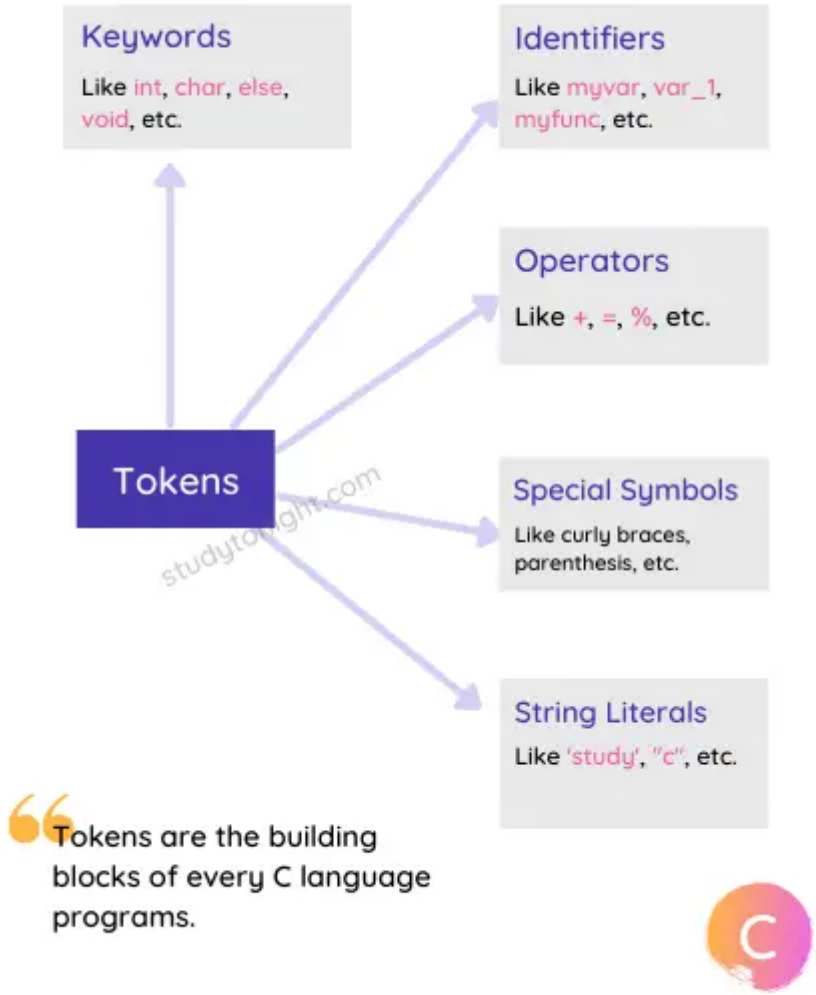
Hence we can say that the words or **Keywords are also tokens**.

What are Identifiers?

Identifiers are words or texts used to identify anything in the C language.

Just like you have a name, using which everyone calls you, it can be John, Ron, Scarlett, Monica, etc., similarly in the C language, when we define a variable or a function, or a structure, etc. we name them so that it becomes easier for us to identify them and use them whenever required. These names are called **Identifiers**.

Identifiers are also tokens. Again, because every meaningful symbol, word, etc. in the C language are C Tokens, hence Identifiers are also tokens.



C Keywords

Now we know, that keywords are reserved words that have special meaning in the C language.

The meaning of C language keywords has already been described in the C compiler, which means the C compiler knows what these words mean and what it has to do with them.

These meanings cannot be changed. Hence, keywords cannot be used as a variable name or a function name (in short as an Identifier) because that would try to change the existing meaning of the keyword, which is not allowed.

You cannot use Keywords as Identifiers. This means you cannot name your variable, function, structure or pointer, or anything else, using a keyword.

There are a total of 32 keywords in the C language.

List of Keywords in C:

The following is a table of keywords available in the C language.

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
const	extern	return	union
char	float	short	unsigned
continue	for	signed	volatile
default	goto	sizeof	void
do	if	static	while

All these keywords were defined in the C89 standard, which was the first version of the C language, while some **more keywords** were **added in the later C99** standard.

The keywords added by C99 are:

_Bool	_Imaginary	restrict
_Complex	inline	

Each keyword has its own meaning attached to it, for example, the `int` keyword is used to define an **integer datatype** value, `if` / `else` are used for defining **conditional statements**, and so on. We will learn most of them as we progress in this tutorial.

Important Points to Note:

Here are some useful points to remember about Keywords.

- A keyword can not be used as an Identifier (*remember this always*).
- Keywords must be written in lower case. Apart from some C99 based keywords, all other keywords are in lowercase.
- Keywords hold special meaning for the C compiler, so respect that, otherwise you will get a compile-time error.

The `main()` is not a keyword, but you should treat it as a keyword only. Because this function is a special function. If you name any variable as **main**, you may confuse the Compiler.

C Identifiers

In the C language identifiers are the names given to variables, constants, functions, structures, pointers, or any other user-defined data, to identify them.

Because it's up to the user to define the Identifiers, hence there are some rules that one has to follow, which are defined by the C language, to avoid unnecessary errors in the compiler.

Rules for defining an Identifier:

- An identifier can only have **alphanumeric characters**(a-z , A-Z , 0-9) and **underscore**(`_`).
- The **first character** of an identifier can only contain **alphabet**(a-z, A-Z) or **underscore** (`_`).
- Identifiers are case-sensitive in the C language. For example, **name** and **Name** will be treated as two different identifiers.
- Keywords are not allowed to be used as Identifiers.
- **No special characters**, such as a **semicolon**, **period**, **whitespaces**, **slash**, or **comma** are permitted to be used in or as an Identifier.

Using Identifiers and Keywords:

Let's see a few examples where we'll use keywords and identifiers. We will define a C variable, so do not get confused, we will be learning about variables very soon.

When we declare a variable or any function in any C language program, to use it we must provide a name to it, that name is then used throughout the program, for example:

```
int myvariable = "Studytonight";
```

Here `myvariable` is the name or identifier for the variable which stores the value "**Studytonight**" in it. And `int` is the keyword. And all of them are tokens.

Let's have another example,

```
int money;
double salary;
```

Example of Valid Identifiers:

Here are some valid identifiers,

```
total, avg1, difference_1;
```

Example of Invalid Identifiers:

Here are some invalid identifiers,

```
$myvar; // incorrect
x!y;  // again incorrect
```

Using Keywords as Identifiers

What happens when we use keywords as identifiers?

```
#include <stdio.h>

int main() {
   // using void as name of variable (identifier)
   int void = 0;

   return 0;
}
```

error: two or more data types in declaration specifiers
int void = 0;
^

Run Code →

The C language compiler will think that we have mistakenly used two data types together and will give an error. Because `void` is also a data type in the C language(hence it is a keyword).

Frequently Asked Questions

Here are some frequently asked questions related to keywords and identifiers.

Q1. What is the difference between Identifier and Variable?

The **identifier is just the name** given to the variable, while a variable represents a memory field where some data is stored based on its data type. It's just like what is the difference between you and your name. Your name is just a word that people call you with, while you are a human being.

Conclusion

This tutorial was all about keywords and identifiers in C language. They both are parts of the token. Tokens in C are defined as the smallest individual element in C and are the basic building blocks of creating a program. We have covered keywords and identifiers in detail along with their valid syntax.

Data Types in C Language

studytonight.com/c/datatype-in-c.php

As the name suggests, a Datatype defines the **type of data** being used. Whenever we define a variable or use any data in the C language program, we have to specify the type of the data, so that the compiler knows what type of data to expect.

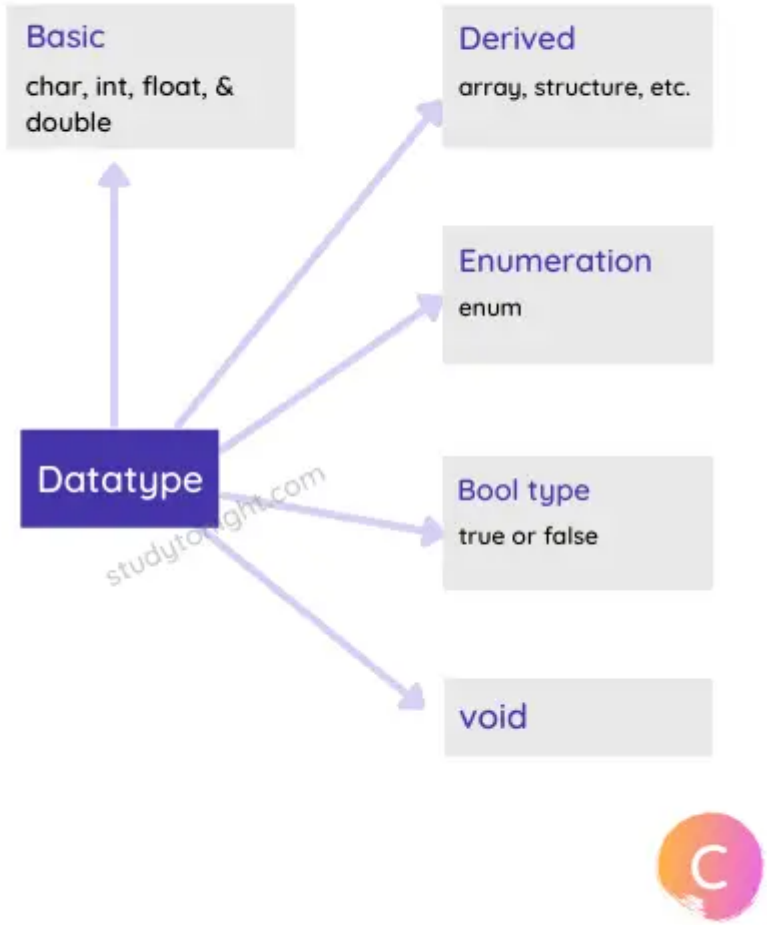
For example, you may want to use a **number** like **1**, **2**, **100**, or a **decimal point** number like **99.95**, **10.5**, or a **text**, like **"Studytonight"**, all these values are **handled differently by the C language compiler**, hence, we use data types to define the type of data used in any program.

Each data type occupies some memory, has a range of values and a set of operations it allows to be performed on itself. In this tutorial, we have explained the different data types used in C language. For code examples, checkout Using C Datatypes tutorial.

Data Types in C

Broadly, there are 5 different categories of data types in the C language, they are:

Type	Example
Basic	character, integer, floating-point, double.
Derived	Array, structure, union, etc.
Enumeration	enums
Bool type	true or false
void	Empty value



Primary Data types in C

The C language has 5 basic (primary or primitive) data types, they are:

1. **Character:** We use the keyword `char` for character data type. It is used to store single bit characters and occupies **1 byte** of memory. We can store alphabets from A-Z(and a-z) and 0-9 digits using `char`. For example,

   ```
   char a = 'a';
   char b = 'A';
   char c = '0';
   char d = 0; //error
   ```

   For `char` datatype, it is necessary to enclose our data in **single quotes**. We can perform addition and subtraction operations on `char` but the ASCII value should not exceed 127.

2. **Integer:** We use the keyword `int` for integer data type. The `int` data type is used to store non-fractional numbers which includes positive, negative and zero values. The range of int is **-2,147,483,648 to 2,147,483,647** and it occupies 2 or 4 bytes of memory, depending on the system you're using. For example,

```
int a = 5550;
int b = -90,
int c = 0;
int d = -0.5; //invalid
```

   We can perform addition, subtraction, division, multiplication, bitwise and modulo operations on `int` data type.

3. **Floating-point:** We use the keyword `float` for floating-point data type. `float` is used to store **decimal numbers**. It occupies 4 bytes of memory and ranges from **1e-37 to 1e+37**. For example,

```
float a = 0.05;
float b = -0.005.
float c = 1;  // it will become c = 1.000000 because of type-casting
```

   We can perform addition, subtraction, division, and multiplication operations on `float` data type.

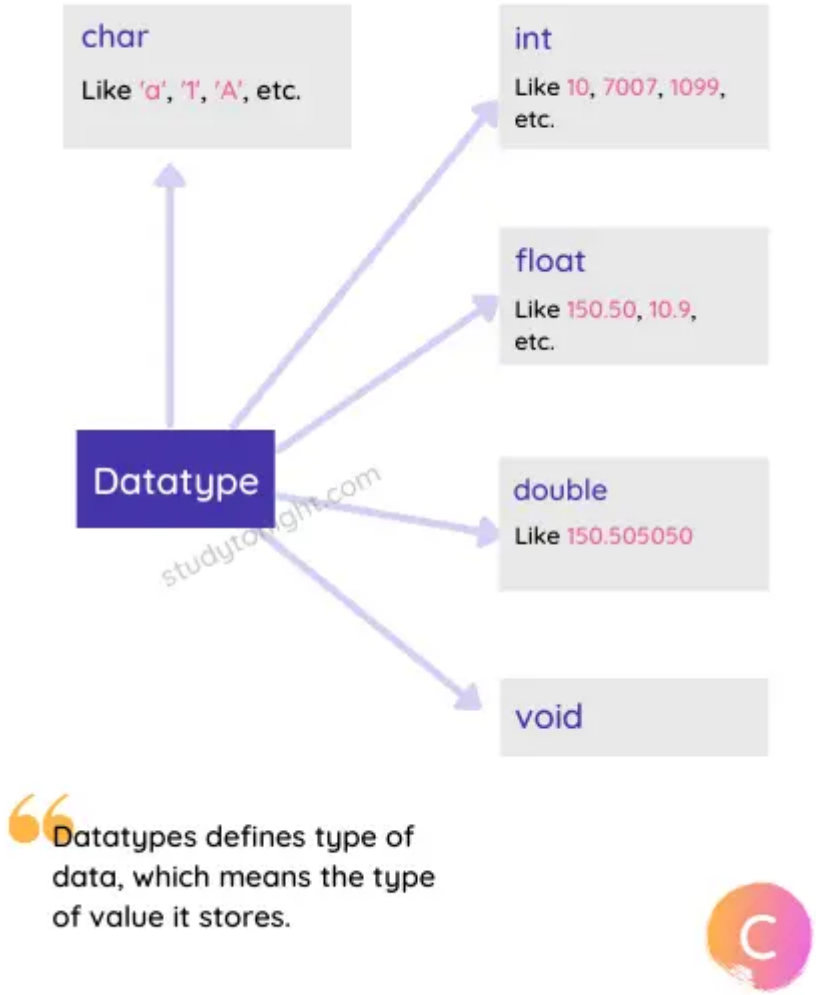
4. **Double:** We use the keyword `double` for double data type. `double` is used to store **decimal numbers**. It occupies **8 bytes** of memory and ranges from **1e-37 to 1e+37**.

```
double a = 10.09;
double b = -67.9;
```

   `double` has more precision than `flaot` so `double` gives more accurate results as compared to `float`. We can perform addition, subtraction, division and multiplication operations on `double` data type.

5. **Void:** This means no value. This data type is mostly used when we define functions. The `void` data type is used when a function does not return anything. It occupies **0 bytes** of memory. We use the `void` keyword for void data type.

```
void function() {
   //your code goes here
}
```



Each data type has a **size** defined in **bits/bytes** and has a **range** for the values that these data types can hold.

Size of different Data types in C

The size for different data types depends on the compiler and processor types, in short, it depends on the Computer on which you are running the C language and the version of the C compiler that you have installed.

char is 1 byte

The `char` datatype is **1 byte** in size or **8 bits**. This is mostly the same and is not affected by the processor or the compiler used.

int can be 2 bytes/4 bytes

There is a very easy way to remember the size for `int` datatype. The size of `int` datatype is usually equal to the word length of the execution environment of the program. In simpler words, for a **16-bit environment**, `int` is **16 bits** or **2 bytes**, and for a **32-bit environment**, `int` is **32 bits** or **4 bytes**.

float is 4 bytes

The `float` datatype is **4 bytes** or **32 bits** in size. It is a **single-precision data type** that is used to hold decimal values. It is used for storing large values.

`float` is a faster data type as compared to `double`, because `double` data type works with very large values, hence it is slow.

double is 8 bytes

The `double` datatype is **8 bytes** or **64 bits** in size. It can store values that are **double the size of what a float data type can store**, hence it is called `double`.

In the 64 bits, **1 bit** is for **sign** representation, **11 bits** for the **exponent**, and the rest **52 bits** are used for the **mantissa**.

The `double` data type can hold approximately **15 to 17 digits**, before the decimal and after the decimal.

void is 0 bytes

The `void` data type means nothing, hence it doesn't have a size.

Good Read: Understand what are Bits and Bytes

Before moving on to the range of values for these data types, there is one more important concept to learn, which is **Datatype modifiers**.

C Data type Modifiers:

In the C language, there are **4 datatype modifiers**, that are used along with the basic data types to categorize them further.

For example, if you say, there is a playground, it can be a park, a playground or a stadium, but if you be more specific and say, there is a Cricket ground or a Football stadium, that would make it even more precise.

Similarly, there are modifiers in the C language, to **make the primary data types more specific**.

Following are the modifiers:

1. signed
2. unsigned
3. long
4. short

As the name suggests, **signed** and **unsigned** are used to represent the **signed(+ and -)** and **unsigned(only +) values** for any data type. And **long** and **short** affects the **range of the values** for any datatype.

For example, **signed int**, **unsigned int**, **short int**, **long int**, etc. are all valid data types in the C language.

```
long long num = 123456789987654321; // we cannot store a value this big value using int data type.
```

Now let's see the range for different data types formed as a result of the 5 primary data types along with the modifiers specified above.

C Data type Value Range

In the table below we have the range for different data types in the C language.

Type	Typical Size in Bits	Minimal Range	Format Specifier
`char`	8	-127 to 127	`%c`
`unsigned char`	8	0 to 255	`%c`
`signed char`	8	-127 to 127	`%c`
`int`	16 or 32	-32,767 to 32,767	`%d` , `%i`
`unsigned int`	16 or 32	0 to 65,535	`%u`
`signed int`	16 or 32	Same as int	`%d` , `%i`
`short int`	16	-32,767 to 32,767	`%hd`
`unsigned short int`	16	0 to 65,535	`%hu`
`signed short int`	16	Same as short int	`%hd`
`long int`	32	-2,147,483,647 to 2,147,483,647	`%ld` , `%li`
`long long int`	64	$-(2^{63} - 1)$ to $2^{63} - 1$ (Added by C99 standard)	`%lld` , `%lli`
`signed long int`	32	Same as long int	`%ld` , `%li`
`unsigned long int`	32	0 to 4,294,967,295	`%lu`
`unsigned long long int`	64	$2^{64} - 1$ (Added by C99 standard)	`%llu`

Type	Typical Size in Bits	Minimal Range	Format Specifier
`float`	32	1E-37 to 1E+37 with six digits of precision	`%f`
`double`	64	1E-37 to 1E+37 with ten digits of precision	`%lf`
`long double`	80	1E-37 to 1E+37 with ten digits of precision	`%Lf`

As you can see in the table above, with different combinations of the datatype and modifiers the range of value changes.

When we want to print the value for any variable with any data type, we have to use a **format specifier** in the `printf()` statement.

What happens if the value is out of Range?

Well, if you try to assign a value to any datatype which is more than the allowed range of value, then the C language compiler will give an error. Here is a simple code example to show this,

```
#include <stdio.h>

int main() {
   // allowed value up to 65535
   unsigned short int x = 65536;

   return 0;
}
```

warning: large integer implicitly truncated to unsigned type [-Woverflow]
unsigned short int x = 65536;
^

Run Code →

When a **type modifier is used without any data type**, then the `int` data type is set as the default data type. So, `unsigned` means `unsigned int`, `signed` means `signed int`, `long` means `long int`, and `short` means `short int`.

What does `signed` and `unsigned` means?

This is a little tricky to explain, but let's try.

In simple words, the `unsigned` modifier means **all positive values**, while the `signed` modifier means **both positive and negative values**.

When the compiler gets a numeric value, it converts that value into a binary number, which means a combination of 0 and 1. For example, **32767** in binary is **01111111 11111111**, and **1** in binary is **01 (or 0001)**, **2** is **0010**, and so on.

In the case of a **signed integer**, the **highest order bit** or the first digit from left (in binary) is used as the **sign flag**. If the **sign flag is 0**, the number is **positive**, and if it is **1**, the number is **negative**.

And because one bit is used for showing if the number is positive or negative, hence there is one less bit to represent the number itself, hence the range is less.

For **signed int**, **11111111 11111111 means -32,767** and because the first bit is a **sign flag** to mark it as a negative number, and rest represent the number. Whereas in the case of **unsigned int**, **11111111 11111111** means **65,535**.

Derived Data types in C

While there are 5 primary data types, there are some derived data types too in the C language which are used to store complex data.

Derived data types are nothing but primary data types but a little twisted or grouped together like an **array**, **structure**, **union**, and **pointers**. These are discussed in detail later.

Conclusion:

In the next tutorial, we will learn about variables and there you will learn the actual usage of data types, with man code examples. So let's move on.

Using C Datatypes (with Examples)

studytonight.com/c/using-datatypes-examples.php

The Datatype defines the **type of data** being used. We have covered **C datatypes** in detail in the previous tutorial.

The C language has 5 basic (primary or primitive) data types, they are:

1. **Character** - `char`
2. **Integer** - `int`
3. **Floating-point** - `float`
4. **Double** - `double`
5. **Void** - `void`

Let's learn about each one of them one by one.

To learn about the **size of data types**, **range of values** for datatypes, and various **type modifiers** like `signed`, `unsigned`, `long`, and `short` - Visit **C datatypes in detail**

1. char Datatype

The `char` datatype refers to character values, enclosed in single quotes, with a range of **-127 to 127**.

As it's clear from the range, we can even use **small integer values** in the `char` datatype.

For example,

```
char status = 'Y';
```

2. int Datatype

The `int` datatype is used to store whole numbers, which are values without any decimal part or exponent part.

The `int` datatype can store **decimal** (base 10), **octal** (base 8), and **hexadecimal** (base 16) values.

```
// simple int value
int a = 100;

// negative value
a = -100;

// unsigned int value - with suffix U or u
int x = 1000U;

// long int value
long int long_val = 3500L;
```

With the value of `int` data type, we can use suffix `U` or `u`, to tell the compiler that the value is for `unsigned int` data type and suffix `L` or `l` for a `long int` value.

Learn more about type identifiers(`signed`, `unsigned`, `long`, and `short`) and how they change the primary datatypes when used with them - Visit **C datatypes guide for beginners**

3. float Datatype

The `float` data type is used to store **real numbers** which may have a decimal (fraction) part or an exponential part. It is a single-precision number.

Let's see some examples for `float` value,

```
float x = 127.675;

// with suffix F or f
float y = 1000.5454F;
```

Just like `int` datatype, the `float` can also be used with type modifiers - Learn more about it, visit **C datatypes guide for beginners**

4. double Datatype

The real numbers which are big enough that they cannot be stored in `float` datatype, is stored as a `double` datatype. It is a double-precision number. A double datatype value can hold above 15 to 17 digits before the decimal point and 15 to 17 digits after the decimal point.

Here is an example,

```
double x = 424455236424564.24663224663322;
```

We should only use the `double` datatype when we need such large numbers, otherwise not, because using `double` datatype **makes the program slow**.

Conclusion:

In this tutorial, we saw some examples for different data types available in the C language. To understand about the C data types in detail, visit **C datatypes guide for beginners** .

C Variables (with Examples)

studytonight.com/c/variables-in-c.php

In this tutorial, we will be learning what are **Variables**, how to **declare and define a variable** in the C language, and what are **constants and literals** in the C language.

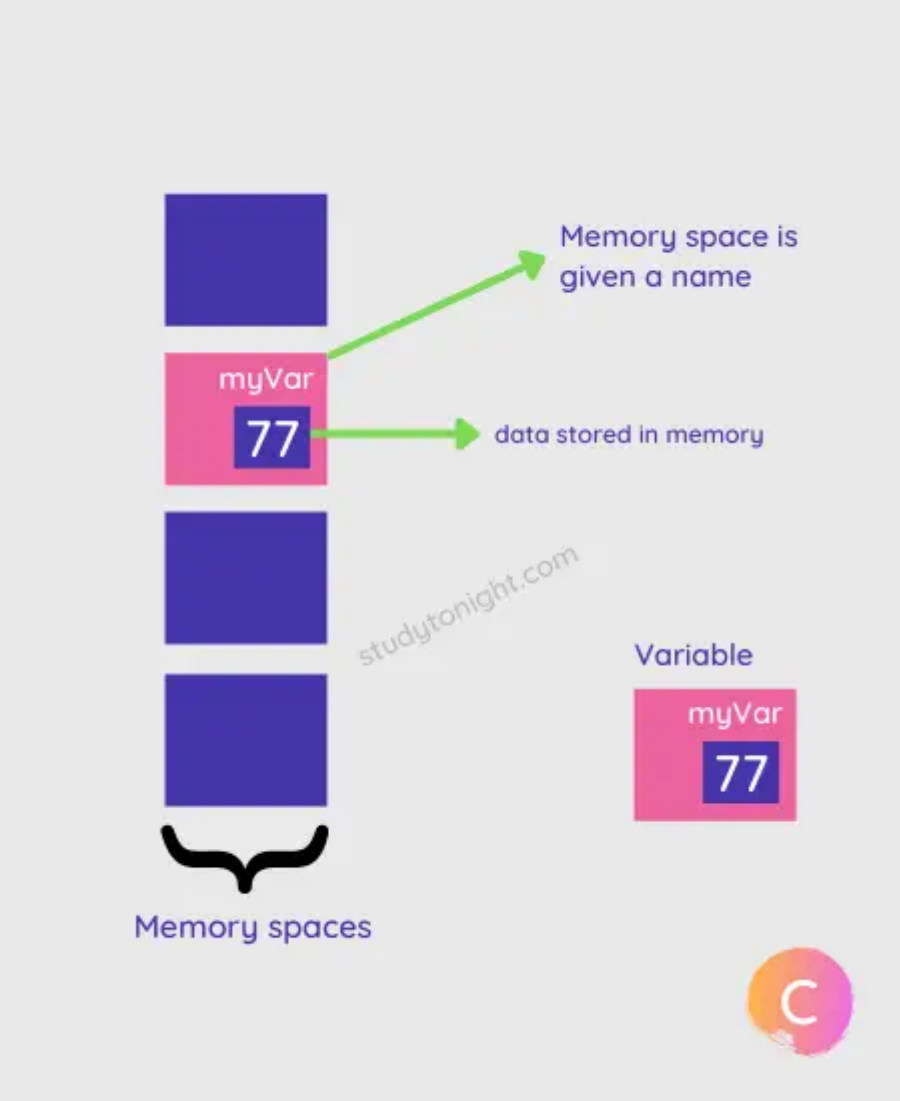
Variable is like a container (storage space) with a name in which you can store data.


C variable example

It's like **your home has an address** (House No./Flat No.) so whenever you order something online, you just have to provide your home address and the delivery executive is able to find it.

In the C language, if we want to use some **data value in our program**, how can we do it? Well, it's done using variable.

When we create a variable in a C program, the C compiler allocates a storage space, depending upon the datatype of the variable(8 bits for `char`, 16/32 bits for `int`, etc.), and then that storage space is given a name which is the **variable name**.



Once a variable is created, we can store value in it. We can change the value stored in a variable as many times as we want, but we should make sure we store the correct data type value.

Syntax for using Variable:

Here is how you can **create or declare** a new variable in the C language,

```
data_type var_name;
```

where, **data_type** is a **valid data type** (along with datatype modifiers, if required) and **var_name** is the **name of the variable**.

For example,

```
int marks;
```

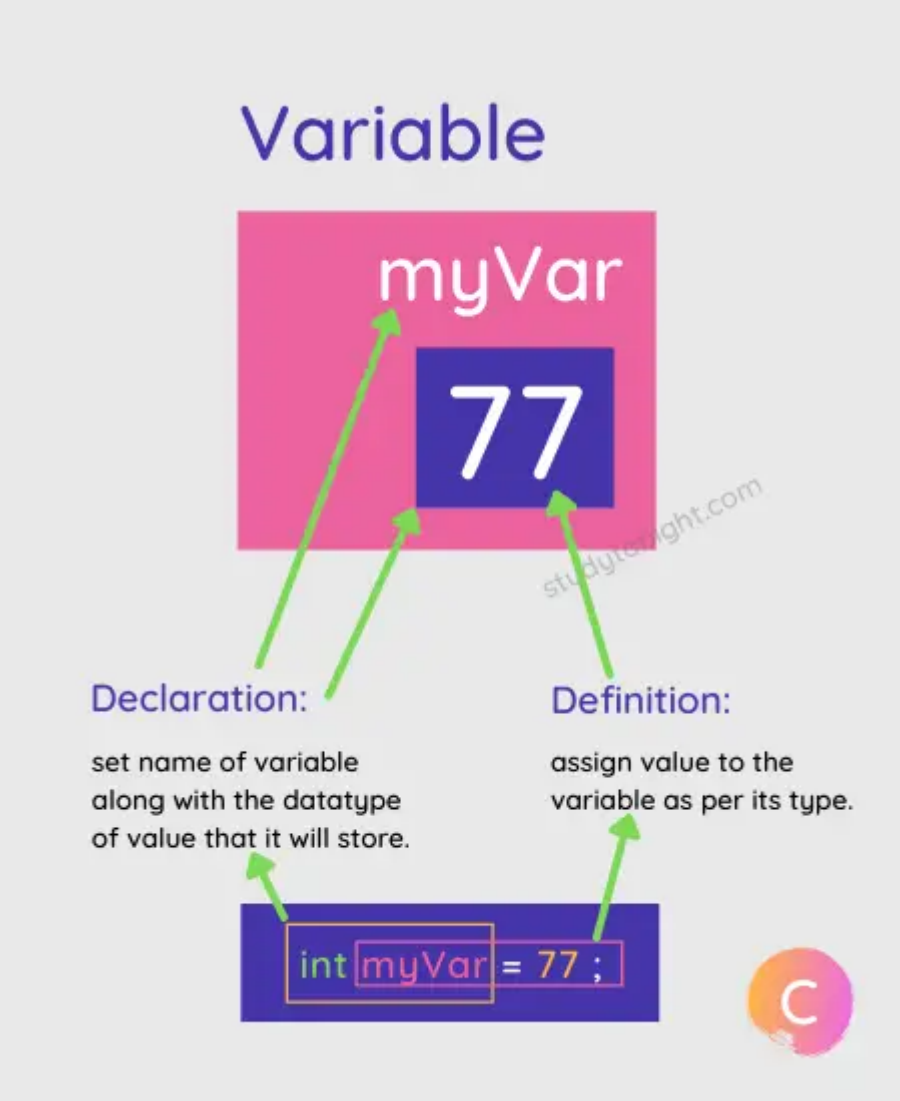
Here, `marks` is the name of the variable, and it can store values of `int` type.

Once we have declared or created the variable, then we can assign a value to it. This is called **variable definition**.

```
// variable declaration
int marks;
// variable definition
marks = 10;
```

We can do **declaration and definition in a single step** too, like this (recommended).

```
int marks = 10;
```



You can **change the value of the variable**, whenever you want. But make sure the **value is of correct data type**.

```
// variable defined
int marks = 10;
// assign a new value
marks = 33;
```

Datatype of the Variable

A **variable** in C language must be given a type, which defines what type of data can be stored in the variable. If you do not provide any datatype, then the C compiler will give **compile-time** error or syntax error.

The datatype can be `char`, `int`, `float`, `double`, or `short int`, `long int`, etc. data type along with modifiers.

You can learn about **C Datatypes** to see the list of various data types supported in the C language.

Let's take an example,

```
// char type variable
char status = 'Y';

// int type variable
int marks = 95;

// float type variable
float percentage = 94.6;

// double type variable
double long = 76.997429;
```

If you try to assign an **incorrect datatype value** to a variable, then the compiler may giver error, or it will automatically **convert the value into the datatype of the variable**.

For example,

```
#include <stdio.h>

int main() {
   // assign incorrect value
   int x = 10.58;
   printf("Value is %d", x);
   return 0;
}
```

Value is 10

Run Code →

As you can see in the output, the C compiler **removed the part after the decimal**, because `int` datatype can only store whole numbers.

We will learn about `printf()` function and `%d` (used to print the value of the `int` datatype) in the next tutorial, with more code examples.

Datatype cannot be changed

Once we define a variable with some datatype then we cannot change the datatype of that variable.

For example,

```
// int variable
int marks = 10;

float marks;    // error
```

Rules to name a Variable

When you create a variable, you should always give a meaningful name to the variable. And follow the below rules for naming the variable:

1. Variable name must **not start with a digit**.
2. The variable name can consist of alphabets, digits, and special symbols like underscore `_`.
3. Blank or spaces are not allowed in the variable name.
4. Keywords are not allowed as a variable name.
5. Upper and lower case names are treated as different, as **C is case-sensitive**, so it is suggested to keep the variable names in lower case.

Let's see a few examples for incorrect names as per the above rules:

```
int 1var;    // incorrect - should not start with number
int var1;    // correct

int my$var    // incorrect - special characters not allowed
int my_var1;    // correct

int my var;    // incorrect - spaces not allowed

char else;    // can't use Keywords

int count;    // valid variable name
int Count;    // new variable
int COUNT;    // new variable
```

Creating a Variable - Behind the Scenes

Declaration of variables must be done before they are used in the program. The declaration does the following things.

1. It tells the compiler what the variable name is.
2. It specifies what type of data the variable will hold.
3. Until the variable is defined the compiler doesn't have to worry about allocating memory space to the variable.
4. The declaration is more like informing the compiler that there exists a variable with the following datatype which is used in the program.

5. We can even declare a variable outside the `main()` function, using the `extern` keyword.

```
extern int a;
extern float b;
extern double c, d;
```

Defining or **Initializing** a variable means the compiler has to now assign storage to the variable because it will be used in the program.

We can even **declare multiple variables** of the same data type in a single line by using a comma to separate them.

For example,

```
int a;
float b, c;
```

Initializing a variable means providing it with a value.

```
int a = 10;
```

Difference between Variable and Identifier?

An Identifier is a name given to any variable, function, structure, pointer, or any other entity in a programming language. While a variable, as we have just learned in this tutorial is a named memory location to store data that is used in the program.

Identifier	Variable
The identifier is the **name given to a variable**, function, etc.	While variable is used to name a memory location that stores data.
An identifier can be a variable, but not all identifiers are variables.	All variable names are identifiers.

Example:
`// studytonight is identifier for a variable`
`int studytonight;`
 	
`// studytonight is identifier for a function`
`int studytonight() {`
`  ...`
`}`
	Example:
	`// int variable`
	`int a;`
	`// float variable`
	`float a;`

Another great analogy to understand the difference between Identifier and Variable is:

You can think of an Identifier `int x` to be a variable's name, but it can also be a function's name `int x() { }` and still, be an identifier.

Just like **Obama** is the name of a person, but also the name of a foundation.

Conclusion

In this tutorial we learned about C Variables, how to create a variable, assign value to a variable and what happens when a variable is created. We also learned about the importance of datatype while creating a new variable.

C Literals/Constants

studytonight.com/c/c-literals.php

If we want to use some data value in a C program, we can also **directly use a value**, instead of creating a variable to store the value. When we use a value directly in C program **without creating a variable**, it is known as a **Literal**.

A literal is used when we want to **use a fixed value** in the program, hence literals are also called **Constants**.

For example, **1**, **100**, **'Y'**, **10.5**, etc. Whereas, when we use a variable in C, we can change the value stored in the variable.

In the language, literals can be of 5 types, they are:

1. Integer Literal
2. Character Literal
3. Floating-point Literal
4. String Literals
5. Backslash Character Literals (Escape Sequences)

1. Integer Literal

Any numeric value without any decimal or exponential part, used in the C program is an integer literal.

Integer literals are of 3 types:

1. Decimal Numer (base 10)
2. Octal Number (base 8) - Uses digits **0 up to 7**. Number 10 is the same as 8 for the octal system.
3. Hexadecimal Number (base 16) - Uses digits **0 up to 9**, and alphabets **A to F**. (A, B, C, D, E, F stands for 10, 11, 12, 13, 14, and 15)

For example,

```
Decimal: 7, -10 etc
Octal: 023, 045 etc
Hexadecimal: 0x2a, 0x521 etc
```

In the C language, the **octal** number starts with a **0**, and the **hexadecimal** number starts with a **0x**.

You can also **use type suffix** with integer values to make it easier for the compiler to understand the type of the integer value. If you follow the value with `L`, the compiler will treat it as a `long` value, if you add `U` after the value, the compiler treats the value as **unsigned**.

For example,

```
3500L, 14l    // long value

99U, 89u    // unsigned int value
```

2. Floating-point Literals

Any valid floating-point value when used directly in a C program, is called a floating-point literal.

The floating-point numbers have a decimal part (fraction) or an exponential part.

For example,

```
100.50
0.000127
-0.77E-5
```

E-5 stands for 10^{-5}

If we **add a type suffix** `F` or `f` after the value, then the compiler will treat the value as `float` type by default.

For example,

```
127.7F, 40f etc.
```

3. Character Literals

A valid character datatype value when used directly in the C program is called a character literal.

The character values are enclosed within a single quote.

For example,

```
'A', 'B', 'c', '#', etc.
```

4. String Literals

A **sequence of characters** is called a String. We will learn about C strings in detail in the later section of this tutorial series.

A string literal is a value with multiple characters enclosed within double-quotes.

For example,

```
"study", "tonight", "c programming", etc.
```

5. Backslash Character Literals

Also known as **Escape sequences**, these combinations of characters with the backslash character `\` have special meaning for the C compiler.

A backslash character is used to **escape characters with special meaning** and make them normal characters for the compiler.

For example, if you use a single quotation mark, the compiler will think that it is the starting or end of a character value, but what if you want a single quote character.

```
'\''    \\ this is character with value '
```

In the example above the **first and the last single quote** marks the **start and end of the character value**, whereas the backslash escape character informs the compiler to treat the single quotation after it as a normal character.

Code	Meaning
\b	Backspace
\f	Form feed
\n	New line
\r	Carriage return
\t	Horizontal tab
\"	Double quote
\'	Single quote
\\	Backslash
\v	Vertical tab
\a	Alert
\?	Question mark
\N	Octal contant
\xN	Hexadecimal constant

The **newline** and **horizontal tab** code are quite **commonly used**, and you will be using them a lot as you start coding in the C language. So remember `\n` is for **newline** and `\t` is for the **horizontal tab**.

Conclusion:

With this, we have covered all the **Constants** used in the C language which are also called **Literals**. Do not ignore the backslash escape literals as they are also very important and widely used in the C programs.

C Constant value Variables - const Keyword

studytonight.com/c/c-const-variable.php

If you want to create a **variable whose value cannot be changed**, then you can use the `const` keyword to create a constant value variable.

The variables which have type `const`, cannot be changed by the program. We can provide a value while defining the variable, and then, throughout the program, that variable will hold the same value.

Using `const` Keyword

Here is a simple example for using the const keyword,

```
const int totalmarks = 100;
```

In the above code, we have defined a variable with name `totalmarks` and assigned it a value **100**. Because we have used the `const` keyword while defining the variable hence we cannot change the value of the `totalmarks` variable.

Change `const` Variable value

If you will try to change the value of the `const` variable in the program after it has been defined, then the compiler will give an error.

For example,

```
#include <stdio.h>

int main() {
   // initialize a constant variable
   const int total_marks = 10;

   // try changing value
   total_marks = 80;

   return 0;
}
```

error: assignment of read-only variable 'total_marks'
total_marks = 80;
^

Run Code →

As you can see in the error message, the compiler says that the variable is a **read-only variable**, hence its **value cannot be changed**.

Because const variables are read-only variables, the compiler can place these variables into the **read-only memory** (**ROM**).

When we define a variable as `const`, then nothing in that program can change the value of that variable.

Hence, if the value of the `const` variable **changes**, then you can say that something outside of the program changed the value, it can be the hardware device or any external event.

Conclusion:

In this tutorial, we learned how to create constant value variables, which are the variable whose values cannot be changed by the program.

C Input and Output - printf()/scanf(), and more.

studytonight.com/c/c-input-output-function.php

Input means to provide the program with some data to be used in it and **Output** means to display data on the screen or write the data to a printer or a file.

The C programming language provides standard library functions to read any given input and display output on the console.

Before moving forward with input and output in C language, check these topics to understand the concept better :

- C Syntax Rules
- Compile and Run C Program
- Data Types in C
- Variables in C

While dealing with input-output operations in C, we use the following two streams:

- **Standard Input (stdin)**
- **Standard Output (stdout)**

Standard input or **stdin** is used for taking input and **Standard output** or **stdout** is used for giving output. The functions used for standard input and output are present in the **stdio.h** header file. Hence, to use those functions, we need to include the **stdio.h** header file in our program, as shown below.

```
#include <stdio.h>
```

C language offers us several built-in functions for performing input/output operations. Following are the functions used for standard input and output:

1. `printf()` function - Show Output
2. `scanf()` function - Take Input
3. `getchar()` and `putchar()` function
4. `gets()` and `puts()` function

In C Language, output devices like computer monitor, printer, etc. are treated as files and the same process is followed to write output to these devices as would have been followed to write the output to a file.

1. The `printf()` function

The `printf()` function is the most used function in the C language. This function is defined in the **stdio.h** header file and is used to show output on the console (standard output).

Following is how the `printf()` function is defined in the C **stdio.h** library.

```
int printf(const char *format, ...);
```

It writes the C string pointed by the format pointer to the standard output (stdout). On success, the **total number of characters written** is returned.

This function is used to **print a simple text sentence** or **value of any variable** which can be of `int`, `char`, `float`, or any other datatype.

`printf()` Code Examples

Let's start with a simple example.

1. Print a sentence

Let's print a simple sentence using the `printf()` function.

```
#include <stdio.h>

int main() {
   // using printf()
   printf("Welcome to Studytonight");

   return 0;
}
```

Welcome to Studytonight

Run Code →

We have seen many code examples like above, up until now in this tutorial series.

To understand the complete code and structure of a basic C language program, check **Hello World Program in C**.

2. Print an Integer value

We can use the `printf()` function to print integer value coming from a variable using the `%d` format specifier.

For example,

```
#include <stdio.h>

int main() {
   int x = 10;
   // using printf()
   printf("Value of x is: %d", x);

   return 0;
}
```

Value of x is: 10

Run Code →

In the program, above we have used the `%d` format specifier, to specify the type of the value that will be added there. The format specifiers `%d` and `%i` are used for integer value.

3. Print a Character value

The `%c` format specifier is used to print character variable value using the `printf()` function.

```
#include <stdio.h>

int main() {
   // using printf()
   char gender = 'M';

   printf("John's Gender is: %c", gender);
   return 0;
}
```

Inside the `printf()` function we have used a **single quotation**, it will work fine because we have used double quotation outside. We can also use **backslash character** to escape the single quote like this `\'`

4. Print a Float and a Double value

In the code example below, we have used the `printf()` function to print values of a `float` and `double` variable.

For `float` value we use the `%f` format specifier and for `double` value we use the `%lf` format specifier.

```
#include <stdio.h>
int main() {
   // using printf()
   float num1 = 15.50;
   double num2 = 15556522.0978678;

   printf("Value of num1 is: %f \n", num1);
   printf("Value of num2 is: %lf", num2);

   return 0;
}
```

Value of num1 is: 15.500000
Value of num2 is: 15556522.097868

We have used the `\n` Escape sequence which is used for newline at the end of the first `printf()` statement so that the next `printf()` statement output is **shown in the next line**.

5. Print multiple outputs

We can use a single `printf()` function to display values of multiple variables.

```
#include <stdio.h>

int main() {
   // using printf() for multiple outputs
   int day = 20;
   int month = 11;
   int year = 2021;

   printf("The date is: %d-%d-%d", day, month, year);

   return 0;
}
```

As you can see in the code example above, we can **do formatting as well** while printing values of multiple variables using the `printf()` function.

We can also perform some simple calculations inside `printf()`. Here is a simple example for that,

```
#include <stdio.h>
int main()
{
    int a = 5, b = 6;
    printf("%d", a + b);
    return 0;
}
```

11

2. The `scanf()` function

When we want to take input from the user, we use the `scanf()` function and store the input value into a variable.

Following is how the `scanf()` function is defined in the C **stdio.h** library.

```
int scanf(const char *format, ...);
```

It reads data from stdin and stores it according to the parameter format into the locations pointed by the additional arguments. On success, the function returns the **number of items of the argument list** successfully filled.

The `scanf()` function can be used to take any datatype input from user, all we have to take care is that the variable in which we store the value should have the same datatype.

Here is the **syntax** for `scanf()` :

```
scanf("%x", &variable);
```

where, `%x` is the format specifier.

Using the format specifier, we tell the compiler what type of data is in a variable and & is the address operator which tells the compiler the address of the variable so that the compiler can assign the variable with the value entered by the user.

`scanf()` Code Examples

Let's start with a simple example.

1. Input Integer value

If we have to take an integer value input from the user, we have to define an integer variable and then use the `scanf()` function.

```
#include <stdio.h>
int main() {
   // using scanf()
   int user_input;

   printf("Please enter a number: ");
   scanf("%d", &user_input);
   printf("You entered: %d", user_input);

   return 0;
}
```

Run Code →

Please enter a number: 7
You entered: 7

While running the code example above, there is a button for Input at top-right corner of editor, you can click on it and provide custom value for input.

In the above code example, we have used `%d` format specifier to inform the `scanf()` function that user input will be of type **int**.

And we have also used `&` symbol before the name of the variable, because `&user_input` refers to the address of the `user_input` variable where the input value will be stored.

2. Input Float value

Just like integer value, we can take input for any different datatype. Let's see an example for float type value.

```
#include <stdio.h>
int main() {
   // using scanf()
   float user_input;

   printf("Please enter a decimal number: ");
   scanf("%f", &user_input);
   printf("You entered: %f", user_input);

   return 0;
}
```

Please enter a decimal number: 7.007
You entered: 7.007

We have used the `%f` format specifier and defined a `float` type variable. Try doing the same for taking a `double` type value as user input. The format specifier for `double` is `%lf`.

3. Input Character value

Let's see how we can take a simple character input from user.

```
#include <stdio.h>
int main() {
   // using scanf()
   char gender;

   printf("Please enter your gender (M, F or O): ");
   scanf("%c", &gender);
   printf("Your gender: %c", gender);

   return 0;
}
```

Please enter your gender (M, F or O): M
Your gender: M

4. Take Multiple Inputs from User

In the below code example, we are taking multiple inputs from the user and saving them into different variables.

```
#include <stdio.h>
int main() {
   // using scanf() for multiple inputs
   char gender;
   int age;

   printf("Enter your age and then gender(M, F or O): ");
   scanf("%d %c", &age, &gender);
   printf("You entered: %d and %c", age, gender);

   return 0;
}
```

Enter your age and then gender(M, F or O): 32 M
You entered: 32 and M

Return Value of `printf()` & `scanf()`

The `printf()` function returns the number of characters printed by it, and `scanf()` function returns the number of characters read by it.

```
int i = printf("studytonight");
printf("Value of i is: %d", i);
```

studytonightValue of i is: 12

In this program `printf("studytonight");` will return `12` as result, which will be stored in the variable `i`, because the word **studytonight** has 12 characters.

The first `printf()` statement will print the statement **studytonight** on the output too.

Format Specifiers

To print values of different data types using the `printf()` statement and while taking input using the `scanf()` function, it is mandatory to use format specifiers.

It is a way to tell the compiler what type of data is in a variable. Some examples are `%c`, `%d`, `%f`, etc.

Here is a list of all the format specifiers.

Datatype	Format Specifier
`int`	`%d`, `%i`
`char`	`%c`
`float`	`%f`
`double`	`%lf`
`short int`	`%hd`
`unsigned int`	`%u`
`long int`	`%li`
`long long int`	`%lli`
`unsigned long int`	`%lu`
`unsigned long long int`	`%llu`
`signed char`	`%c`
`unsigned char`	`%c`
`long double`	`%Lf`

3. `getchar()` & `putchar()` functions

The `getchar()` function reads a character from the terminal and returns it as an **integer**. This function reads only a single character at a time.

Here is the syntax for the `getchar()` function:

```
int getchar(void);
```

You can use this method in a loop in case you want to read more than one character.

The `putchar()` function displays the character passed to it on the screen and returns the same character. This function too displays only a single character at a time.

Here is the syntax for the `putchar()` function:

```
int putchar(int character);
```

In case you want to display more than one character, use `putchar()` method in a loop.

```
#include <stdio.h>

void main( )
{
    int c;
    printf("Enter a character");
    /*
        Take a character as input and
        store it in variable c
    */
    c = getchar();
    /*
        display the character stored
        in variable c
    */
    putchar(c);
}
```

Enter a character: Studytonight
S

When you will compile the above code, it will ask you to enter a value. When you will enter the value, it will display the value you have entered.

4. `gets()` & `puts()` functions

The `gets()` function reads a line from **stdin**(standard input) into the buffer pointed to by `str` pointer, until either a terminating newline or EOF (end of file) occurs.

Here is the syntax for the `gets()` function:

```
char* gets(char* str);
```

The `puts()` function writes the string `str` with a newline character ('\n') at the end to **stdout**. On success, a non-negative value is returned.

Here is the syntax for the `gets()` function:

```
int puts(const char* str);
```

`str` is the pointer to an array of **chars** where the C string is stored. (Don't worry if you are not able to understand this now.)

```
#include <stdio.h>

void main()
{
    /* character array of length 100 */
    char str[100];
    printf("Enter a string: ");
    gets(str);
    puts(str);
    getch();
    return 0;
}
```

Enter a string: Studytonight
Studytonight

When you will compile the above code, it will ask you to enter a string. When you will enter the string, it will display the value you have entered.

The `gets()` function is considered dangerous to use and should be avoided. We get a warning when we compile any code in which we have used `gets()` function. This is because the function doesn't know how big the buffer is, so it continues reading until it finds a newline or encounters EOF, and may overflow the bounds of the buffer it was given. We can use alternatives to `gets()` function, like the `fgets()` function.

The `fgets()` function

Here is the syntax for the `fgets()` function:

```
char* fgets(char* str, int num, FILE* stream);
```

It reads characters and stores them as a C string into `str` until **(num-1)** characters have been read or either a newline or the end-of-file is reached, whichever happens first. A newline character makes `fgets` stop reading, but it is considered a valid character by the function and included in the string copied to `str`. On success, the function returns the `str`.

For example,

```
#include <stdio.h>
#define MAX 10
int main()
{
    char str[MAX];
    fgets(str, MAX, stdin);
    printf("The string is: %s", str);
    return 0;
}
```

abcdefghijkl
The string is: abcdefghij

Difference between `scanf()` and `gets()`

The main difference between these two functions is that `scanf()` **stops reading characters when it encounters a space**, but `gets()` **reads space as a character too**.

If you enter a name as **Study Tonight** using `scanf()` it will only read and store **Study** and will leave the part of the string after space. But `gets()` function will read it completely.

Let's see the `scanf()` function in action:

```
#include <stdio.h>
int main() {
   // using scanf()
   char n1[50], n2[50];
   printf("Please enter n1: ");
   scanf("%s", n1);

   printf("You entered: %s", n1);
   return 0;
}
```

Please enter n1: study tonight
You entered: study

Now let's see the `gets()` function in action:

```
#include <stdio.h>
int main() {
   // using scanf()
   char n1[50], n2[50];
   printf("Please enter n1: ");
   gets(n1);

   printf("You entered: %s", n1);
   return 0;
}
```

Please enter n1: study tonight
You entered: study tonight

Conclusion

In this tutorial we learned about Input and Output in the C language. With many code examples, we understood how to use `printf()` function and `scanf()` function to show output to user and take input from the user.

We also learned about a few other functions used for showing output and taking user input.

C Input and Output - printf()/scanf(), and more.

studytonight.com/c/c-input-output-function.php

Input means to provide the program with some data to be used in it and **Output** means to display data on the screen or write the data to a printer or a file.

The C programming language provides standard library functions to read any given input and display output on the console.

Before moving forward with input and output in C language, check these topics to understand the concept better :

- C Syntax Rules
- Compile and Run C Program
- Data Types in C
- Variables in C

While dealing with input-output operations in C, we use the following two streams:

- **Standard Input (stdin)**
- **Standard Output (stdout)**

Standard input or **stdin** is used for taking input and **Standard output** or **stdout** is used for giving output. The functions used for standard input and output are present in the **stdio.h** header file. Hence, to use those functions, we need to include the **stdio.h** header file in our program, as shown below.

```
#include <stdio.h>
```

C language offers us several built-in functions for performing input/output operations. Following are the functions used for standard input and output:

1. `printf()` function - Show Output
2. `scanf()` function - Take Input
3. `getchar()` and `putchar()` function
4. `gets()` and `puts()` function

In C Language, output devices like computer monitor, printer, etc. are treated as files and the same process is followed to write output to these devices as would have been followed to write the output to a file.

1. The `printf()` function

The `printf()` function is the most used function in the C language. This function is defined in the **stdio.h** header file and is used to show output on the console (standard output).

Following is how the `printf()` function is defined in the C **stdio.h** library.

```
int printf(const char *format, ...);
```

It writes the C string pointed by the format pointer to the standard output (stdout). On success, the **total number of characters written** is returned.

This function is used to **print a simple text sentence** or **value of any variable** which can be of `int`, `char`, `float`, or any other datatype.

`printf()` Code Examples

Let's start with a simple example.

1. Print a sentence

Let's print a simple sentence using the `printf()` function.

```
#include <stdio.h>

int main() {
   // using printf()
   printf("Welcome to Studytonight");

   return 0;
}
```

Welcome to Studytonight

Run Code →

We have seen many code examples like above, up until now in this tutorial series.

To understand the complete code and structure of a basic C language program, check **Hello World Program in C**.

2. Print an Integer value

We can use the `printf()` function to print integer value coming from a variable using the `%d` format specifier.

For example,

```
#include <stdio.h>

int main() {
   int x = 10;
   // using printf()
   printf("Value of x is: %d", x);

   return 0;
}
```

Value of x is: 10

Run Code →

In the program, above we have used the `%d` format specifier, to specify the type of the value that will be added there. The format specifiers `%d` and `%i` are used for integer value.

3. Print a Character value

The `%c` format specifier is used to print character variable value using the `printf()` function.

```
#include <stdio.h>

int main() {
   // using printf()
   char gender = 'M';

   printf("John's Gender is: %c", gender);
   return 0;
}
```

Inside the `printf()` function we have used a **single quotation**, it will work fine because we have used double quotation outside. We can also use **backslash character** to escape the single quote like this `\'`

4. Print a Float and a Double value

In the code example below, we have used the `printf()` function to print values of a `float` and `double` variable.

For `float` value we use the `%f` format specifier and for `double` value we use the `%lf` format specifier.

```
#include <stdio.h>
int main() {
   // using printf()
   float num1 = 15.50;
   double num2 = 15556522.0978678;

   printf("Value of num1 is: %f \n", num1);
   printf("Value of num2 is: %lf", num2);

   return 0;
}
```

Value of num1 is: 15.500000
Value of num2 is: 15556522.097868

We have used the `\n` Escape sequence which is used for newline at the end of the first `printf()` statement so that the next `printf()` statement output is **shown in the next line**.

5. Print multiple outputs

We can use a single `printf()` function to display values of multiple variables.

```
#include <stdio.h>

int main() {
   // using printf() for multiple outputs
   int day = 20;
   int month = 11;
   int year = 2021;

   printf("The date is: %d-%d-%d", day, month, year);

   return 0;
}
```

As you can see in the code example above, we can **do formatting as well** while printing values of multiple variables using the `printf()` function.

We can also perform some simple calculations inside `printf()`. Here is a simple example for that,

```
#include <stdio.h>
int main()
{
    int a = 5, b = 6;
    printf("%d", a + b);
    return 0;
}
```

11

2. The `scanf()` function

When we want to take input from the user, we use the `scanf()` function and store the input value into a variable.

Following is how the `scanf()` function is defined in the C **stdio.h** library.

```
int scanf(const char *format, ...);
```

It reads data from stdin and stores it according to the parameter format into the locations pointed by the additional arguments. On success, the function returns the **number of items of the argument list** successfully filled.

The `scanf()` function can be used to take any datatype input from user, all we have to take care is that the variable in which we store the value should have the same datatype.

Here is the **syntax** for `scanf()` :

```
scanf("%x", &variable);
```

where, `%x` is the format specifier.

Using the format specifier, we tell the compiler what type of data is in a variable and & is the address operator which tells the compiler the address of the variable so that the compiler can assign the variable with the value entered by the user.

`scanf()` Code Examples

Let's start with a simple example.

1. Input Integer value

If we have to take an integer value input from the user, we have to define an integer variable and then use the `scanf()` function.

```
#include <stdio.h>
int main() {
   // using scanf()
   int user_input;

   printf("Please enter a number: ");
   scanf("%d", &user_input);
   printf("You entered: %d", user_input);

   return 0;
}
```

Run Code →

Please enter a number: 7
You entered: 7

While running the code example above, there is a button for Input at top-right corner of editor, you can click on it and provide custom value for input.

In the above code example, we have used `%d` format specifier to inform the `scanf()` function that user input will be of type **int**.

And we have also used `&` symbol before the name of the variable, because `&user_input` refers to the address of the `user_input` variable where the input value will be stored.

2. Input Float value

Just like integer value, we can take input for any different datatype. Let's see an example for float type value.

```
#include <stdio.h>
int main() {
   // using scanf()
   float user_input;

   printf("Please enter a decimal number: ");
   scanf("%f", &user_input);
   printf("You entered: %f", user_input);

   return 0;
}
```

Please enter a decimal number: 7.007
You entered: 7.007

We have used the `%f` format specifier and defined a `float` type variable. Try doing the same for taking a `double` type value as user input. The format specifier for `double` is `%lf`.

3. Input Character value

Let's see how we can take a simple character input from user.

```
#include <stdio.h>
int main() {
   // using scanf()
   char gender;

   printf("Please enter your gender (M, F or O): ");
   scanf("%c", &gender);
   printf("Your gender: %c", gender);

   return 0;
}
```

Please enter your gender (M, F or O): M
Your gender: M

4. Take Multiple Inputs from User

In the below code example, we are taking multiple inputs from the user and saving them into different variables.

```
#include <stdio.h>
int main() {
   // using scanf() for multiple inputs
   char gender;
   int age;

   printf("Enter your age and then gender(M, F or O): ");
   scanf("%d %c", &age, &gender);
   printf("You entered: %d and %c", age, gender);

   return 0;
}
```

Enter your age and then gender(M, F or O): 32 M
You entered: 32 and M

Return Value of `printf()` & `scanf()`

The `printf()` function returns the number of characters printed by it, and `scanf()` function returns the number of characters read by it.

```
int i = printf("studytonight");
printf("Value of i is: %d", i);
```

studytonightValue of i is: 12

In this program `printf("studytonight");` will return `12` as result, which will be stored in the variable `i`, because the word **studytonight** has 12 characters.

The first `printf()` statement will print the statement **studytonight** on the output too.

Format Specifiers

To print values of different data types using the `printf()` statement and while taking input using the `scanf()` function, it is mandatory to use format specifiers.

It is a way to tell the compiler what type of data is in a variable. Some examples are `%c`, `%d`, `%f`, etc.

Here is a list of all the format specifiers.

Datatype	Format Specifier
`int`	`%d`, `%i`
`char`	`%c`
`float`	`%f`
`double`	`%lf`
`short int`	`%hd`
`unsigned int`	`%u`
`long int`	`%li`
`long long int`	`%lli`
`unsigned long int`	`%lu`
`unsigned long long int`	`%llu`
`signed char`	`%c`
`unsigned char`	`%c`
`long double`	`%Lf`

3. `getchar()` & `putchar()` functions

The `getchar()` function reads a character from the terminal and returns it as an **integer**. This function reads only a single character at a time.

Here is the syntax for the `getchar()` function:

```
int getchar(void);
```

You can use this method in a loop in case you want to read more than one character.

The `putchar()` function displays the character passed to it on the screen and returns the same character. This function too displays only a single character at a time.

Here is the syntax for the `putchar()` function:

```
int putchar(int character);
```

In case you want to display more than one character, use `putchar()` method in a loop.

```
#include <stdio.h>

void main( )
{
    int c;
    printf("Enter a character");
    /*
        Take a character as input and
        store it in variable c
    */
    c = getchar();
    /*
        display the character stored
        in variable c
    */
    putchar(c);
}
```

Enter a character: Studytonight
S

When you will compile the above code, it will ask you to enter a value. When you will enter the value, it will display the value you have entered.

4. `gets()` & `puts()` functions

The `gets()` function reads a line from **stdin**(standard input) into the buffer pointed to by `str` pointer, until either a terminating newline or EOF (end of file) occurs.

Here is the syntax for the `gets()` function:

```
char* gets(char* str);
```

The `puts()` function writes the string `str` with a newline character ('\n') at the end to **stdout**. On success, a non-negative value is returned.

Here is the syntax for the `gets()` function:

```
int puts(const char* str);
```

`str` is the pointer to an array of **chars** where the C string is stored. (Don't worry if you are not able to understand this now.)

```
#include <stdio.h>

void main()
{
    /* character array of length 100 */
    char str[100];
    printf("Enter a string: ");
    gets(str);
    puts(str);
    getch();
    return 0;
}
```

Enter a string: Studytonight
Studytonight

When you will compile the above code, it will ask you to enter a string. When you will enter the string, it will display the value you have entered.

The `gets()` function is considered dangerous to use and should be avoided. We get a warning when we compile any code in which we have used `gets()` function. This is because the function doesn't know how big the buffer is, so it continues reading until it finds a newline or encounters EOF, and may overflow the bounds of the buffer it was given. We can use alternatives to `gets()` function, like the `fgets()` function.

The `fgets()` function

Here is the syntax for the `fgets()` function:

```
char* fgets(char* str, int num, FILE* stream);
```

It reads characters and stores them as a C string into `str` until **(num-1)** characters have been read or either a newline or the end-of-file is reached, whichever happens first. A newline character makes `fgets` stop reading, but it is considered a valid character by the function and included in the string copied to `str`. On success, the function returns the `str`.

For example,

```
#include <stdio.h>
#define MAX 10
int main()
{
    char str[MAX];
    fgets(str, MAX, stdin);
    printf("The string is: %s", str);
    return 0;
}
```

abcdefghijkl
The string is: abcdefghij

Difference between `scanf()` and `gets()`

The main difference between these two functions is that `scanf()` **stops reading characters when it encounters a space**, but `gets()` **reads space as a character too**.

If you enter a name as **Study Tonight** using `scanf()` it will only read and store **Study** and will leave the part of the string after space. But `gets()` function will read it completely.

Let's see the `scanf()` function in action:

```
#include <stdio.h>
int main() {
   // using scanf()
   char n1[50], n2[50];
   printf("Please enter n1: ");
   scanf("%s", n1);

   printf("You entered: %s", n1);
   return 0;
}
```

Please enter n1: study tonight
You entered: study

Now let's see the `gets()` function in action:

```
#include <stdio.h>
int main() {
   // using scanf()
   char n1[50], n2[50];
   printf("Please enter n1: ");
   gets(n1);

   printf("You entered: %s", n1);
   return 0;
}
```

Please enter n1: study tonight
You entered: study tonight

Conclusion

In this tutorial we learned about Input and Output in the C language. With many code examples, we understood how to use `printf()` function and `scanf()` function to show output to user and take input from the user.

We also learned about a few other functions used for showing output and taking user input.

C Operators (with Live Examples)

studytonight.com/c/operators-in-c.php

The C language supports a rich set of built-in operators. An **operator** is a **symbol** that tells the compiler to perform a certain mathematical or logical operations, based on the values provided to the operator.

Operators are used in programs to manipulate data and variables.

Before moving forward with Operators in C language, we recommend you learn about C variables and datatypes:

- C Variables
- C Literals
- C Datatypes
- Using C Datatypes (Examples)
- Compile and run C Program

C Operators

C operators can be classified into the following types:

- Arithmetic operators
- Relational operators
- Logical operators
- Bitwise operators
- Assignment operators
- Conditional operators
- Special operators

Let's understand each one of these operator types, one by one with working code examples.

What is an Operand?

An operand is a value on which any operator works. For example, when we say **4+5**, here, numbers **4** and **5** are operands whereas + is an operator.

Different operators work with different numbers of operands like the `+` operator requires two operands or values.

C Arithmetic Operators

The C language supports all the basic arithmetic operators such as **addition**, **subtraction**, **multiplication**, **division**, etc.

The following table shows all the basic arithmetic operators along with their descriptions.

Operator	Description	Example (where `a` and `b` are variables with some integer value)
`+`	adds two operands (values)	`a+b`
`-`	subtract second operands from first	`a-b`
`*`	multiply two operands	`a*b`
`/`	divide numerator by the denominator, i.e. divide the operand on the left side with the operand on the right side	`a/b`
`%`	This is the **modulus operato**r, it returns the remainder of the division of two operands as the result	`a%b`
`++`	This is the **Increment operator** - increases integer value by one. This operator needs only a **single operand**.	`a++` or `++a`
`--`	This is the **Decrement operator** - decreases integer value by one. This operator needs only a **single operand**.	`--b` or `b--`

To learn in what order the arithemtic operators are executed, visit C Operator Precedence.

Example: Basic Arithmetic Operators

Let's see a code example to understand the use of the basic arithmetic operators in C programs.

```
#include <stdio.h>

int main() {

    int a = 50, b = 23, result;

    // addition
    result = a+b;
    printf("Addition of a & b = %d \n",result);

    // subtraction
    result = a-b;
    printf("Subtraction of a & b = %d \n",result);

    // multiplication
    result = a*b;
    printf("Multiplication of a & b = %d \n",result);

    // division
    result = a/b;
    printf("Division of a & b = %d \n",result);

    return 0;

}
```

Addition of a & b = 73
Subtraction of a & b = 27
Multiplication of a & b = 1150
Division of a & b = 2

Run Code →

Example: Using Modulus Operator (`%`)

The modulus operator returns the remainder value after the division of the provided values.

```
#include <stdio.h>

int main() {

   int a = 23, b = 20, result;

   // Using Modulus operator
   result = a%b;

   printf("result = %d",result);

   return 0;

}
```

result = 3

Run Code →

Example: Using Increment and Decrement Operators

The increment operator is used to increase the value of any numeric value by 1, whereas the decrement operator is used to decrease the value of any numeric value by 1.

```
#include <stdio.h>

int main() {

   int a = 10, b = 20, c, d;

   /*
      Using increment operator
   */
   printf("Incrementing value of a = %d \n", ++a);

   /*
      Using decrement operator
   */
   printf("Decrementing value of b = %d \n", --b);

   // first print value of a, then decrement a
   printf("Decrementing value of a = %d \n", a--);
   printf("Value of a = %d \n", a);

   // first print value of b, then increment b
   printf("Incrementing value of b = %d \n", b++);
   printf("Value of b = %d \n", b);

   return 0;

}
```

Incrementing value of a = 11
Decrementing value of b = 19
Decrementing value of a = 11
Value of a = 10
Incrementing value of b = 19
Value of b = 20

In the code example above, we have used the increment operator as `++a` and `b++`, while the decrement operator as `--b` and `a--`.

When we use the increment and decrement operator as a **prefix** (means before the operand), then first the increment operation is done and that value is used, like in the first two `printf()` functions, we get the updated values of `a` and `b`.

Whereas when we use the increment and decrement operators as **postfix** (means after the operand), then first the larger expression is evaluated which is `printf()` in this case and then the value of the operand is updated.

C Relational operators

The relational operators (or **comparison** operators) are used to check the relationship between two operands. It checks whether two operands are **equal** or **not equal** or **less than** or **greater than**, etc.

It returns **1** if the relationship checks **pass**, otherwise, it returns **0**.

For example, if we have **two numbers 14 and 7**, if we say **14 is greater than 7**, this is **true**, hence this check will **return 1** as the result with relationship operators. On the other hand, if we say **14 is less than 7**, this is **false**, hence it will **return 0**.

The following table shows all relational operators supported in the C language.

Operator	Description	Example (`a` and `b`, where `a` = 10 and `b` = 11)
==	Check if two operands are equal	`a == b`, returns 0
!=	Check if two operands are not equal.	`a != b`, returns 1 because `a` is not equal to `b`
>	Check if the operand on the left is greater than the operand on the right	`a > b`, returns 0
<	Check operand on the left is smaller than the right operand	`a < b`, returns 1
>=	check left operand is greater than or equal to the right operand	`a >= b`, returns 0
<=	Check if the operand on left is smaller than or equal to the right operand	`a <= b`, returns 1

To learn in what order the relational operators are executed, visit C Operator Precedence.

Example: Relational Operators

When we use relational operators, then based on the result of the comparison done, if it's **true**, then the output is **1** and if it's **false**, then the output is **0**. We will see the same in the example below.

```
#include <stdio.h>

int main() {

   int a = 10, b = 20, result;

   // Equal
   result = (a==b);
   printf("(a == b) = %d \n",result);

   // less than
   result = (a<b);
   printf("(a < b) = %d \n",result);

   // greater than
   result = (a>b);
   printf("(a > b) = %d \n",result);

   // less than equal to
   result = (a<=b);
   printf("(a <= b) = %d \n",result);

   return 0;

}
```

(a == b) = 0
(a < b) = 1
(a > b) = 0
(a <= b) = 1

Run Code →

In the code example above, `a` has value **10**, and `b` has value **20**, and then different comparisons are done between them.

In the C language, **true** is any value **other than zero**. And **false** is **zero**.

C Logical Operators

C language supports the following 3 logical operators.

Operator	Description	Example (`a` and `b`, where `a` = 1 and `b` = 0)

Operator	Description	Example (`a` and `b`, where `a` = 1 and `b` = 0)
`&&`	Logical AND	`a && b`, returns 0
`||`	Logical OR	`a || b`, returns 1
`!`	Logical NOT	`!a`, returns 0

These operators are used to perform logical operations and used with conditional statements like C if-else statements.

1. With **AND** operator, only **if both operands are true**, the **result is true**.
2. With the **OR** operator, if a **single operand is true**, then the **result will be true**.
3. The **NOT** operator **changes true to false**, and **false to true**.

Example: Logical Operators

In the code example below, we have used the logical operators.

```
#include <stdio.h>

int main() {

   int a = 1, b = 0, result;

   // And
   result = (a && b);
   printf("a && b = %d \n",result);

   // Or
   result = (a || b);
   printf("a || b = %d \n",result);

   // Not
   result = !a;
   printf("!a = %d \n",result);

   return 0;

}
```

(a && b) = 0
(a || b) = 1
(!a) = 0

Run Code →

Bitwise Operators in C

Bitwise operators perform manipulations of data at the bit level. These operators also perform the **shifting of bits from right to left**. Bitwise operators are not applied to `float` or `double`, `long double`, `void`, etc.(Learn about C float and double datatype).

The following table contains the bitwise operators. There are **6 bitwise operators** in the C language.

Operator	Description	Example
`&`	Bitwise AND	
`|`	Bitwise OR	
`^`	Bitwise Exclusive OR (XOR)	
`~`	One's complement (NOT)	
`>>`	Shift right	
`<<`	Shift left	

The bitwise AND, OR, and NOT operator works the same way as the Logical AND, OR, and NOT operators, except that the bitwise operators work **bit by bit**.

Below we have a **truth table** for showing how these operators work with different values.

a	b	a & b	a | b	a ^ b
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Bitwise operators can produce any arbitrary value as result. It is not mandatory that the result will either be 0 or 1.

Bitwise `>>` and `<<` operators

The bitwise shift operator **shifts the bit value**, either to the left or right. The **left operand** specifies the **value to be shifted** and the **right operand** specifies the **number of positions** that the bits in the value have to be shifted. Both operands have the same precedence.

Understand, how bits shift from left to right and vice versa.

```
a = 00010000
b = 2
a << b = 01000000
a >> b = 00000100
```

In case of `a << b`, **2 bits** are shifted to left in **00010000** and additional zeros are added to the opposite end, that is right, hence the value becomes **01000000**

And for `a >> b`, **2 bits** are shifted from the right, hence two zeros are removed from the right and two are added on the left, hence the value becomes **00000100**

Please note, shift doesn't work like rotating, which means, the bits shifted are not added at the other end. The **bits that are shifted are lost**.

Bitwise One's Complement (`~`) Operator

The one's complement operator, will change all the 1's in the operand to 0, and all the 0's are set to 1.

For example, if the orginal byte is **00101100**, then after one's complement it will become **11010011**.

Example: Bitwise Left & Right shift Operators

Let's see an example to understand the bitwise operators in C programs.

```
#include <stdio.h>

int main() {

   int a = 0001000, b = 2, result;

   // <<
   result = a<<b;
   printf("a << b = %d \n",result);

   // >>
   result = a>>b;
   printf("a >> b = %d \n",result);

   return 0;

}
```

a << b = 2048
a >> b = 128

Run Code →

C Assignment Operators

The sssignment operators are used to assign value to a variable. For example, if we want to assign a value **10** to a variable `x` then we can do this by using the assignment operator like: `x = 10;` Here, `=` (equal to) operator is used to assign the value.

In the C language, the `=` (equal to) operator is **used for assignment** however it has several other variants such as `+=` , `-=` to combine two operations in a single statement.

You can see all the assignment operators in the table given below.

Operator	Description	Example (`a` and `b` are two variables, with where a=10 and b=5)
`=`	assigns values from right side operand to left side operand	`a=b` , `a` gets value 5
`+=`	adds right operand to the left operand and assign the result to left operand	`a+=b` , is same as `a=a+b` , value of `a` becomes 15
`-=`	subtracts right operand from the left operand and assign the result to left operand	`a-=b` , is same as `a=a-b` , value of `a` becomes 5
`*=`	mutiply left operand with the right operand and assign the result to left operand	`a*=b` , is same as `a=a*b` , value of `a` becomes 50
`/=`	divides left operand with the right operand and assign the result to left operand	`a/=b` , is same as `a=a/b` , value of `a` becomes 2
`%=`	calculate modulus using two operands and assign the result to left operand	`a%=b` , is same as `a=a%b` , value of `a` becomes 0

When we combine the arithmetic operators with the assignment operator `=` , then we get the **shorthand form** of all the arthmetic operators.

Example: Using Assignment Operators

Below we have a code example in which we have used all the different forms of assignment operator, starting from the basic assignment.

```
#include <stdio.h>

int main() {

   int a = 10;

   // Assign
   int result = a;
   printf("result = %d \n",result);

   // += operator
   result += a;
   printf("result = %d \n",result);

   // -= operator
   result -= a;
   printf("result = %d \n",result);

   // *= operator
   result *= a;
   printf("result = %d \n",result);

   return 0;

}
```

result = 10
result = 20
result = 10
result = 100

Run Code →

C Ternary Operator (`?`)

The ternary operator, also known as the conditional operators in the C language can be used for statements of the form if-then-else.

The **basic syntax** for using ternary operator is:

```
(Expression1)? Expression2 : Expression3;
```

Here is how it works:

- The question mark `?` in the syntax represents the if part.
- The first expression (expression 1) returns either **true** or **false**, based on which it is decided whether (expression 2) will be executed or (expression 3)
- If (expression 1) returns **true** then the (expression 2) is executed.

- If (expression 1) returns **false** then the expression on the right side of `:` i.e (expression 3) is executed.

Example: Using Ternary Operator

Here is a code example to show how to use ternary operator.

```
#include <stdio.h>

int main() {

   int a = 20, b = 20, result;

   /* Using ternary operator
      - If a == b then store a+b in result
      - otherwise store a-b in result
   */
   result = (a==b)?(a+b):(a-b);

   printf("result = %d",result);

   return 0;

}
```

result = 40

Run Code →

C Special Operator - `&`, `*`, `sizeof`, etc.

Apart from arithmetic, relational, logical, assignment, etc. operators, C language uses some other operator such as:

1. `sizeof` operator
2. `&` operator
3. `*` operator
4. The `.` (dot) and `->` (arrow) operators
5. `[]` operator, etc.

`sizeof` to find size of any entity(variable, array, etc.), `&` operator to find address of a variable, etc. You can see a list of such operators in the below table.

Operator	Description	Example

Operator	Description	Example
`sizeof`	returns the **size(length in bytes) of entity**, for eg. a variable or an array, etc.	`sizeof(x)` will return size of the variable `x`
`&`	returns the memory address of the variable	`&x` will return address of the variable `x`
`*`	represents pointer to an object. The `*` operator returns the value stored at a memory address.	`m = &x` (memory address of variable `x`) `*m` will return the value stored at memory address `m`
`.` (dot) operator	used to access individual elements of a C structure or C union.	If `emp` is a structure with an element `int age` in it, then `emp.age` will return the value of age.
`->` (arrow) operator	used to access structure or union elements using a pointer to structure or union.	If `p` is a pointer to the `emp` structure, then we can access `age` element using `p->age`
`[]` operator	used to access array elements using indexing	if `arr` is an array, then we can access its values using `arr[index]` , where `index` represents the array index starting from zero

We will learn about `*` , dot operator, arrow operator and `[]` operator as we move on in this tutorial series, for now let's see how to use the `sizeof` and `&` operators.

Example: Using `sizeof` and `&` Operators

Here is a code example, try running in the live code compiler using the **Run code** button.

```
#include <stdio.h>

int main() {

   int a = 20;
   char b = 'B';
   double c = 17349494.249324;

   // sizeof operator
   printf("Size of a is: %d \n", sizeof(a));
   printf("Size of b is: %d \n", sizeof(b));
   printf("Size of c is: %d \n", sizeof(c));

   // & operator
   printf("Memory address of a: %d \n", &a);

   return 0;

}
```

Size of a is: 4
Size of b is: 1
Size of c is: 8
Memory address of a: 1684270284

Run Code →

Frequently Asked Questions

Here are some frequently asked questions for C operators.

Q1. What are operators in C?

Operators are symbols known to the C compiler, which are used to perform operations on data. Operators can be used to perform operation directly on some value(C Literals) or on C variables. In the C language we can perform arithmetic operations, logical and relational operations, bitwise operations, etc. using the C operators on data.

Q2. What are the different types of operators C supports?

The C language supports the following type of operators:

- Arithmetic operators
- Relational operators
- Logical operators
- Bitwise operators

- Assignment operators
- Ternary operator
- Special operators like `sizeof`, `&`, `*`, `.` (dot) operator, `->` (arrow) operator, `[]` (square bracket) operator, etc.

Q3. What does `*` operator do in C?

The * operator in the C language is a unary operator that returns the value of the object located at the **address**, specified after the `*` operator. For example `q = *m` will store the value stored at memory address `m` in the `q` variable, if `m` contains a memory address.

The `*` operator is also used to perform **multiplication** of two values, where it acts as an **arithmetic operator**.

Q4. What does `!=` mean in C?

It is a symbol of not equal to(`!=`) operator and used to check whether two values are not equal to each other or not. It is a **relational operator** and its opposite operator is an equal(`==`) operator which is used to check equality between two values or variables.

If two values are not equal, then we will get **1** as the result of the comparison.

Q5. What is `&` and `*` operators in C?

Both are special types of operators and are used to perform memory-related operations. The `&` operator is used to get the **address of a variable** and the `*` operator is the **complement** of the `&` operator and is used to **get the value of the object** for located at a memory address.

Q6. What does %d do in C?

It is a **format specifier** that is used to print formatted output to the console. In the C language, it is used with the `printf()` function(C Input Output) to display integer value to the console. To print `float`, C provides `%f`, for `char` we use `%c`, for `double` we use `%lf`, etc.

Conclusion:

In this tutorial, we learned all about the C language operators in detail along with a lot of code examples to see all the operators in action.

Decision making in C

studytonight.com/c/decision-making-in-c.php

Decision making is about deciding the order of execution of statements based on certain conditions or repeat a group of statements until certain specified conditions are met. C language handles decision-making by supporting the following statements,

- `if` statement
- `switch` statement
- conditional operator statement (`? :` operator)
- `goto` statement

Decision making with `if` statement

The `if` statement may be implemented in different forms depending on the complexity of conditions to be tested. The different forms are,

1. Simple `if` statement
2. `if....else` statement
3. Nested `if....else` statement
4. Using `else if` statement

Simple `if` statement

The general form of a simple `if` statement is,

```
if(expression)
{
    statement inside;
}
    statement outside;
```

If the *expression* returns true, then the **statement-inside** will be executed, otherwise **statement-inside** is skipped and only the **statement-outside** is executed.

Example:

```
#include <stdio.h>

void main( )
{
    int x, y;
    x = 15;
    y = 13;
    if (x > y )
    {
        printf("x is greater than y");
    }
}
```

x is greater than y

`if...else` statement

The general form of a simple `if...else` statement is,

```
if(expression)
{
    statement block1;
}
else
{
    statement block2;
}
```

If the *expression* is true, the **statement-block1** is executed, else **statement-block1** is skipped and **statement-block2** is executed.

Example:

```
#include <stdio.h>

void main( )
{
    int x, y;
    x = 15;
    y = 18;
    if (x > y )
    {
        printf("x is greater than y");
    }
    else
    {
        printf("y is greater than x");
    }
}
```

y is greater than x

Nested `if....else` statement

The general form of a nested `if...else` statement is,

```
if( expression )
{
    if( expression1 )
    {
        statement block1;
    }
    else
    {
        statement block2;
    }
}
else
{
    statement block3;
}
```

if *expression* is false then **statement-block3** will be executed, otherwise the execution continues and enters inside the first `if` to perform the check for the next `if` block, where if *expression 1* is true the **statement-block1** is executed otherwise **statement-block2** is executed.

Example:

```
#include <stdio.h>

void main( )
{
    int a, b, c;
    printf("Enter 3 numbers...");
    scanf("%d%d%d",&a, &b, &c);
    if(a > b)
    {
        if(a > c)
        {
            printf("a is the greatest");
        }
        else
        {
            printf("c is the greatest");
        }
    }
    else
    {
        if(b > c)
        {
            printf("b is the greatest");
        }
        else
        {
            printf("c is the greatest");
        }
    }
}
```

`else if` ladder

The general form of else-if ladder is,

```
if(expression1)
{
    statement block1;
}
else if(expression2)
{
    statement block2;
}
else if(expression3 )
{
    statement block3;
}
else
    default statement;
```

The expression is tested from the top(of the ladder) downwards. As soon as a **true** condition is found, the statement associated with it is executed.

Example :

```
#include <stdio.h>

void main( )
{
    int a;
    printf("Enter a number...");
    scanf("%d", &a);
    if(a%5 == 0 && a%8 == 0)
    {
        printf("Divisible by both 5 and 8");
    }
    else if(a%8 == 0)
    {
        printf("Divisible by 8");
    }
    else if(a%5 == 0)
    {
        printf("Divisible by 5");
    }
    else
    {
        printf("Divisible by none");
    }
}
```

Points to Remember

1. In `if` statement, a single statement can be included without enclosing it into curly braces `{ ... }`

   ```
   int a = 5;
   if(a > 4)
       printf("success");
   ```

   No curly braces are required in the above case, but if we have more than one statement inside `if` condition, then we must enclose them inside curly braces.

2. `==` must be used for comparison in the expression of `if` condition, if you use `=` the expression will always return **true**, because it performs assignment not comparison.
3. Other than **0(zero)**, all other values are considered as **true**.

   ```
   if(27)
       printf("hello");
   ```

   In above example, **hello** will be printed.

Switch statement in C

studytonight.com/c/switch-statement-in-c.php

When you want to solve multiple option type problems, for example: Menu like program, where one value is associated with each option and you need to choose only one at a time, then, `switch` statement is used.

Switch statement is a control statement that allows us to choose only one choice among the many given choices. The expression in `switch` evaluates to return an integral value, which is then compared to the values present in different cases. It executes that block of code which matches the case value. If there is no match, then **default** block is executed(if present). The general form of `switch` statement is,

```
switch(expression)
{
    case value-1:
        block-1;
        break;
    case value-2:
        block-2;
        break;
    case value-3:
        block-3;
        break;
    case value-4:
        block-4;
            break;
    default:
            default-block;
        break;
}
```

Rules for using `switch` statement

1. The expression (after switch keyword) must yield an **integer** value i.e the expression should be an integer or a variable or an expression that evaluates to an integer.
2. The case **label** values must be unique.
3. The case label must end with a colon(:)
4. The next line, after the **case** statement, can be any valid C statement.

Points to Remember

1. We don't use those expressions to evaluate switch case, which may return floating point values or strings or characters.
2. `break` statements are used to **exit** the switch block. It isn't necessary to use `break` after each block, but if you do not use it, then all the consecutive blocks of code will get executed after the matching block.

```
int i = 1;
switch(i)
{
    case 1:
        printf("A");        // No break
    case 2:
        printf("B");        // No break
    case 3:
        printf("C");
        break;
}
```

A B C

The output was supposed to be only **A** because only the first case matches, but as there is no `break` statement after that block, the next blocks are executed too, until it a `break` statement in encountered or the execution reaches the end of the `switch` block.

3. **default** case is executed when none of the mentioned case matches the `switch` expression. The default case can be placed anywhere in the `switch` case. Even if we don't include the default case, `switch` statement works.
4. Nesting of `switch` statements are allowed, which means you can have `switch` statements inside another `switch` block. However, nested `switch` statements should be avoided as it makes the program more complex and less readable.

Example of `switch` statement

```
#include<stdio.h>
void main( )
{
    int a, b, c, choice;
    while(choice != 3)
    {
        /* Printing the available options */
        printf("\n 1. Press 1 for addition");
        printf("\n 2. Press 2 for subtraction");
        printf("\n Enter your choice");
        /* Taking users input */
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                printf("Enter 2 numbers");
                scanf("%d%d", &a, &b);
                c = a + b;
                printf("%d", c);
                break;
            case 2:
                printf("Enter 2 numbers");
                scanf("%d%d", &a, &b);
                c = a - b;
                printf("%d", c);
                break;
            default:
                printf("you have passed a wrong key");
                printf("\n press any key to continue");
        }
    }
}
```

Difference between `switch` and `if`

- `if` statements can evaluate `float` conditions. `switch` statements cannot evaluate `float` conditions.
- `if` statement can evaluate relational operators. `switch` statement cannot evaluate relational operators i.e they are not allowed in `switch` statement.

Loops in C

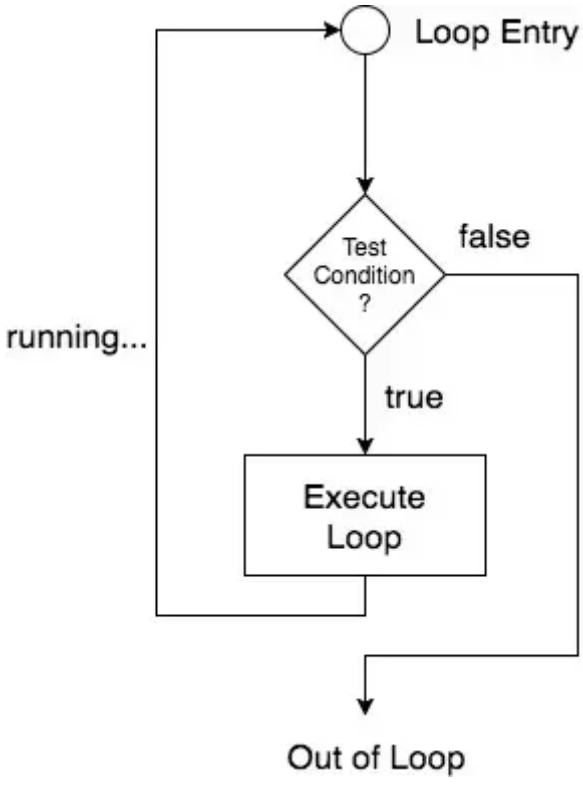
studytonight.com/c/loops-in-c.php

During programming, sometimes we might need to execute a certain code statement **again and again**. We can write the code statement as many times as we need it to execute but that would be very inefficient, because what if you want a code statement to execute a 100 times? This is why we use loops.

In any programming language including C language, loops are used to execute a single statement or a set of statements, repeatedly, until a particular condition is satisfied.

How Loops in C works?

The below diagram depicts a loop execution,



As per the above diagram, if the Test Condition is **true**, then the loop is executed, and if it is **false** then the execution breaks out of the loop. After the loop is successfully executed the execution again starts from the Loop entry and again checks for the Test condition, and this keeps on repeating.

The sequence of statements to be executed is kept inside the curly braces `{ }` known as the **Loop body**. After every execution of the loop body, **condition** is verified, and if it is found to be **true** the loop body is executed again. When the condition check returns **false**, the loop body is not executed, and execution breaks out of the loop.

Loops are broadly classified into two types:

1. Entry controlled loops

In this kind of loop, the condition is checked before executing the loop's body. So, if the condition is never true, it won't execute even once. For example, `for` and `while` loop.

2. Exit controlled loops

In this kind of loop, the condition is checked after the loop's body is executed, i.e., in the end. Hence, even if the condition is not fulfilled, this loop will execute one time. The do-while loop is an example of exit controlled loop.

Types of Loop in C

There are 3 types of Loop in C language, namely:

1. `while` loop
2. `for` loop
3. `do while` loop

---

1. `while` loop in C

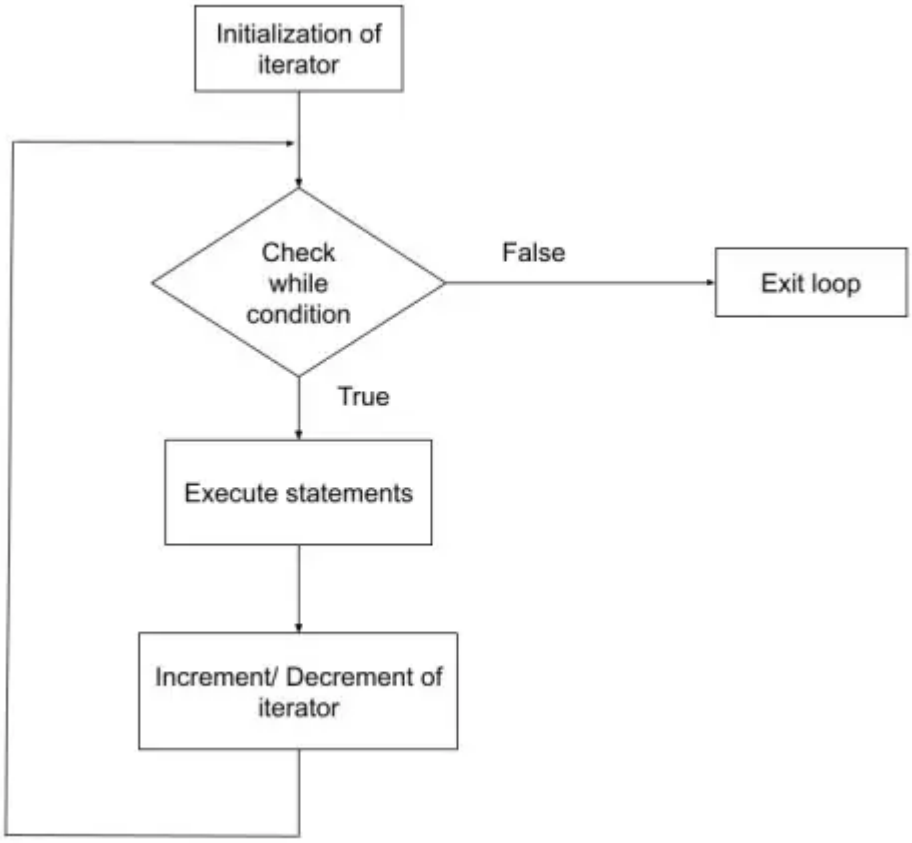
The `while` loop is an **entry controlled** loop. It is completed in 3 steps.

- Variable initialization.(e.g `int x = 0;`)
- condition(e.g `while(x <= 10)`)
- Variable increment or decrement (`x++` or `x--` or `x = x + 2`)

Syntax of `while` Loop:

```
variable initialization;
while(condition)
{
    statements;
    variable increment or decrement;
}
```

The following flowchart shows the flow of execution when we use a `while` loop.



Here, we can see that firstly, we initialize our iterator. Then we check the condition of `while` loop. If it is **false**, we exit the loop and if it is **true**, we enter the loop. After entering the loop, we execute the statements inside the `while` loop, update the iterator and then again check the condition. We do the same thing unless the condition is **false**.

Program to print your name n times using `while` loop

In this program we will use the `while` looop to print a word a given number of time.

```
#include <stdio.h>
int main()
{
    int n;
    printf("Enter the number of times you want to print your name:");
    scanf("%d", &n);
    char name[30];
    printf("Enter your name:");
    scanf("%s", name);
    while(n) {
        //here we are checking if n is non-zero
        printf("%s\n", name);
        n--;    //decrementing n
    }
    return 0;
}
```

Enter the number of times you want to print your name:3
Enter your name:studytonight
studytonight
studytonight
studytonight

Let's dry run of the above code:

Firstly, we input **n = 3**, then **name = studytonight?**.

Now, we reach the `while` loop so we check the condition; n = 3, which is nonzero, so we enter the loop. We execute the `printf()` statement and print name on the console and then decrement `n`, so now n = 2. We again check the condition; n = 2, which is nonzero, so we enter the loop and print name and decrement `n`. Now n = 1. We check the condition again; n is 1 which is nonzero so we again enter the loop and execute the statements. Now we have **n = 0**. We check the condition; n is zero now so we don't enter the loop. We exit the loop and start executing the statements after it.

Let's see another example.

Program to print first 10 natural numbers using `while` loop

```
#include<stdio.h>

void main( )
{
    int x;
    x = 1;
    while(x <= 10)
    {
        printf("%d\t", x);
        /* below statement means, do x = x+1, increment x by 1 */
        x++;
    }
}
```

1 2 3 4 5 6 7 8 9 10

---

2. `for` loop in C

The `for` loop in C is used to execute a set of statements repeatedly until a particular condition is satisfied. We can say it is an **open ended loop**. General format is,

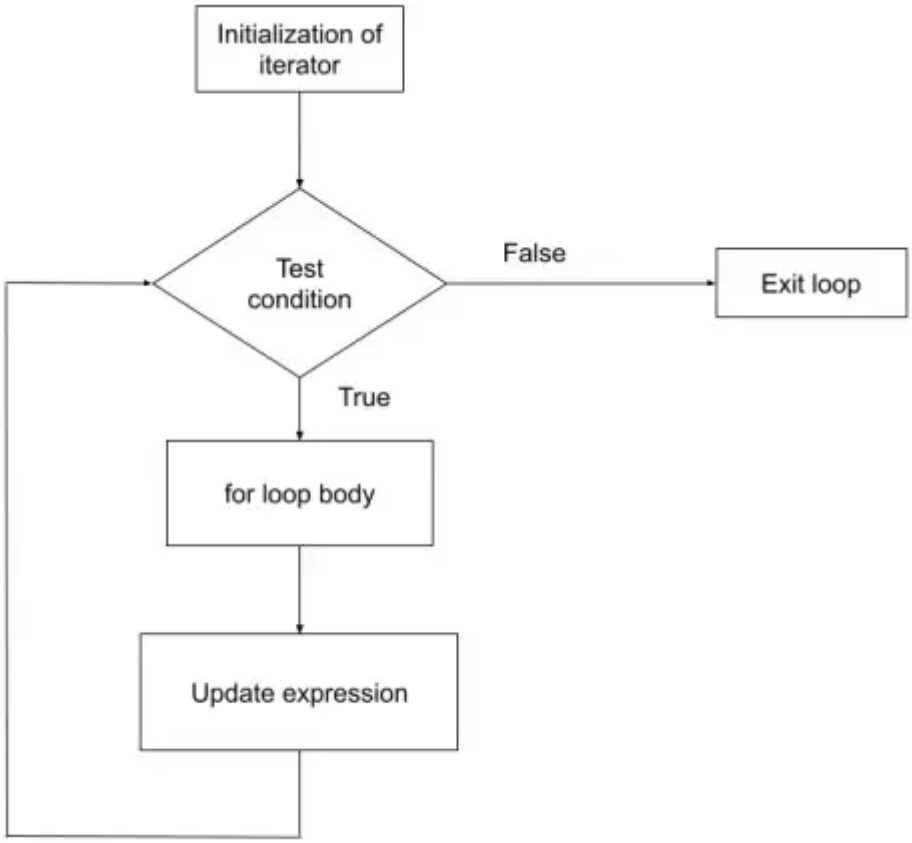
```
for(initialization; condition; increment/decrement)
{
    statement-block;
}
```

In the `for` loop in C language, we have exactly two mandatory semicolons, one after **initialization** and second after the **condition**. In this loop we can have more than one initialization or increment/decrement as well, separated using comma operator. But it can have **only one condition**.

The `for` loop is executed as follows:

1. It first evaluates the initialization code.
2. Then it checks the condition expression.
3. If it is **true**, it executes the for-loop body.
4. Then it evaluate the increment/decrement condition and again follows from step 2.
5. When the condition expression becomes **false**, it exits the loop.

Following is a flowchart explaining how the `for` loop executes.



We first initialize our iterator. Then we check the condition of the loop. If it is **false**, we exit the loop and if it is **true**, we enter the loop. After entering the loop, we execute the statements inside the `for` loop, update the iterator and then again check the condition. We do the same thing unless the test condition returns **false**.

Program to print your name n times using `for` loop

```
#include <stdio.h>
int main()
{
   int n;
   printf("Enter the number of times you want to print your name:");
   scanf("%d", &n);
   char name[25];
   printf("Enter your name:");
   scanf("%s", name);
   for(int i = 1; i <= n; i++) {    //here we are checking if n is non-zero
       printf("%s\n", name);
   }
   return 0;
}
```

Enter the number of times you want to print your name:4
Enter your name:studytonight
studytonight
studytonight

Let's dry run of the above code:

Firstly, we input n = 3, then name = studytonight.

Now, we reach the `for` loop so we initialize **i** with 1. We check the condition; 1 <= 3, so we enter the loop. We execute the `printf()` statement and print name on the console. We again reach the `for` loop. We increment **i** by 1; so now i = 2. We again check the condition; 2 <= 3, so we enter the loop and print name. Now i is incremented again to 3. We check the condition again; 3 <= 3, so we enter the loop and execute the statements. Now we have i = 4. We check the condition; 4 > 3, so we don't enter the loop. We exit the loop and start executing the statements after it.

Program to print first 10 natural numbers using `for` loop

```
#include<stdio.h>

void main( )
{
    int x;
    for(x = 1; x <= 10; x++)
    {
        printf("%d\t", x);
    }
}
```

1 2 3 4 5 6 7 8 9 10

---

3. Nested `for` loop in C

We can also have nested `for` loops, i.e one `for` loop inside another `for` loop in C language. This type of loop is generally used while working with multi-dimensional arrays. To learn more about arrays and how `for` loops are used in arrays, check out our tutorial on arrays in C. Basic syntax for nested `for` loop is,

```
for(initialization; condition; increment/decrement)
{
    for(initialization; condition; increment/decrement)
    {
        statement ;
    }
}
```

Program to print half Pyramid of numbers using Nested loops

```
#include<stdio.h>

void main( )
{
    int i, j;
    /* first for loop */
    for(i = 1; i < 5; i++)
    {
        printf("\n");
        /* second for loop inside the first */
        for(j = i; j > 0; j--)
        {
            printf("%d", j);
        }
    }
}
```

1
21
321
4321
54321

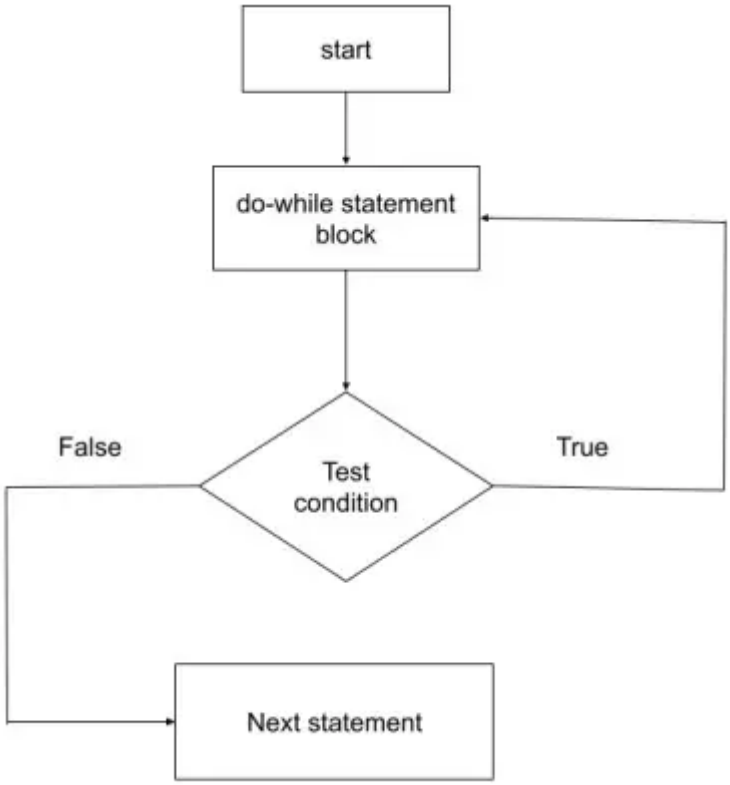
4. `do while` loop in C

In some situations it is necessary to execute body of the loop once before testing the condition. Such situations can be handled with the help of `do-while` loop. The `do` statement evaluates the body of the loop first and at the end, the condition is checked using `while` statement. It means that the body of the loop will be executed at least once, even though the starting condition inside `while` is initialized to be **false**. General syntax is,

```
do
{
    .....
    .....
}
while(condition)
```

Remember that the semicolon at the end of do-while loop is mandatory. It denotes end of the loop.

Following is the flowchart for do-while loop:



We initialize our iterator. Then we enter body of the do-while loop. We execute the statement and then reach the end. At the end, we check the condition of the loop. If it is **false**, we exit the loop and if it is **true**, we enter the loop. We keep repeating the same thing unless the condition turns **false**.

Program to print your name N times using `do-while` loop

```
#include <stdio.h>
int main()
{
    int n;
    printf("Enter the number of times you want to print your name:");
    scanf("%d", &n);
    char name[25];
    printf("\nEnter your name:");
    scanf("%s", name);
    do{
        printf("%s\n", name);
        n--;
    }while(n < 0);
    return 0;
}
```

Enter the number of times you want to print your name: 10
Enter your name: studytonight
studytonight

Run Code →

Dry run of the above code:

Firstly, we input n = 10, then name = studytonight.

Now, we enter the do-while loop because we check the condition only at the end. When we reach the end, we check the condition; n = 10, which is greater than zero, so we exit the loop and start executing the statements after it. Here we can see that even if the condition was always **false**, the loop got executed once.

Program to print first 10 multiples of 5 using `do-while` loop

```
#include<stdio.h>

void main()
{
    int a, i;
    a = 5;
    i = 1;
    do
    {
        printf("%d\t", a*i);
        i++;
    }
    while(i <= 10);
}
```

5 10 15 20 25 30 35 40 45 50

Infinite Loops in C

We come across infinite loops in our code when the compiler does not know where to stop. It does not have an exit. This means that either there is no condition to be checked or the condition is incorrect. This is why an iterator is very important in our loops. And a proper condition that ends.

Let's see a few examples of infinite loops in C:

```
#include <stdio.h>
int main()
{
    for(int i = 0; ; i++)
        printf("Infinite loop\n");
    return 0;
}
```

The above code has no condition in place, hence it will keep on executing.

```
#include <stdio.h>
int main()
{
    int i = 0;
    while(i == 0)
        printf("Infinite loop\n");
    return 0;
}
```

In the code above, we are not changing the value on `i`, hence the condition in the `while` loop will never fail.

```
#include <stdio.h>
int main()
{
    do{
        printf("Infinite loop\n");
    } while(1);
    return 0;
}
```

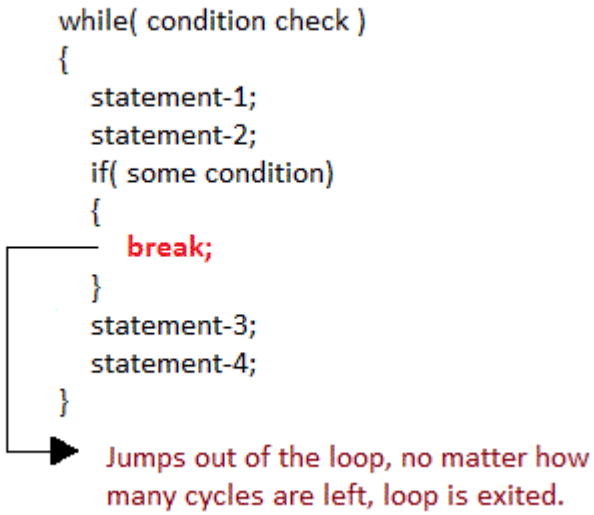
Another example, with a constant value as condition, which is always **true** hence the code will keep on executing.

Jumping Out of Loops in C

Sometimes, while executing a loop, it becomes necessary to skip a part of the loop or to leave the loop as soon as certain condition becomes **true**. This is known as jumping out of loop.

1. break statement in C

When `break` statement is encountered inside a loop, the loop is **immediately exited** and the program continues to execute with the statements after the loop.



Let's see a code example,

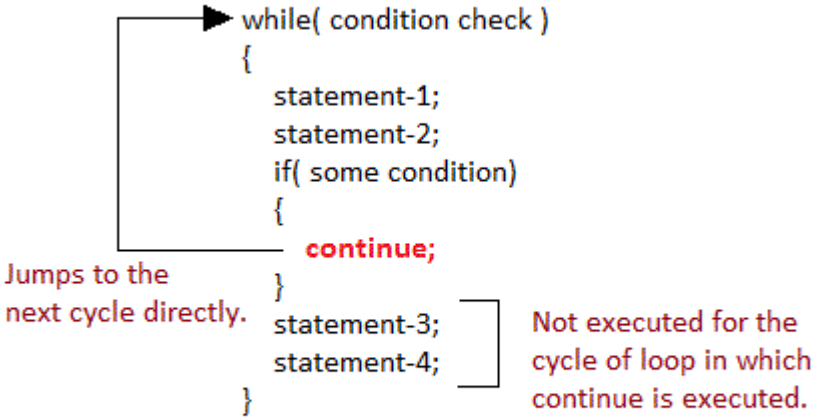
```
#include <stdio.h>
int main()
{
   int n;
   printf("Enter the number of times you want to print your name:");
   scanf("%d", &n);
   char name[25];
   printf("\nEnter your name:");
   scanf("%s", name);
   for(int i = 1; i <= n; i++) {
        if(i % 5 == 0)
            break;
        printf("%s\n", name);
   }
   return 0;
}
```

Enter the number of times you want to print your name:7
Enter your name:studytonight
studytonight
studytonight
studytonight
studytonight

In the above code, as soon as we find an index which is divisible by **5**, the loop breaks and control is shifted out of the loop.

2. continue statement in C

It causes the control to go directly to the test-condition and then continue the loop execution. On encountering `continue`, the execution leaves the current cycle of loop, and starts with the next cycle.



Let's see a code example,

```
#include <stdio.h>
int main()
{
   int n;
   printf("Enter the number of times you want to print your name:");
   scanf("%d", &n);
   char name[25];
   printf("\nEnter your name:");
   scanf("%s", name);
   for(int i = 1; i <= n; i++) {
        if(i % 2 == 0)
            continue;
        printf("%d : %s\n",i,name);
   }
   return 0;
}
```

Enter the number of times you want to print your name:5
Enter your name:studytonight
1 : studytonight
3 : studytonight
5 : studytonight

In the above example, whenever we come across an even index, we move on to the next index because of the continue statement.

Conclusion

In this tutorial, we have learned about `for` , `while` and `do-while` loops in C and why they are important, along with seeing them in action with multiple code examples. We also learnt about `break` and `continue` statements.

Arrays in C

studytonight.com/c/arrays-in-c.php

In C language, `arrays` are referred to as structured data types. An array is defined as **finite ordered collection of homogenous** data, stored in contiguous memory locations.

Here the words,

- **finite** *means* data range must be defined.
- **ordered** *means* data must be stored in continuous memory addresses.
- **homogenous** *means* data must be of similar data type.

Some uses of Arrays in C

Following are some usecases where using an array will make things simpler:

- to store list of Employee or Student names,
- to store marks of students,
- or to store list of numbers or characters etc.

Since arrays provide an easy way to represent data, it is classified amongst the data structures in C. Other data structures in c are **structure**, **lists**, **queues**, **trees** etc.

Array is a **linear data structure** which means that the elements in an array are stored in a continuous manner in the memory. This makes accessing the elements easier. Array elements are indexed in an order, starting from **0 to n-1**, where **n** is the size of the array.

Suppose we have a classroom of 50 students. Our array will consist of 50 elements with indexes from **0 to 49**, where one place will be there for every student's data.

Arrays can be **single** or **multi dimensional**. The number of dimensions in an array is equal to the number of indexing. A 2-D array (also called a matrix) has two indexes(one for row and other for column) and so on.

Arrays make our work easy because instead of declaring 100 variables, we can declare an array of size 100.

Advantages of Arrays

- In one go, we can initialise storage for more than one value. Because you can create an array of 10, 100 or 1000 values.
- They make accessing elements easier by providing random access. By random access we mean you can directly access any element in an array if you know its index.

- Sorting and searching operations are easy on arrays.

Disadvantages of Arrays

- Due to its fixed size, we cannot increase the size of an array during runtime. That means once you have created an array, then it's size cannot be changed.
- Insertion and deletion of elements can be costly, in terms of time taken.

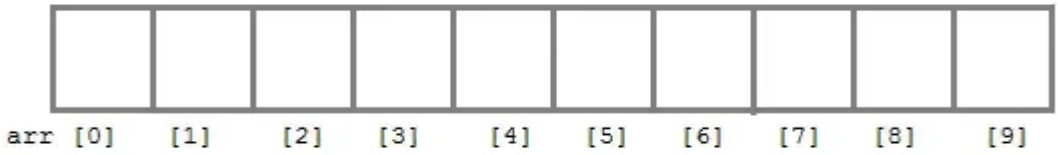
Declaring Arrays in C

Like any other variable, arrays must be declared(created) before they are used. General form of array declaration is,

```
data-type variable-name[size];
```

Let's see some code example real quick,

```
/* Example of array declaration */
char a[5];    /* char type value array */
float ar[9];  /* float type value array */
int arr[10];  /* int type value array */
```



In the code above, in the first line, `char` is the data type, `a` is the name of array and 5 is the size.

In the next line, `float` is the data type, `ar` is the name of the array and size of array is 9.

And in the last line of code, `int` is the data types, and `arr` is the name of the array and 10 is the size of array. It means array `arr` can only contain 10 elements of `int` type.

The **index** of an array starts from `0` to **size-1** i.e first element of any array will be stored at `arr[0]` address and the last element will be at `arr[size - 1]` .

Initialization of Array in C

After an array is declared it must be initialized. Otherwise, it will contain **garbage** value(any random value). An array can be initialized at either **compile time** or at **runtime**. That means, either we can provide values to the array in the code itself, or we can add user input value into the array.

Compile time Array initialization in C

Compile time initialization of array means we provide the value for the array in the code, when we create the array,

```
data-type array-name[size] = { list of values };
```

Let's see a few simple examples,

/* Here are a few examples */ int marks[4] = { 67, 87, 56, 77 }; // integer array initialization float area[5] = { 23.4, 6.8, 5.5 }; // float array initialization int marks[4] = { 67, 87, 56, 77, 59 }; // Compile time error
One important thing to remember is that when you will give more values(array elements) than the declared array size than the **compiler** will give an error.

```
#include<stdio.h>

void main()
{
    int i;
    int arr[] = {2, 3, 4};      // Compile time array initialization
    for(i = 0 ; i < 3 ; i++)
    {
        printf("%d\t",arr[i]);
    }
}
```

2 3 4

Runtime Array initialization in C

An array can also be initialized at runtime using `scanf()` function. This approach is usually used for initializing large arrays, or to initialize arrays with user specified values.

To input elements in an array, we can use a `for` loop or insert elements at a specific index.

For example, to insert element at specific index,

```
scanf("%d", &arr[3]); // will insert element at index 3, i.e. 4th position
```

Let's suppose we have an array `Arr` of **10 elements** of type `int`.

To **insert elements at each index**,

```
for(int i = 0; i < 10; i++)
    Arr[i] = i;
```

Here **i** refers to the index of the elements, starting from **0** and ending at **9** which is less than 10. In each element, we store the value of its index.

To **insert elements at each index according to user**, we can do the following,

```
for(int i = 0; i < 10; i++)
    scanf("%d", &Arr[i]);
```

To **insert elements at alternate index**,

```
for(int i = 0; i < 10; i+=2)
    scanf("%d", &Arr[i]);
```

Notice that in the above example, we are incrementing **i** by 2 each time, `i+=2` means **i = i + 2**. So we input only even indexed elements like Arrr[0], Arr[2], and so on.

Accessing Array elements in C

We already know how to access array elements. Yes, it is using the indexes. So let's see a few examples where we will print entire arrays or some specific values, etc.

To print all elements,

```
for(int i = 0; i < 10; i++)
    printf("%d", Arr[i]);
```

To **access and print elements at specified index**,

```
printf("%d", Arr[0]); //prints first element of the array
printf("%d", Arr[5]); //prints sixth element of the array
```

To access elements at alternate index,

```
for(int i = 0; i < 10; i+=2)
    printf("%d", Arr[i]);
```

If we try to access elements, on index which is more than the size of an array or less than 0, we won't get an error but we will get wrong output (some random garbage value).

Array Programs in C

Let's see a few basic programs where we will be using arrays.

1. Calculate sum of all array elements

The following program is used to calculate the sum of all elements of an array.

```
#include<stdio.h>
void main(){

    int arr[5];
    printf("Enter array elements:"");
    for(int i = 0; i < 5; i++)
        scanf("%d", &arr[i]);

    printf("Array elements are:"");
    for(int i = 0; i < 5; i++)
        printf("%d ", arr[i]);
    int sum = 0;
    for(int i = 0; i < 5; i++)
        sum += arr[i];
    printf("Sum =%d", sum);
}
```

Enter array elements:3 2 4 1 5
Array elements are:3 2 4 1 5
Sum =15

2. Copy data from one array to another

This program will copy all the elements of one array into another, we use a `for` loop.

```
#include <stdio.h>
int main()
{
    float arr1[5] = {1.5, 2.1, 3.7, 4.2, 5.5}, arr2[5];
    for(int i = 0; i < 5; i++)
        arr2[i] = arr1[i];

    for(int i = 0; i < 5; i++)
        printf("%d ", arr2[i]);
    return 0;
}
```

1.5 2.1 3.7 4.2 5.5

If you were thinking that we can assign values from an array to another using an **assignment operator**, like we do in variables, **NO you can't**.

```
Array1 = Array2;  // error
```

We can also create boolean arrays in C. Unlike other data types, boolean arrays have only two values, i.e., true(1) and false(0).

Please note that in order to use bool, we use <stdbool.h> header file in our code. Also, there is no format specifier for bool in C. We use **%d** for input/output.

Suppose a teacher wants to store the data of assignment submission of her class of 5 students. To do so, she makes use of a boolean array where she inputs "yes" or "no" corresponding to the respective roll numbers. Following is the code for this example.

```
#include <stdio.h>
#include <stdbool.h>
int main()
{

    bool submitted[5] = {'0', '1', '1', '0', '1'};

    printf("Submission details are:\n");
    for(int i = 0; i < 5; i++)
        printf("%d : %d\n",i + 1, submitted[i]);
    return 0;
}
```

1 : 0
2 : 1
3 : 1
4 : 0
5 : 1

Two dimensional Arrays

C language supports multidimensional arrays too. The simplest form of a multidimensional array is the two-dimensional array. Here, we have a row index and a column index. Both the row's and column's index begins from **0**.

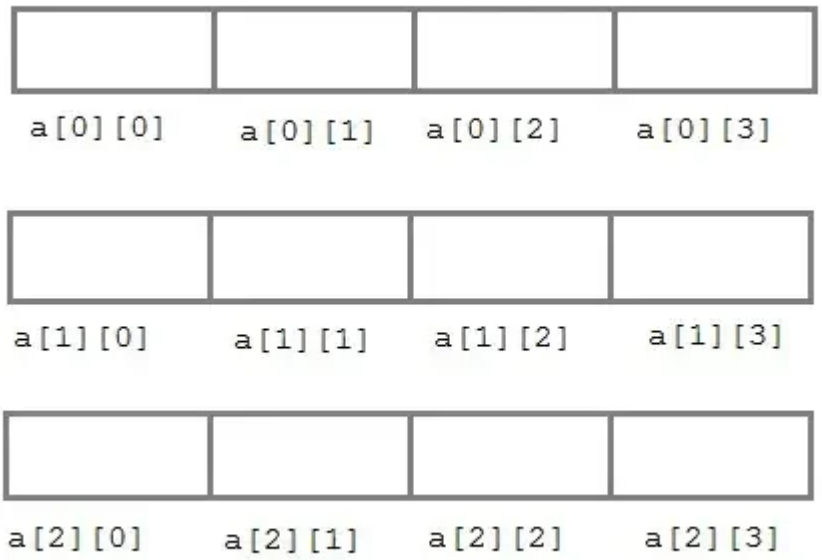
Just like a single-dimensional array, we can do Compile time initialization of two dimensional arrays or runtime initialization of two dimensional arrays.

Two-dimensional arrays are declared as follows,

data-type array-name[row-size][column-size]

Please note and remember, the first `[]` holds row count and second `[]` holds column count.

```
double arr[5][5];
int a[3][4];
```



Compile-time initialization of a two dimensional Array

If you want to do Compile time initialization of two dimensional array, then here is an example,

```
int arr[][3] = {
    {0,0,0},
    {1,1,1}
};

char a[][2] = {
{'a', 'b'},
{'c', 'd'}
};
```

Note: We have not assigned any **row value** to our arrays in the above example. It means we can initialize any number of rows. But, we must always specify **number of columns**, else it will give a compile time error. Here, a `2*3` multi-dimensional matrix is created.

Let's see a few more examples,

```
int arr1[2][2] = {1, 2, 3, 4}; // equivalent to {{1, 2},{3, 4}}
int arr2[2][3] = {1, 2, 3, 4}; // equivalent to {{1, 2, 3},{4}}
int arr3[2][4] = {1, 2, 3, 4}; // equivalent to {{1,2,3,4}}
```

As you can see in the example above, first the values are stored in columns, and then if there is any extra value, it goes to next row.

Runtime initialization of a two dimensional Array

Now let's see how we can initialize a two dimensional array at runtime.

```
#include<stdio.h>
void main()
{
    int arr[3][4];
    int i, j, k;
    printf("Enter array elements:\n");
    for(i = 0; i < 3;i++)
    {
        for(j = 0; j < 4; j++)
        {
            scanf("%d", &arr[i][j]);
        }
    }
    for(i = 0; i < 3; i++)
    {
        for(j = 0; j < 4; j++)
        {
           printf("%d", arr[i][j]);
        }
    }
}
```

Enter array elements:
1 2 3 4 5 6 7 8 9 10 11 12
1 2 3 4 5 6 7 8 9 10 11 12

Using the first `for` loop, we first access the row of the matrix(2D array) and in that particular row, we traverse each column using the inner `for` loop.

Understanding how we access elements in a 2-D array can be tricky. Let us suppose we have an array arr[2][3] with 2 rows and 3 columns.

We use **i** to access the rows of our matrix. In the outer loop(line 7), we initialize **i** as 0 which means we are accessing the first row currently. Now in this `for` loop, we have an inner loop(line 9) which iterates over **j** and is used to access columns. So for **i = 0** (the first row), we traverse across **j** (from 0 to 2). When we enter the inner loop, we have already fixed the value of **i** as 0 and now we iterate over **j** and keep incrementing the values.

So, we have **arr[0][0]**, then we increment **j** by 1 to access **arr[0][1]**, then **arr[0][2]**.

On line 12, we come out of the inner loop when **j = number of columns**. We check the condition of the outer loop. If **i = number of rows**, we exit the loop otherwise we enter it again with an incremented **i**. In the similar way, we access all elements of a matrix.

To **print the elements of different rows** in different lines,

```
for(int i = 0; i < 3; i++)
{
    for(int j = 0; j < 4; j++)
    {
         printf("%d", arr[i][j]);
    }
    printf(“\n”);
}
```

Using 2D Array in Program in C

Let's suppose we want to calculate the total marks of all students in a class. We can do it by using a matrix. Here, the number of rows will represent the number of students and the number of columns will represent the subjects.

So first we'll input marks for the first student in all 3 subjects, then the second and so on. We'll sum the marks rowwise and store it in a separate array.

```
#include <stdio.h>
int main()
{

     float marks[2][3], total[2];

     for(int i = 0; i < 2; i++) {
         for(int j = 0; j < 3; j++){
              scanf("%f", &marks[i][j]);
         }
     }

     for(int i = 0; i < 2; i++) {
         float sum = 0;
         for(int j = 0; j < 3; j++){
              sum += marks[i][j];
         }
         total[i] = sum;
     }

     for(int i = 0; i < 2; i++)
         printf("%d : %f\n", i + 1, total[i]);

     return 0;
}
```

1.2 2.3 4.5 6.7 7.8
1 : 8.000000
2 : 17.900002

Similarly, we can reverse an array, find out the maximum and minimum in an array and perform many other operations.

If you want to learn about string and char arrays, click here.

You can find more programs for array in C to practice.

String and Character Array

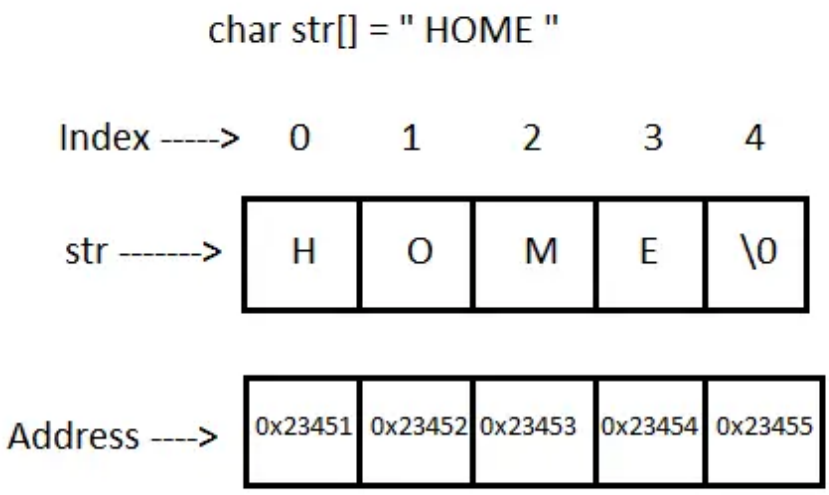
studytonight.com/c/string-and-character-array.php

String is a sequence of characters that are treated as a single data item and terminated by a null character `'\0'`. Remember that the C language does not support strings as a data type. A **string** is actually a one-dimensional array of characters in C language. These are often used to create meaningful and readable programs.

If you don't know what an array in C means, you can check the C Array tutorial to know about Array in the C language. Before proceeding further, check the following articles:

- C Function Calls
- C Variables
- C Datatypes
- C Syntax Rules

For example: The string "home" contains 5 characters including the `'\0'` character which is automatically added by the compiler at the end of the string.



Declaring and Initializing a string variables:

```
// valid
char name[13] = "StudyTonight";
char name[10] = {'c','o','d','e','\0'};

// Illegal
char ch[3] = "hello";
char str[4];
str = "hello";
```

String Input and Output:

- **%s** format specifier to read a string input from the terminal.
- But scanf() function, terminates its input on the first white space it encounters.
- **edit set conversion code %[..]** that can be used to read a line containing a variety of characters, including white spaces.
- The `gets()` function can also be used to read character string with white spaces

```
char str[20];
printf("Enter a string");
scanf("%[^\n]", &str);
printf("%s", str);

char text[20];
gets(text);
printf("%s", text);
```

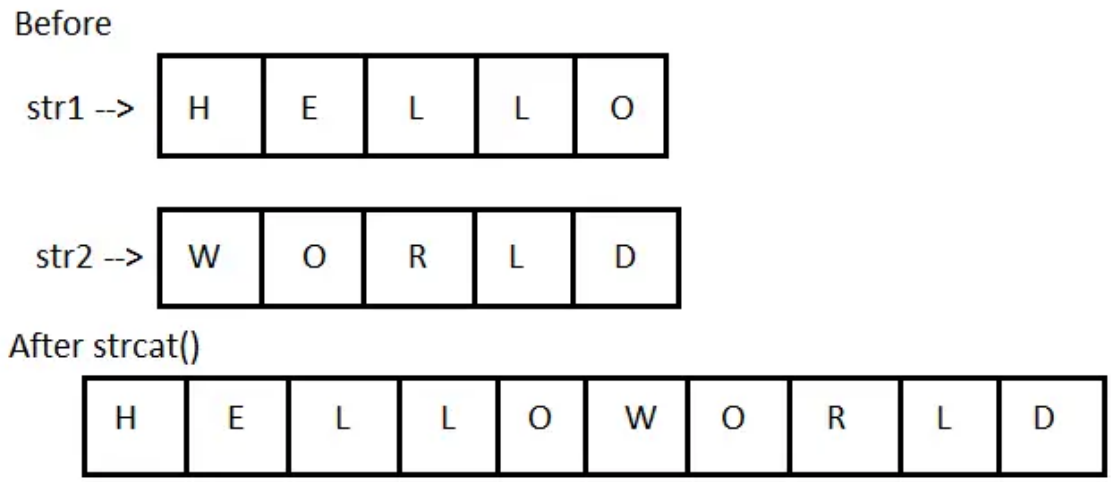
String Handling Functions:

C language supports a large number of string handling functions that can be used to carry out many of the string manipulations. These functions are packaged in the **string.h** library. Hence, you must include **string.h** header file in your programs to use these functions.

The following are the most commonly used string handling functions.

Method	Description
`strcat()`	It is used to concatenate(combine) two strings
`strlen()`	It is used to show the length of a string
`strrev()`	It is used to show the reverse of a string
`strcpy()`	Copies one string into another
`strcmp()`	It is used to compare two string

`strcat()` function in C:



Syntax:

```
strcat("hello", "world");
```

`strcat()` will add the string **"world"** to **"hello"** i.e ouput = helloworld.

`strlen()` and `strcmp()` function:

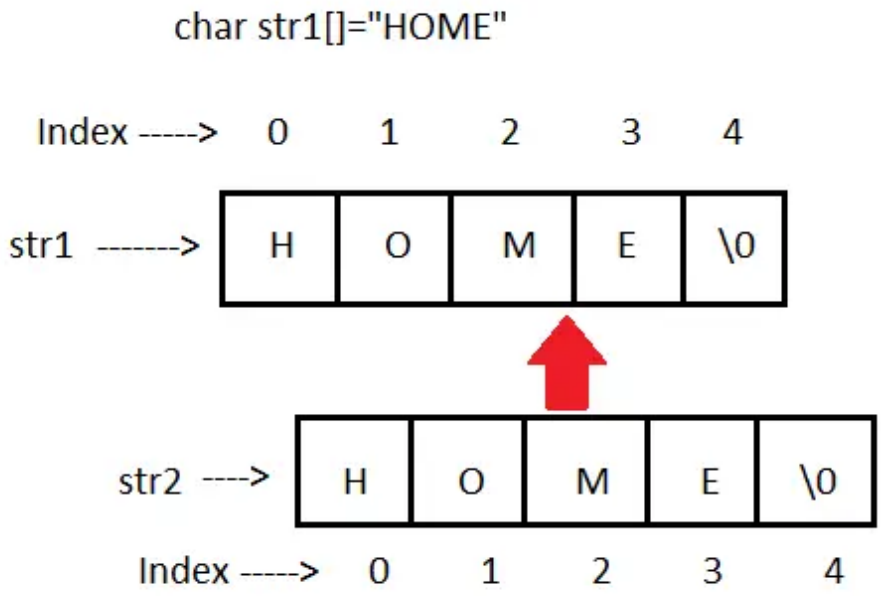
`strlen()` will return the length of the string passed to it and `strcmp()` will return the ASCII difference between first unmatching character of two strings.

```
 int j = strlen("studytonight");
 int i=strcmp("study ", "tonight");
printf("%d %d",j,i);
```

12 -1

`strcpy()` function:

It copies the second string argument to the first string argument.



Example of `strcpy()` function:

```
#include<stdio.h>
#include<string.h>

int main()
{
    char s1[50], s2[50];

    strcpy(s1, "StudyTonight");
    strcpy(s2, s1);

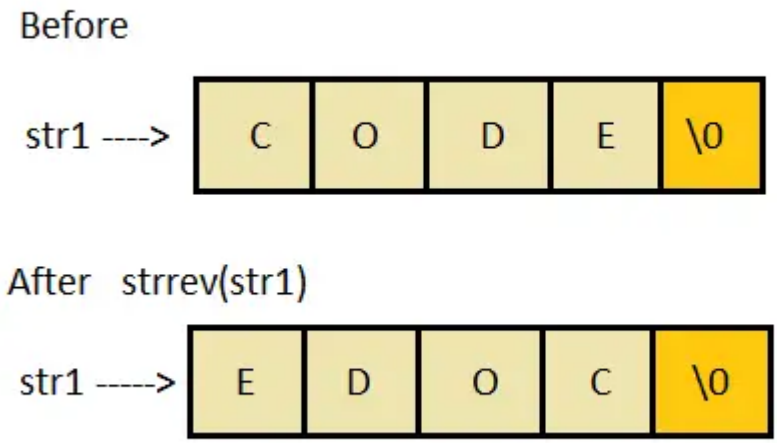
    printf("%s\n", s2);

    return(0);
}
```

StudyTonight

`strrev()` function:

It is used to reverse the given string expression.



Code snippet for `strrev()` :

```
#include <stdio.h>

int main()
{
    char s1[50];

    printf("Enter your string: ");
    gets(s1);
    printf("\nYour reverse string is: %s",strrev(s1));
    return(0);
}
```

Enter your string: studytonight
Your reverse string is: thginotyduts

Storage classes in C

studytonight.com/c/storage-classes-in-c.php

In C language, each variable has a storage class which decides the following things:

- **scope** i.e where the value of the variable would be available inside a program.
- **default initial value** i.e if we do not explicitly initialize that variable, what will be its default initial value.
- **lifetime** of that variable i.e for how long will that variable exist.

The following storage classes are most oftenly used in C programming,

1. **Automatic variables**
2. **External variables**
3. **Static variables**
4. **Register variables**

Automatic variables: `auto`

Scope: Variable defined with `auto` storage class are local to the function block inside which they are defined.

Default Initial Value: Any random value i.e garbage value.

Lifetime: Till the end of the function/method block where the variable is defined.

A variable declared inside a function without any storage class specification, is by default an **automatic variable**. They are created when a function is called and are destroyed **automatically** when the function's execution is completed. Automatic variables can also be called **local variables** because they are local to a function. By default they are assigned **garbage value** by the compiler.

```
#include<stdio.h>

void main()
{
    int detail;
    // or
    auto int details;    //Both are same
}
```

External or Global variable

Scope: Global i.e everywhere in the program. These variables are not bound by any function, they are available everywhere.

Default initial value: 0(zero).

Lifetime: Till the program doesn't finish its execution, you can access global variables.

A variable that is declared outside any function is a **Global Variable**. Global variables remain available throughout the program execution. By default, initial value of the Global variable is 0(zero). One important thing to remember about global variable is that their values can be changed by any function in the program.

```
#include<stdio.h>

int number;     // global variable

void main()
{
    number = 10;
    printf("I am in main function. My value is %d\n", number);
    fun1();     //function calling, discussed in next topic
    fun2();     //function calling, discussed in next topic
}

/* This is function 1 */
fun1()
{
    number = 20;
    printf("I am in function fun1. My value is %d", number);
}
/* This is function 1 */
fun2()
{
    printf("\nI am in function fun2. My value is %d", number);
}
```

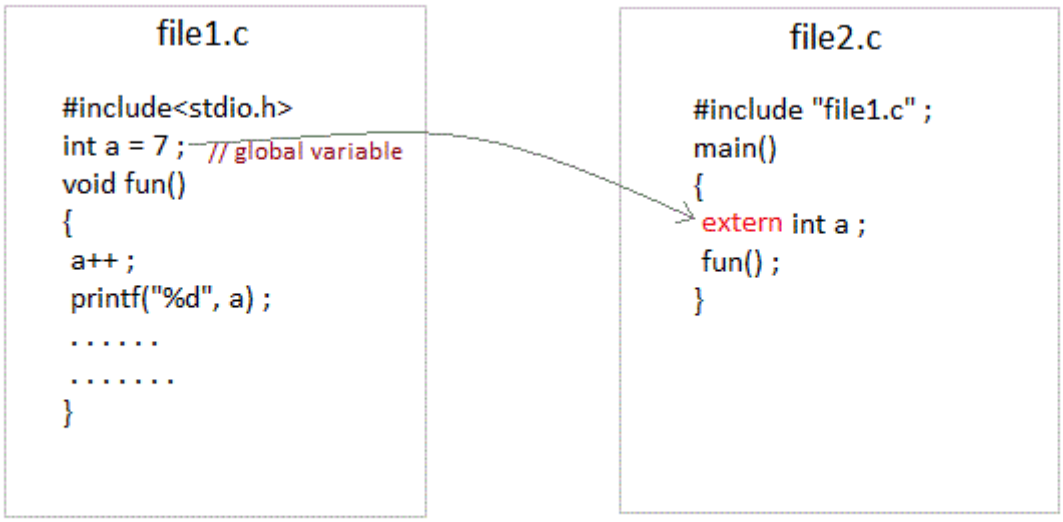
I am in function main. My value is 10 I am in function fun1. My value is 20 I am in function fun2. My value is 20

Here the global variable `number` is available to all three functions and thus, if one function changes the value of the variable, it gets changed in every function.

Note: Declaring the storage class as global or external for all the variables in a program can waste a lot of memory space because these variables have a lifetime till the end of the program. Thus, variables, which are not needed till the end of the program, will still occupy the memory and thus, memory will be wasted.

`extern` keyword

The `extern` keyword is used with a variable to inform the compiler that this variable is declared somewhere else. The `extern` declaration does not allocate storage for variables.



global variable from one file can be used in other using **extern** keyword.

Problem when extern is not used

```
int main()
{
    a = 10;     //Error: cannot find definition of variable 'a'
    printf("%d", a);
}
```

Example using extern in same file

```
int main()
{
    extern int x;   //informs the compiler that it is defined somewhere else
    x = 10;
    printf("%d", x);
}
int x;      //Global variable x
```

Static variables

Scope: Local to the block in which the variable is defined

Default initial value: 0(Zero).

Lifetime: Till the whole program doesn't finish its execution.

A `static` variable tells the compiler to persist/save the variable until the end of program. Instead of creating and destroying a variable every time when it comes into and goes out of scope, `static` variable is initialized only once and remains into existence till the end of the program. A `static` variable can either be internal or external depending upon the place of declaration. Scope of **internal static** variable remains inside the function in which it is defined. **External static** variables remain restricted to scope of file in which they are declared.

They are assigned **0 (zero)** as default value by the compiler.

```
#include<stdio.h>

void test();    //Function declaration (discussed in next topic)

int main()
{
    test();
    test();
    test();
}

void test()
{
    static int a = 0;       //a static variable
    a = a + 1;
    printf("%d\t",a);
}
```

1 2 3

Register variable

Scope: Local to the function in which it is declared.

Default initial value: Any random value i.e garbage value

Lifetime: Till the end of function/method block, in which the variable is defined.

Register variables inform the compiler to store the variable in CPU register instead of memory. Register variables have faster accessibility than a normal variable. Generally, the frequently used variables are kept in registers. But only a few variables can be placed inside registers. One application of register storage class can be in using loops, where the variable gets used a number of times in the program, in a very short span of time.

NOTE: We can never get the address of such variables.

Syntax :

```
register int number;
```

Note: Even though we have declared the storage class of our variable `number` as register, we cannot surely say that the value of the variable would be stored in a register. This is because the number of registers in a CPU are limited. Also, CPU registers are meant to do a lot of important work. Thus, sometimes they may not be free. In such scenario, the variable works as if its storage class is `auto`.

Which storage class should be used and when

To improve the speed of execution of the program and to carefully use the memory space occupied by the variables, following points should be kept in mind while using storage classes:

- We should use `static` storage class only when we want the value of the variable to remain same every time we call it using different function calls.
- We should use `register` storage class only for those variables that are used in our program very oftenly. CPU registers are limited and thus should be used carefully.
- We should use external or global storage class only for those variables that are being used by almost all the functions in the program.
- If we do not have the purpose of any of the above mentioned storage classes, then we should use the automatic storage class.

Functions in C

studytonight.com/c/user-defined-functions-in-c.php

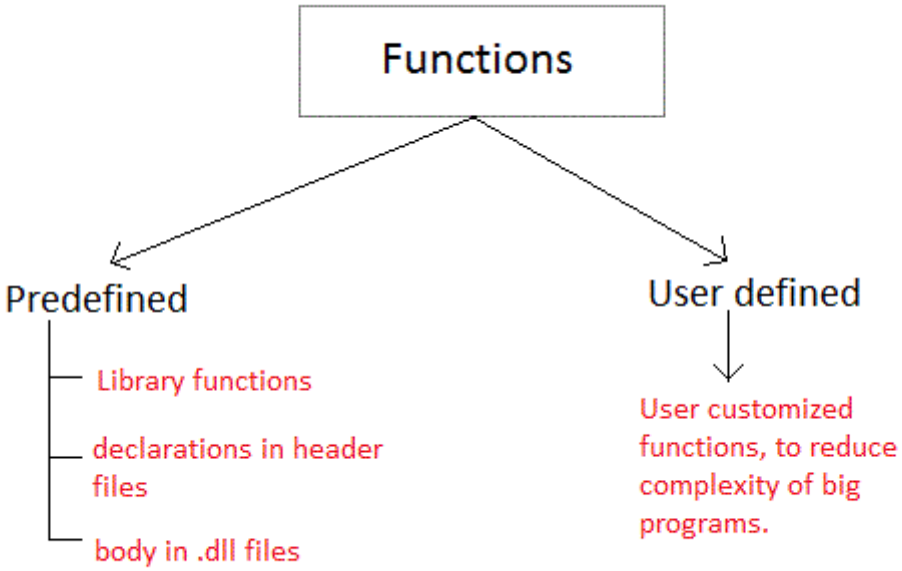
A **function** is a block of code that performs a particular task.

There are many situations where we might need to write same line of code for more than once in a program. This may lead to unnecessary repetition of code, bugs and even becomes boring for the programmer. So, C language provides an approach in which you can declare and define a group of statements once in the form of a function and it can be called and used whenever required.

These functions defined by the user are also know as **User-defined Functions**

C functions can be classified into two categories,

1. **Library functions**
2. **User-defined functions**



Library functions are those functions which are already defined in C library, example `printf()`, `scanf()`, `strcat()` etc. You just need to include appropriate header files to use these functions. These are already declared and defined in C libraries.

A **User-defined functions** on the other hand, are those functions which are defined by the user at the time of writing program. These functions are made for code reusability and for saving time and space.

Benefits of Using Functions

1. It provides modularity to your program's structure.
2. It makes your code reusable. You just have to call the function by its name to use it, wherever required.
3. In case of large programs with thousands of code lines, debugging and editing becomes easier if you use functions.
4. It makes the program more readable and easy to understand.

Function Declaration

General syntax for function declaration is,

```
returntype functionName(type1 parameter1, type2 parameter2,...);
```

Like any variable or an array, a function must also be declared before its used. Function declaration informs the compiler about the function name, parameters is accept, and its return type. The actual body of the function can be defined separately. It's also called as **Function Prototyping**. Function declaration consists of 4 parts.

- returntype
- function name
- parameter list
- terminating semicolon

returntype

When a function is declared to perform some sort of calculation or any operation and is expected to provide with some result at the end, in such cases, a `return` statement is added at the end of function body. Return type specifies the type of value(`int` , `float` , `char` , `double`) that function is expected to return to the program which called the function.

Note: In case your function doesn't return any value, the return type would be `void` .

functionName

Function name is an identifier and it specifies the name of the function. The function name is any valid C identifier and therefore must follow the same naming rules like other variables in C language.

parameter list

The parameter list declares the type and number of arguments that the function expects when it is called. Also, the parameters in the parameter list receives the argument values when the function is called. They are often referred as **formal parameters**.

Time for an Example

Let's write a simple program with a `main()` function, and a user defined function to multiply two numbers, which will be called from the `main()` function.

```
#include<stdio.h>

int multiply(int a, int b);     // function declaration

int main()
{
    int i, j, result;
    printf("Please enter 2 numbers you want to multiply...");
    scanf("%d%d", &i, &j);

    result = multiply(i, j);        // function call
    printf("The result of muliplication is: %d", result);

    return 0;
}

int multiply(int a, int b)
{
    return (a*b);       // function defintion, this can be done in one line
}
```

Function definition Syntax

Just like in the example above, the general syntax of function definition is,

```
returntype functionName(type1 parameter1, type2 parameter2,...)
{
    // function body goes here
}
```

The first line *returntype* **functionName(type1 parameter1, type2 parameter2,...)** is known as **function header** and the statement(s) within curly braces is called **function body**.

Note: While defining a function, there is no semicolon(`;`) after the parenthesis in the function header, unlike while declaring the function or calling the function.

functionbody

The function body contains the declarations and the statements(algorithm) necessary for performing the required task. The body is enclosed within curly braces `{ ... }` and consists of three parts.

- **local** variable declaration(if required).
- **function statements** to perform the task inside the function.
- a **return** statement to return the result evaluated by the function(if return type is `void`, then no return statement is required).

Calling a function

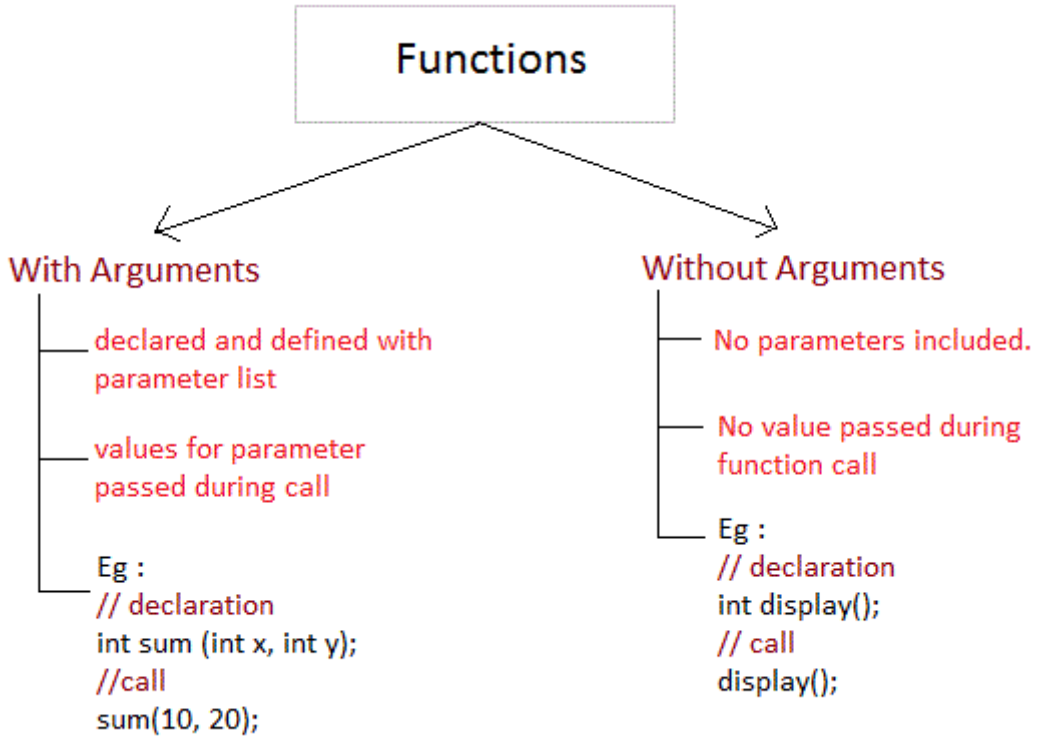
When a function is called, control of the program gets transferred to the function.

```
functionName(argument1, argument2,...);
```

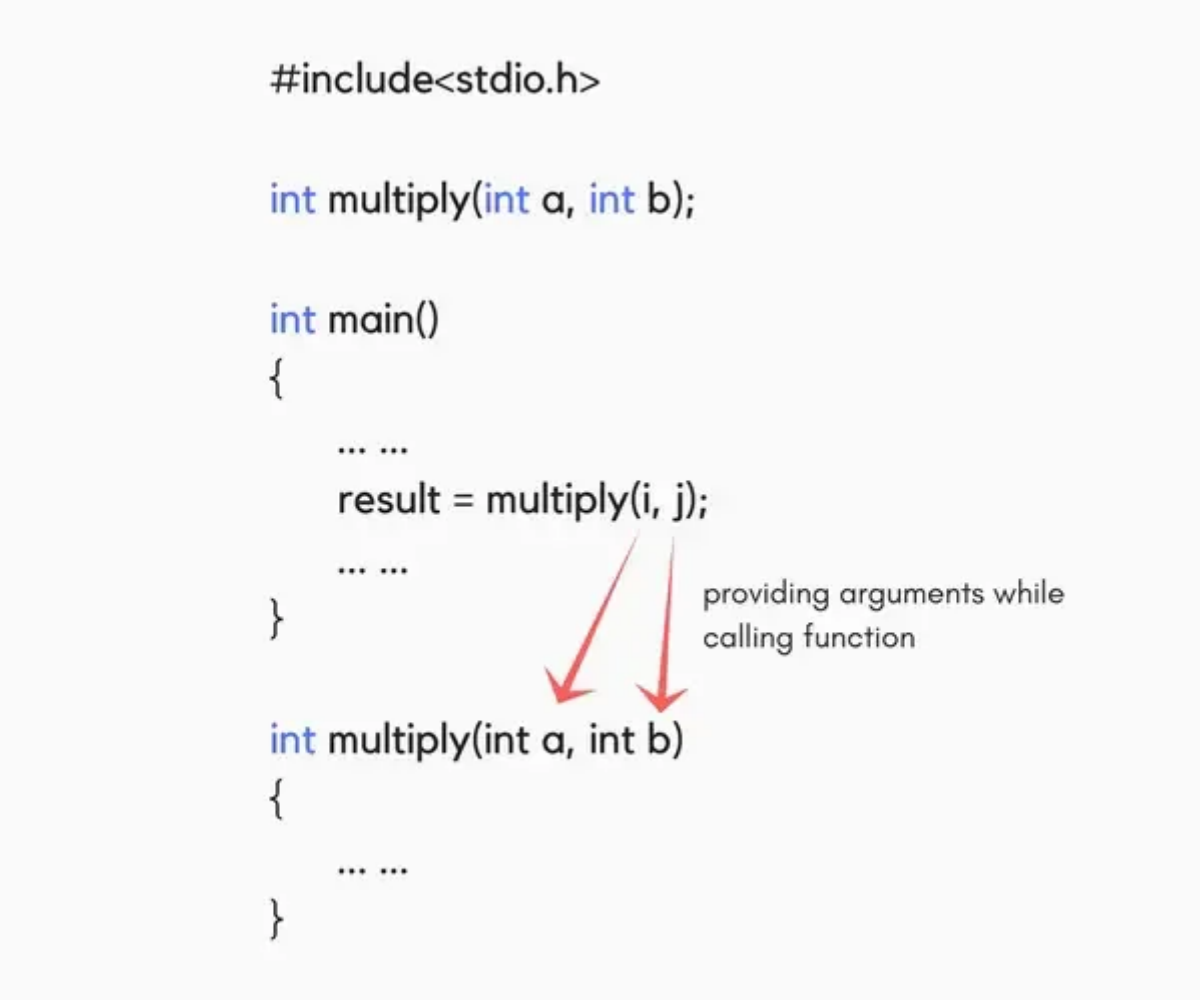
In the example above, the statement `multiply(i, j);` inside the `main()` function is function call.

Passing Arguments to a function

Arguments are the values specified during the function call, for which the formal parameters are declared while defining the function.



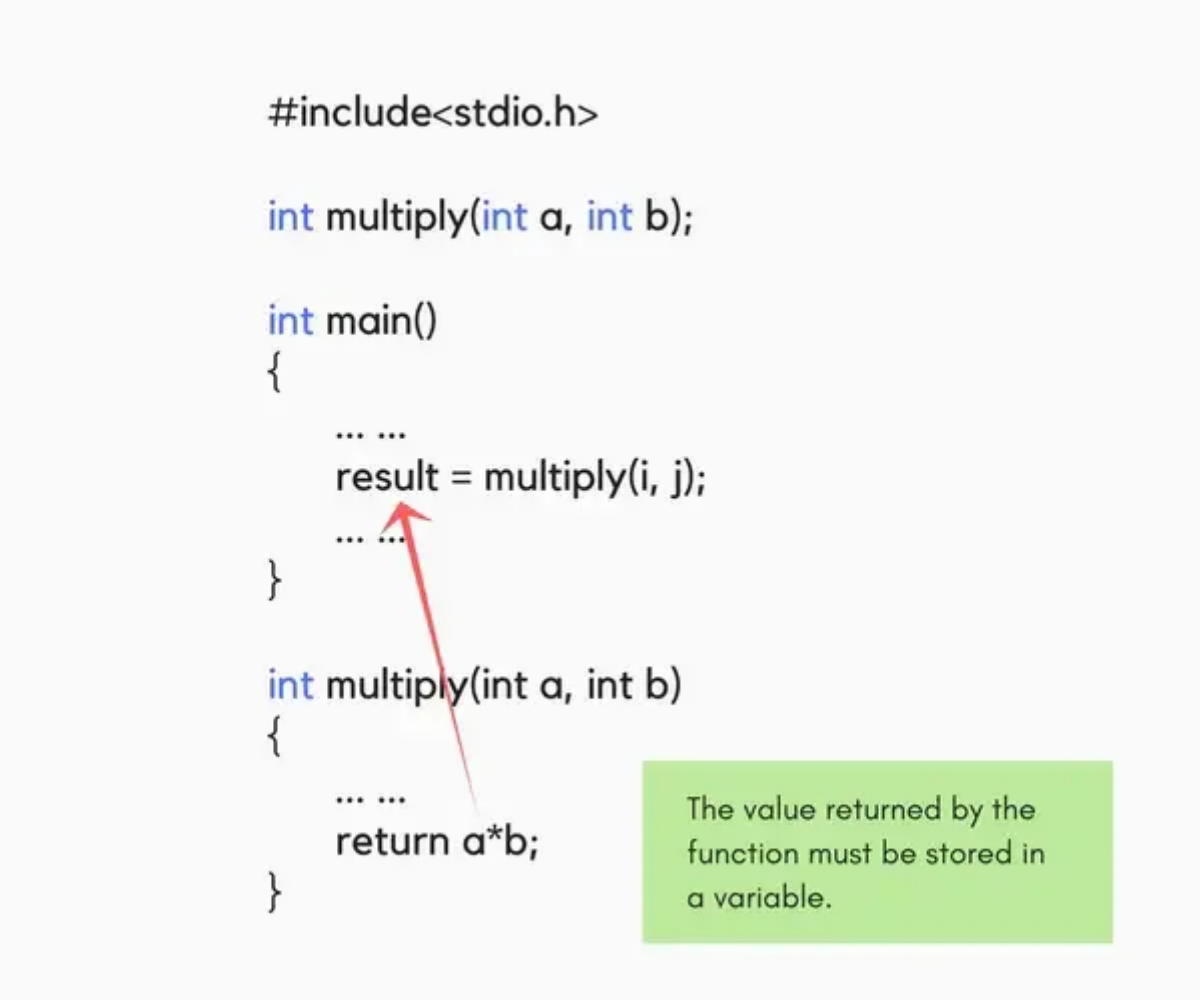
It is possible to have a function with parameters but no return type. It is not necessary, that if a function accepts parameter(s), it must return a result too.



While declaring the function, we have declared two parameters `a` and `b` of type `int`. Therefore, while calling that function, we need to pass two arguments, else we will get compilation error. And the two arguments passed should be received in the function definition, which means that the function header in the function definition should have the two parameters to hold the argument values. These received arguments are also known as **formal parameters**. The name of the variables while declaring, calling and defining a function can be different.

Returning a value from function

A function may or may not return a result. But if it does, we must use the `return` statement to output the result. `return` statement also ends the function execution, hence it must be the last statement of any function. If you write any statement after the `return` statement, it won't be executed.



The datatype of the value returned using the `return` statement should be same as the return type mentioned at function declaration and definition. If any of it mismatches, you will get compilation error.

In the next tutorial, we will learn about the different types of user defined functions in C language and the concept of Nesting of functions which is used in recursion.

Type of User-defined Functions in C

studytonight.com/c/type-of-functions-and-recursion.php

There can be 4 different types of user-defined functions, they are:

1. Function with no arguments and no return value
2. Function with no arguments and a return value
3. Function with arguments and no return value
4. Function with arguments and a return value

Below, we will discuss about all these types, along with program examples.

Function with no arguments and no return value

Such functions can either be used to display information or they are completely dependent on user inputs.

Below is an example of a function, which takes 2 numbers as input from user, and display which is the greater number.

```
#include<stdio.h>

void greatNum();       // function declaration

int main()
{
    greatNum();        // function call
    return 0;
}

void greatNum()        // function definition
{
    int i, j;
    printf("Enter 2 numbers that you want to compare...");
    scanf("%d%d", &i, &j);
    if(i > j) {
        printf("The greater number is: %d", i);
    }
    else {
        printf("The greater number is: %d", j);
    }
}
```

Function with no arguments and a return value

We have modified the above example to make the function `greatNum()` return the number which is greater amongst the 2 input numbers.

```
#include<stdio.h>

int greatNum();       // function declaration

int main()
{
    int result;
    result = greatNum();        // function call
    printf("The greater number is: %d", result);
    return 0;
}

int greatNum()        // function definition
{
    int i, j, greaterNum;
    printf("Enter 2 numbers that you want to compare...");
    scanf("%d%d", &i, &j);
    if(i > j) {
        greaterNum = i;
    }
    else {
        greaterNum = j;
    }
    // returning the result
    return greaterNum;
}
```

Function with arguments and no return value

We are using the same function as example again and again, to demonstrate that to solve a problem there can be many different ways.

This time, we have modified the above example to make the function `greatNum()` take two `int` values as arguments, but it will not be returning anything.

```
#include<stdio.h>

void greatNum(int a, int b);       // function declaration

int main()
{
    int i, j;
    printf("Enter 2 numbers that you want to compare...");
    scanf("%d%d", &i, &j);
    greatNum(i, j);        // function call
    return 0;
}

void greatNum(int x, int y)        // function definition
{
    if(x > y) {
        printf("The greater number is: %d", x);
    }
    else {
        printf("The greater number is: %d", y);
    }
}
```

Function with arguments and a return value

This is the best type, as this makes the function completely independent of inputs and outputs, and only the logic is defined inside the function body.

```
#include<stdio.h>

int greatNum(int a, int b);       // function declaration

int main()
{
    int i, j, result;
    printf("Enter 2 numbers that you want to compare...");
    scanf("%d%d", &i, &j);
    result = greatNum(i, j); // function call
    printf("The greater number is: %d", result);
    return 0;
}

int greatNum(int x, int y)        // function definition
{
    if(x > y) {
        return x;
    }
    else {
        return y;
    }
}
```

Nesting of Functions

C language also allows nesting of functions i.e to use/call one function inside another function's body. We must be careful while using nested functions, because it may lead to infinite nesting.

```
function1()
{
    // function1 body here

    function2();

    // function1 body here
}
```

If function2() also has a call for function1() inside it, then in that case, it will lead to an infinite nesting. They will keep calling each other and the program will never terminate.

Not able to understand? Lets consider that inside the `main()` function, function1() is called and its execution starts, then inside function1(), we have a call for function2(), so the control of program will go to the function2(). But as function2() also has a call to function1() in its body, it will call function1(), which will again call function2(), and this will go on for infinite times, until you forcefully exit from program execution.

What is Recursion?

Recursion is a special way of nesting functions, where a function calls itself inside it. We must have certain conditions in the function to break out of the recursion, otherwise recursion will occur infinite times.

```
function1()
{
    // function1 body
    function1();
    // function1 body
}
```

Example: Factorial of a number using Recursion

```
#include<stdio.h>

int factorial(int x);       //declaring the function

void main()
{
    int a, b;

    printf("Enter a number...");
    scanf("%d", &a);
    b = factorial(a);       //calling the function named factorial
    printf("%d", b);
}

int factorial(int x) //defining the function
{
    int r = 1;
    if(x == 1)
        return 1;
    else
        r = x*factorial(x-1);       //recursion, since the function calls
itself

    return r;
}
```

Similarly, there are many more applications of recursion in C language. Go to the programs section, to find out more programs using recursion.

Now that we have learned about the Stack in Data Structure, you can also check out these topics:

Types of Function calls in C

studytonight.com/c/types-of-function-calls.php

Functions are called by their names, we all know that, then what is this tutorial for? Well if the function does not have any arguments, then to call a function you can directly use its name. But for functions with arguments, we can call a function in two different ways, based on how we specify the arguments, and these two ways are:

1. Call by Value
2. Call by Reference

Call by Value

Calling a function by value means, we pass the values of the arguments which are stored or copied into the formal parameters of the function. Hence, the original values are unchanged only the parameters inside the function changes.

```
#include<stdio.h>

void calc(int x);

int main()
{
    int x = 10;
    calc(x);
    // this will print the value of 'x'
    printf("\nvalue of x in main is %d", x);
    return 0;
}

void calc(int x)
{
    // changing the value of 'x'
    x = x + 10 ;
    printf("value of x in calc function is %d ", x);
}
```

value of x in calc function is 20 value of x in main is 10

In this case, the actual variable `x` is not changed. This is because we are passing the argument by value, hence a copy of `x` is passed to the function, which is updated during function execution, and that copied value in the function is destroyed when the function ends(goes out of scope). So the variable `x` inside the `main()` function is never changed and hence, still holds a value of `10`.

But we can change this program to let the function modify the original `x` variable, by making the function `calc()` return a value, and storing that value in x.

```
#include<stdio.h>

int calc(int x);

int main()
{
    int x = 10;
    x = calc(x);
    printf("value of x is %d", x);
    return 0;
}

int calc(int x)
{
    x = x + 10 ;
    return x;
}
```

value of x is 20

Call by Reference

In call by reference we pass the address(reference) of a variable as argument to any function. When we pass the address of any variable as argument, then the function will have access to our variable, as it now knows where it is stored and hence can easily update its value.

In this case the formal parameter can be taken as a **reference** or a **pointers** (don't worry about pointers, we will soon learn about them), in both the cases they will change the values of the original variable.

```
#include<stdio.h>

void calc(int *p);      // functin taking pointer as argument

int main()
{
    int x = 10;
    calc(&x);       // passing address of 'x' as argument
    printf("value of x is %d", x);
    return(0);
}

void calc(int *p)       //receiving the address in a reference pointer
variable
{
    /*
        changing the value directly that is
        stored at the address passed
    */
    *p = *p + 10;
}
```

value of x is 20

NOTE: If you do not have any prior knowledge of pointers, do study Pointers first. Or just go over this topic and come back again to revise this, once you have learned about pointers.

How to pass Array to a Function in C

studytonight.com/c/array-in-function-in-c.php

Whenever we need to pass a list of elements as argument to any functions in C language, it is prefered to do so using an `array`. But how can we pass an array as argument to a function? Let's see how its done.

Declaring Function with array as a parameter

There are two possible ways to do so, one by using call by value and other by using call by reference.

1. We can either have an array as a parameter.

   ```
   int sum (int arr[]);
   ```

2. Or, we can have a pointers in the parameter list, to hold the base address of our array.

   ```
   int sum (int* ptr);
   ```

   We will study the second way in details later when we will study pointers.

Returning an Array from a function

We don't return an array from functions, rather we return a pointer holding the base address of the array to be returned. But we must, make sure that the array exists after the function ends i.e. the array is not local to the function.

```
int* sum (int x[])
{
    // statements
    return x ;
}
```

We will discuss about this when we will study pointers with arrays.

Passing arrays as parameter to function

Now let's see a few examples where we will pass a single array element as argument to a function, a one dimensional array to a function and a multidimensional array to a function.

Passing a single array element to a function

Let's write a very simple program, where we will declare and define an array of integers in our `main()` function and pass one of the array element to a function, which will just print the value of the element.

```
#include<stdio.h>

void giveMeArray(int a);

int main()
{
    int myArray[] = { 2, 3, 4 };
    giveMeArray(myArray[2]);        //Passing array element myArray[2] only.
    return 0;
}

void giveMeArray(int a)
{
    printf("%d", a);
}
```

4

Passing a complete One-dimensional array to a function

To understand how this is done, let's write a function to find out average of all the elements of the array and print it.

We will only send in the name of the array as argument, which is nothing but the address of the starting element of the array, or we can say the starting memory address.

```
#include<stdio.h>

float findAverage(int marks[]);

int main()
{
    float avg;
    int marks[] = {99, 90, 96, 93, 95};
    avg = findAverage(marks);       // name of the array is passed as argument.
    printf("Average marks = %.1f", avg);
    return 0;
}

float findAverage(int marks[])
{
    int i, sum = 0;
    float avg;
    for (i = 0; i <= 4; i++) {
        sum += marks[i];
    }
    avg = (sum / 5);
    return avg;
}
```

94.6

Passing a Multi-dimensional array to a function

Here again, we will only pass the name of the array as argument.

```
#include<stdio.h>

void displayArray(int arr[3][3]);

int main()
{
    int arr[3][3], i, j;
    printf("Please enter 9 numbers for the array: \n");
    for (i = 0; i < 3; ++i)
    {
        for (j = 0; j < 3; ++j)
        {
            scanf("%d", &arr[i][j]);
        }
    }
    // passing the array as argument
    displayArray(arr);
    return 0;
}

void displayArray(int arr[3][3])
{
    int i, j;
    printf("The complete array is: \n");
    for (i = 0; i < 3; ++i)
    {
        // getting cursor to new line
        printf("\n");
        for (j = 0; j < 3; ++j)
        {
            // \t is used to provide tab space
            printf("%d\t", arr[i][j]);
        }
    }
}
```

Please enter 9 numbers for the array: 1 2 3 4 5 6 7 8 9 The complete array is: 1 2 3 4 5 6 7 8 9

C Structures

studytonight.com/c/structures-in-c.php

Structure is a user-defined datatype in C language which allows us to combine data of different types together. Structure helps to construct a complex data type which is more meaningful. It is somewhat similar to an Array, but an array holds data of similar type only. But structure on the other hand, can store data of any type, which is practical more useful.

For example: If I have to write a program to store Student information, which will have Student's name, age, branch, permanent address, father's name etc, which included string values, integer values etc, how can I use arrays for this problem, I will require something which can hold data of different types together.

In structure, data is stored in form of **records**.

Defining a structure

`struct` keyword is used to define a structure. `struct` defines a new data type which is a collection of primary and derived data types.

Syntax:

```
struct [structure_tag]
{
    //member variable 1
    //member variable 2
    //member variable 3
    ...
}[structure_variables];
```

As you can see in the syntax above, we start with the `struct` keyword, then it's optional to provide your structure a name, we suggest you to give it a name, then inside the curly braces, we have to mention all the member variables, which are nothing but normal C language variables of different types like `int`, `float`, `array` etc.

After the closing curly brace, we can specify one or more structure variables, again this is optional.

Note: The closing curly brace in the structure type declaration must be followed by a semicolon(`;`).

Example of Structure

```
struct Student
{
    char name[25];
    int age;
    char branch[10];
    // F for female and M for male
    char gender;
};
```

Here `struct Student` declares a structure to hold the details of a student which consists of 4 data fields, namely `name` , `age` , `branch` and `gender` . These fields are called **structure elements or members**.

Each member can have different datatype, like in this case, `name` is an array of `char` type and `age` is of `int` type etc. **Student** is the name of the structure and is called as the **structure tag**.

Declaring Structure Variables

It is possible to declare variables of a **structure**, either along with structure definition or after the structure is defined. **Structure** variable declaration is similar to the declaration of any normal variable of any other datatype. Structure variables can be declared in following two ways:

---

1) Declaring Structure variables separately

```
struct Student
{
    char name[25];
    int age;
    char branch[10];
    //F for female and M for male
    char gender;
};

struct Student S1, S2;      //declaring variables of struct Student
```

---

2) Declaring Structure variables with structure definition

```
struct Student
{
    char name[25];
    int age;
    char branch[10];
    //F for female and M for male
    char gender;
}S1, S2;
```

Here `S1` and `S2` are variables of structure `Student` . However this approach is not much recommended.

Accessing Structure Members

Structure members can be accessed and assigned values in a number of ways. Structure members have no meaning individually without the structure. In order to assign a value to any structure member, the member name must be linked with the **structure** variable using a dot `.` operator also called **period** or **member access** operator.

For example:

```
#include<stdio.h>
#include<string.h>

struct Student
{
    char name[25];
    int age;
    char branch[10];
    //F for female and M for male
    char gender;
};

int main()
{
    struct Student s1;

    /*
        s1 is a variable of Student type and
        age is a member of Student
    */
    s1.age = 18;
    /*
        using string function to add name
    */
    strcpy(s1.name, "Viraaj");
    /*
        displaying the stored values
    */
    printf("Name of Student 1: %s\n", s1.name);
    printf("Age of Student 1: %d\n", s1.age);

    return 0;
}
```

Name of Student 1: Viraaj Age of Student 1: 18

We can also use `scanf()` to give values to structure members through terminal.

```
scanf(" %s ", s1.name);
scanf(" %d ", &s1.age);
```

Structure Initialization

Like a variable of any other datatype, structure variable can also be initialized at compile time.

```
struct Patient
{
    float height;
    int weight;
    int age;
};

struct Patient p1 = { 180.75 , 73, 23 };    //initialization
```

or,

```
struct Patient p1;
p1.height = 180.75;     //initialization of each member separately
p1.weight = 73;
p1.age = 23;
```

Array of Structure

We can also declare an array of **structure** variables. in which each element of the array will represent a **structure** variable. **Example :** `struct employee emp[5];`

The below program defines an array `emp` of size 5. Each element of the array `emp` is of type `Employee` .

```
#include<stdio.h>

struct Employee
{
    char ename[10];
    int sal;
};

struct Employee emp[5];
int i, j;
void ask()
{
    for(i = 0; i < 3; i++)
    {
        printf("\nEnter %dst Employee record:\n", i+1);
        printf("\nEmployee name:\t");
        scanf("%s", emp[i].ename);
        printf("\nEnter Salary:\t");
        scanf("%d", &emp[i].sal);
    }
    printf("\nDisplaying Employee record:\n");
    for(i = 0; i < 3; i++)
    {
        printf("\nEmployee name is %s", emp[i].ename);
        printf("\nSlary is %d", emp[i].sal);
    }
}
void main()
{
    ask();
}
```

Nested Structures

Nesting of structures, is also permitted in C language. Nested structures means, that one structure has another stucture as member variable.

Example:

```
struct Student
{
    char[30] name;
    int age;
    /* here Address is a structure */
    struct Address
    {
        char[50] locality;
        char[50] city;
        int pincode;
    }addr;
};
```

Structure as Function Arguments

We can pass a structure as a function argument just like we pass any other variable or an array as a function argument.

Example:

```
#include<stdio.h>

struct Student
{
    char name[10];
    int roll;
};

void show(struct Student st);

void main()
{
    struct Student std;
    printf("\nEnter Student record:\n");
    printf("\nStudent name:\t");
    scanf("%s", std.name);
    printf("\nEnter Student rollno.:\t");
    scanf("%d", &std.roll);
    show(std);
}

void show(struct Student st)
{
    printf("\nstudent name is %s", st.name);
    printf("\nroll is %d", st.roll);
}
```

typedef in C

studytonight.com/c/typedef.php

`typedef` is a keyword used in C language to assign alternative names to existing datatypes. Its mostly used with user defined datatypes, when names of the datatypes become slightly complicated to use in programs. Following is the general syntax for using `typedef`,

```
typedef <existing_name> <alias_name>
```

Lets take an example and see how `typedef` actually works.

```
typedef unsigned long ulong;
```

The above statement define a term `ulong` for an `unsigned long` datatype. Now this `ulong` identifier can be used to define `unsigned long` type variables.

```
ulong i, j;
```

Application of typedef

`typedef` can be used to give a name to user defined data type as well. Lets see its use with structures.

```
typedef struct
{
    type member1;
    type member2;
    type member3;
} type_name;
```

Here **type_name** represents the stucture definition associated with it. Now this **type_name** can be used to declare a variable of this stucture type.

```
type_name t1, t2;
```

Structure definition using typedef

Let's take a simple code example to understand how we can define a structure in C using `typedef` keyword.

```
#include<stdio.h>
#include<string.h>

typedef struct employee
{
    char name[50];
    int salary;
}emp;

void main( )
{
    emp e1;
    printf("\nEnter Employee record:\n");
    printf("\nEmployee name:\t");
    scanf("%s", e1.name);
    printf("\nEnter Employee salary: \t");
    scanf("%d", &e1.salary);
    printf("\nstudent name is %s", e1.name);
    printf("\nroll is %d", e1.salary);
}
```

`typedef` and Pointers

`typedef` can be used to give an alias name to pointers also. Here we have a case in which use of `typedef` is beneficial during pointer declaration.

In Pointers `*` binds to the right and not on the left.

```
int* x, y;
```

By this declaration statement, we are actually declaring `x` as a pointer of type `int`, whereas `y` will be declared as a plain `int` variable.

```
typedef int* IntPtr;
IntPtr x, y, z;
```

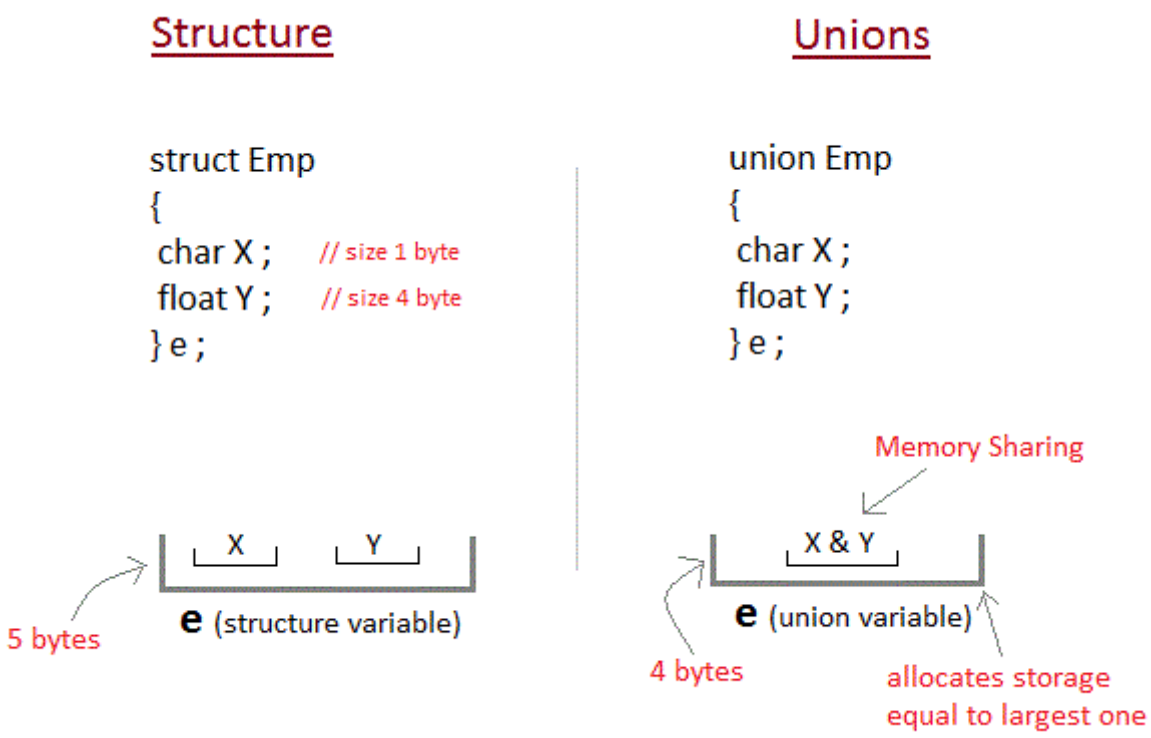
But if we use `typedef` like we have used in the example above, we can declare any number of pointers in a single statement.

NOTE: If you do not have any prior knowledge of pointers, do study Pointers first.

C Unions

studytonight.com/c/unions-in-c.php

Unions are conceptually similar to **structures in C**. The syntax to declare/define a union is also similar to that of a structure. The only differences is in terms of **storage**. In **structure** each member has its own storage location, whereas all members of **union** uses a **single shared memory** location which is equal to the size of its largest data member.



This implies that although a **union** may contain many members of different types, it **cannot handle all the members at the same time**.

Declaring a Union in C

A **union** is declared using the `union` keyword in C.

The **syntax** is as follows

```
union tag_name {
   member definition;
   member definition;
   ...
   member definition;
} union variable(s);
```

For example,

```
union item
{
    int m;
    float x;
    char c;
} It1;
```

This declares a variable `It1` of type `union` item. This `union` contains three members each with a different data type. However only one of them can be used at a time. This is due to the fact that only one location is allocated for all the `union` variables, irrespective of their size. The compiler allocates the storage that is large enough to hold the largest variable type in the union.

In the union declared above the member `x` requires **4 bytes** which is largest amongst the members for a 16-bit machine. Other members of union will share the same memory address.

To define variables of a union, we use `union` keyword as follows:

```
union item it2, it3;
```

Accessing a Union Member in C

We use **member access operator** (`.`) to access members of a union in C. It is used between the union variable name and the union member that we want to access. Syntax for accessing any union member is similar to accessing structure members.

```
union test
{
    int a;
    float b;
    char c;
}t;

t.a;    //to access members of union t
t.b;
t.c;
```

In unions, if we change the value of any one member, the value of other members gets affected.

Using Union in C Program

Here is a program to understand how compiler decides size of a union:

```
#include <stdio.h>

union one {
    int x;
    char y;
} one1;

union two {
    int x;
    char y;
    long z;
} two2;

union three {
    int arr[100];
    char y;
    double d[5];
} three3;

int main()
{
    printf("sizeof(one) = %lu, sizeof(two) = %lu, sizeof(three) = %lu",
sizeof(one1), sizeof(two2), sizeof(three3));
    return 0;
}
```

sizeof(one) = 4, sizeof(two) = 8, sizeof(three) = 400

Run Code →

Let's see another code example,

```
// defining and printing members of a union
#include <stdio.h>

union item
{
    int a;
    float b;
    char ch;
};

int main( )
{
    union item it;
    it.a = 12;
    it.b = 20.2;
    it.ch = 'z';

    printf("%d\n", it.a);
    printf("%f\n", it.b);
    printf("%c\n", it.ch);

    return 0;
}
```

1101109626
20.199940
z

We can also create an **array of `union` type** values. So let's see a code example,

```
#include <stdio.h>
union item
{
    int a;
    float b;
    char ch;
};

int main( )
{
    union item it[10];
    int n;
    printf("Enter the number of records:");
    scanf("%d", &n);
    for(int i = 0; i < n; i++) {
        printf("Enter record %d: ", i + 1);
        scanf("%d %f %c", &it[i].a, &it[i].b, &it[i].ch);
    }
    for(int i = 0; i < n; i++) {
        printf("\nRecord no. %d:\n", i + 1);
        printf("%d %f %c", it[i].a, it[i].b, it[i].ch);
    }

    return 0;
}
```

Enter the number of records:2
Enter record 1: 1
3
a
Enter record 2: 2
4
d

Record no. 1:
1077936225 3.000023 a
Record no. 2:
1082130532 4.000048 d

As you can see here, the values of `int` and `float` get corrupted and only `char` variable prints the expected result. This is because in union, the memory is shared among different data types.

In the above example, value of the `char` variable was stored at last, hence the value of other variables is lost.

Difference Between Structure and Union in C

- In union, we can only initializa the first data member whereas in a structure, we can initialize many data members at once.

- Compiler allocates memory for each member of a structure while for a union, it allocates memory equal to the size of the largest data member.
- Union members share a memory location while structure members have a unique storage location each.
- In a structure, we can access individual members simultaneously while in a union, we can only access one member at a time.
- If we change the value of a member in a structure, it won't affect its other members but in a union, changing the value of one member will affect the others.

Difference in the size of Structure and Union in C

Let's look at the following example to understand better.

```
#include <stdio.h>

struct one {
    int x;
    char y;
    long z;
} one1;

union two {
    int x;
    char y;
    long z;
} two2;

int main()
{
    printf("sizeof(struct) = %lu, sizeof(union) = %lu",sizeof(one1),
sizeof(two2));
    return 0;
}
```

sizeof(struct) = 16, sizeof(union) = 8

We can see that the size of structure is greater than the sum of size of its members while the size of union is equal to the size of its largest member. This is why there is a difference in the size of structure and union with exactly the same data members. Union members share memory while structure members have their own unique memory.

Introduction to C Pointers

studytonight.com/c/pointers-in-c.php

A Pointer in C language is a variable that holds a memory address. This memory address is the address of another variable(mostly) of same data type.

In simple words, if one variable stores the address of second variable then the first variable can be said to **point towards** the second variable.

Before we start understanding what pointers are and what they can do, let's start by understanding what does **"Address of a memory location"** means?

What is a Memory Address in C?

Whenever a variable is defined in C language, a memory location is assigned for it, in which it's value gets stored. We can check this memory address, using the `&` symbol.

If `var` is the name of the variable, then `&var` will give it's address.

Let's write a small program to see memory address of any variable that we define in our program.

```
#include<stdio.h>

void main()
{
    int var = 7;
    printf("Value of the variable var is: %d\n", var);
    printf("Memory address of the variable var is: %x\n", &var);
}
```

Value of the variable var is: 7 Memory address of the variable var is: bcc7a00

Also while using the `scanf()` function, we mention `&var` to take user input for any variable `var` .

```
scanf("%d", &var);
```

This is used to store the user input value to the memory address of the variable `var` .

What is a Pointer in C?

Like we mentioned earlier, a Pointer in C language is a variable that holds a memory address.

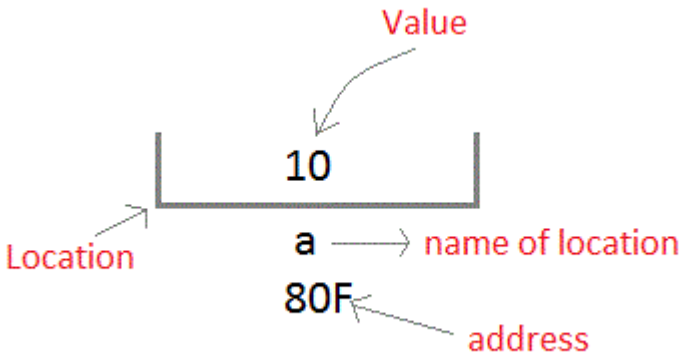
Pointers are used to access memory of a variable and manipulate the value stored in it.

Pointers are one of the most distinct and exciting features of C language. It provides power and flexibility to the language. Although pointers may appear a little confusing and complicated in the beginning, but trust me, once you understand the concept, you will be able to do so much more with C language.

Whenever a **variable** is declared in a program, system allocates a location i.e an address to that variable in the memory, to hold the assigned value. This location has its own address number, which we saw in the program above.

Let us assume that system has allocated memory location `80F` for a variable `a`.

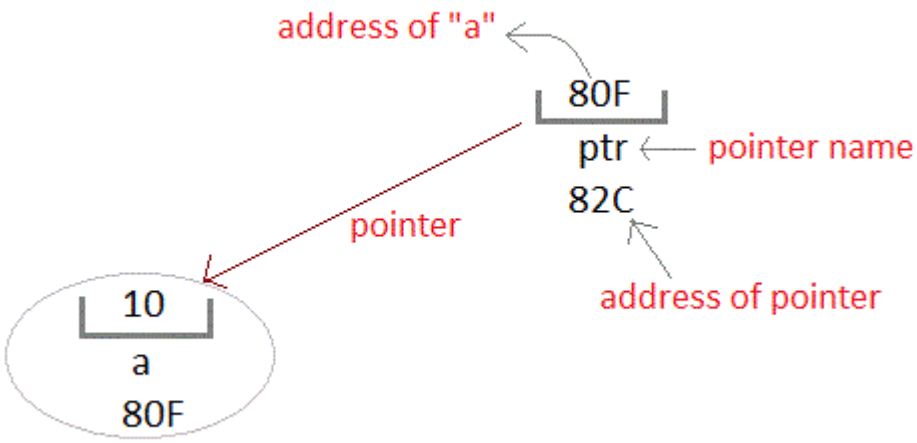
```
int a = 10;
```



We can access the value `10` either by using the variable name `a` or by using its address `80F`.

The question is how we can access a variable using it's address? Since the memory addresses are also just numbers, they can also be assigned to some other variable. The variables which are used to hold memory addresses are called **Pointer variables**.

A **pointer** variable is therefore nothing but a variable which holds an address of some other variable. And the value of a **pointer variable** gets stored in another memory location.



Pointer Variable in C

Like we mentioned above that a pointer is also a variable, but with a little twist, that is, it only stores address of other variables.

So if you have to define a pointer variable, the syntax is a little different.

Following is the **syntax** for declaring a variable as a pointer:

```
type *name;
```

Here, **type** is the data type of the pointer, and the **name** is the name of the pointer variable.

And the `*` operator with the name, informs the compiler that the variable is a pointer.

The data type of the pointer variable should be the same as of the variable to which the pointer is pointing.

Pointer Operators in C

There are two pointer operators in C, they are:

1. `*` operator
2. `&` operator

We have covered operators in C in detail separately.

The `&` operator returns the memory address of its operand. For example,

```
a = &b;
```

In the variable `a` the memory address of the variable `b` will get stored.

The `*` operators is the complement of `&`. Thiss operator returns the value located at the given address.

For example, if `a` contains the memory address of the variable `b`, then the code,

```
c = *a;
```

will store the value of the variable `b` into `c`.

Pointers in C Example

Let's see a basic code example where we will create a pointer and assign it a value.

```
#include<stdio.h>

int main(void)
{
    int x = 99;
    // declare a pointer
    int *ptr;
    // assign value to pointer
    ptr = &x;

    printf("Value at ptr is: %d \n", *ptr);

    printf("Address pointed by ptr is: %p \n", ptr);

    return 0;
}
```

Value at ptr is: 99
Address pointed by ptr is: 0x7fff99c0e6c4

Run Code →

The `%p` format specifier is used for pointer variable.

Benefits of using pointers in C

Below we have listed a few benefits of using pointers:

1. Pointers are more efficient in handling Arrays in C and Structures in C.
2. Pointers allow references to function and thereby helps in passing of function as arguments to other functions.
3. Pointers also provide means by which a function in C can change its calling arguments.
4. It reduces length of the program and its execution time as well.
5. It allows C language to support Dynamic Memory management.

In the next tutorial we will learn syntax of pointers, how to declare and define a pointer, and using a pointer. See you in the next tutorial.

Using Pointers in C

studytonight.com/c/declaring-and-initializing-pointer.php

In this tutorial, we will learn how to declare, initialize and use a pointer in C language.

Before you continue, check these topics out:

- Introduction to Pointers
- C Arrays
- C Loops - `while`, `do while`, `for` loops
- C Functions

A pointer is a variable used to store memory address. Hence, we have to declare and initialise(assign it a value) it just like any other variable. Pointers can be very useful in some use-cases, like:

- They make accessing array elements easier.
- We can implement linked lists, trees, and graphs using pointers.
- We use pointers for accessing dynamically allocated memory.
- We can return more than one value from a function using pointers.

So let's see how we can create pointers, assign values to pointers, perform pointer coversions, pointer arithmetic and pointer comparisons.

Declaring a Pointer in C

The general syntax of pointer declaration is,

```
type *pointer_name;
```

Here, **pointer_name** is the name of the pointer and that should be a valid C identifier.

The datatype of the pointer and the variable to which the pointer variable is pointing must be the same.

Following are some examples of declaring a pointer in C:

```
int *ptr;     //pointer to int
float *ptr;   //pointer to float
char *ptr;    //pointer to char
double *ptr;  //pointer to double
```

Just like a variable, pointer is declared in the same way, just with an additional pointer operator `*`.

Assigning Value to a Pointer Variable

When we declare a pointer, it contains **garbage value**, which means it could be pointing anywhere in the memory. **Pointer Initialization** is the process of assigning the address of a variable to a pointer. In C language, the address operator `&` is used to determine the address of a variable. The `&` (immediately preceding a variable name) returns the address of the variable associated with it.

For example,

```
#include<stdio.h>

int main(void)
{
    int a = 10;
    // declare a pointer
    int *ptr;
    // assign value to pointer
    ptr = &a;

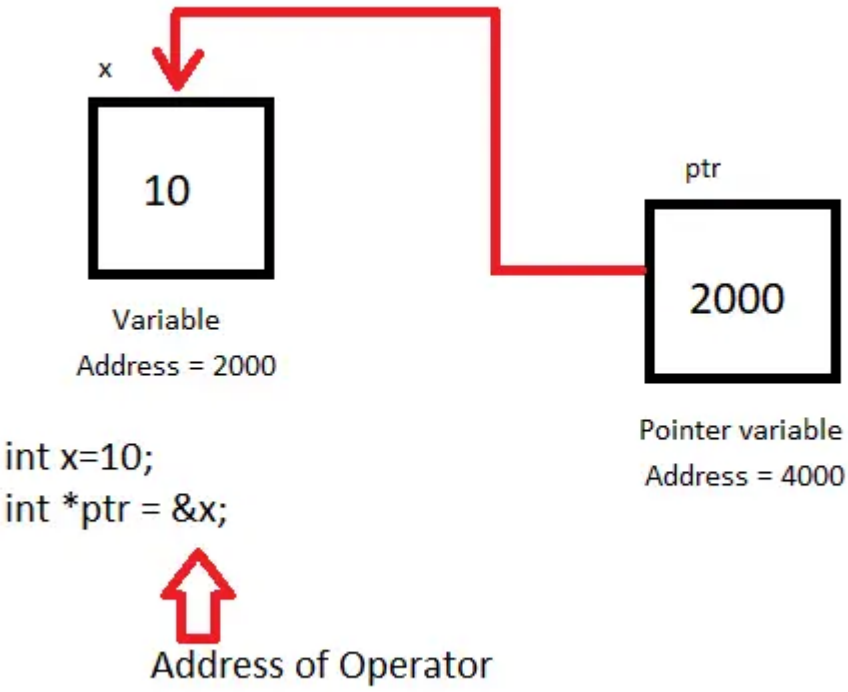
    printf("Value at ptr is: %d \n", *ptr);

    printf("Address pointed by ptr is: %p \n", ptr);

    return 0;
}
```

Value at ptr is: 10
Address pointed by ptr is: 0x7fff99c0e6c4

Run Code →



Pointer variables must always point to variables of the same datatype.

For example, if we have a `float` type variable and an `int` type pointer, then C compiler will give error.

```
#include<stdio.h>

int main(void)
{
    float a = 10;
    // declare a pointer
    int *ptr;
    // assign value to pointer
    ptr = &a;

    printf("Value at ptr is: %d \n", *ptr);

    printf("Address pointed by ptr is: %p \n", ptr);

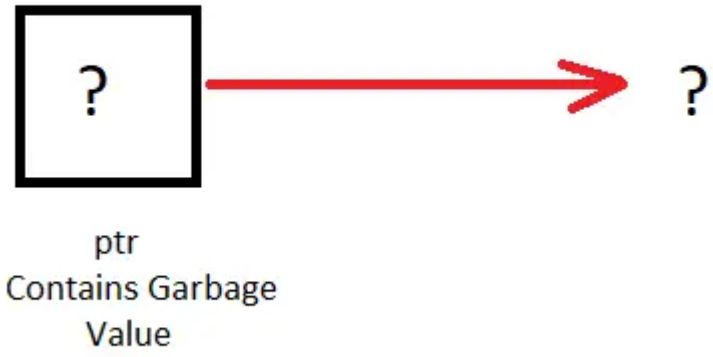
    return 0;
}
```

warning: assignment from incompatible pointer type
ptr = &x; ^

Run Code →

While declaring a pointer variable, if it is not assigned to anything then it contains **garbage value**. And that can lead to unexpected errors in your program. Hence, it is recommended to assign a `NULL` value to it.

When we assign `NULL` to a pointer, it means that it does not point to any valid address. `NULL` denotes the value 'zero'.



A pointer that is assigned a `NULL` value is called a **NULL pointer in C**.

We can give a pointer a null value by assigning it zero to it. For example,

```
int *ptr = 0;
```

The above code will initialize the `ptr` pointer will a null value.

We can also use the **NULL** macro, which is nothing but a predefined constant for null pointer. This is defined in the **<stdio.h>** header library.

```
int *ptr = NULL;
```

Pointer to Pointer Assignment in C

We can use an assignment operator to assign value of a pointer to another pointer variable. But for such assignment, types of both the pointer should be same.

Let's take a code example,

```
#include<stdio.h>

int main(void)
{
    float a = 10;
    // declare two pointers
    int *p1, *p2;

    // assign value to pointer
    p1 = &a;
    // assign value from one pointer to another
    p2 = p1;

    printf("Value at p1 and p2: %d %d \n", *p1, *p2);

    printf("Address pointed by p1 and p2: %p %p \n", p1, p2);

    return 0;
}
```

Value at p1 and p2: 10 10
Address pointed by p1 and p2: 0x7fff99c0e6c4 0x7fff99c0e6c4

As you can see in the code example above, multiple pointers can point to the same variable but they should be of the same data type.

Pointer Type Conversion in C

We can assign a pointer of one type to a pointer of another type by doing pointer type conversion.

1. `void *` pointer

The pointers of type `void *` are known as **Generic pointers**, and they can be assigned to any other type of pointer. Also, any other type of pointer can be assigned to a `void *` pointer.

2. Pointer type conversion

For pointer type other than `void *`, we have to explicitly cast pointer from one type to another. But this can lead to unexpected behaviour for incompatible datatypes.

For example, if we have a variable of type `double`, and we want to use a pointer of type `int` to point to this variable. Even after using explicit cast, the pointer will work as if it is pointing to a `int` type value. Now, a `double` type is of **8 bytes** whereas an `int` type is of **4 bytes**, hence, 4 bytes of information will be lost.

Let's see how we can use explicit cast for pointer conversion.

```
#include<stdio.h>

int main(void)
{
    double a = 1000.10;
    double b;
    // declare a pointer
    int *p1;
    // assign value to pointer with casting
    p1 = (int *) &a;

    // value of 'b' should be same as 'a', but it won't be
    b = *p1;

    printf("Value of a is: %f \n", b);

    return 0;
}
```

Value of a is: -858993459.000000

Run Code →

Pointer and Arrays

Derefrencing Pointer in C

Once a pointer has been assigned the address of a variable, the pointer is **dereferenced**, using the **indirection operator** or **dereferencing operator**, which is a `*`, to access the value of the variable.

For example, if we have,

```
int a  = 5;
int *ptr1 = &a;
float b = 5.5;
float *ptr2 = &b;
// *ptr1 = 2 is equivalent to a = 2
// (*ptr1)++ is equivalent to a++
// float z = *ptr2 + 4.2 is equivalent to float z = b + 4.2;
```

Here is one complete program,

```c
#include <stdio.h>

int main()
{
    int a;
    a = 10;
    int *p = &a;     // declaring and initializing the pointer

    //prints the value of 'a'
    printf("%d\n", *p);
    printf("%d\n", *&a);
    //prints the address of 'a'
    printf("%u\n", &a);
    printf("%u\n", p);

    printf("%u\n", &p);    //prints address of 'p'

    return 0;
}
```

```
10
10
3795480300
3795480300
3795480304
```

Points to Remember while using Pointers

- A pointer variable stores the address of a variable. We use `*` to declare and identify a pointer.
- We can find the address of any variable using the `&` (ampersand) operator.
- The declaration `int *a` doesn't mean that `a` is going to contain an integer value. It means that `a` is going to contain the address of a variable of `int` type.
- We can dereference a pointer variable using a `*` operator. Here, the `*` can be read as **'value at'**.

Since you have now learned the basics of Pointers in C, you can check out some C Pointer Programs where pointers are used for different use-cases.

Pointer to a Pointer in C(Double Pointer)

studytonight.com/c/pointer-to-pointer.php

Pointers are used to store the address of other variables of similar datatype. But if you want to store the address of a pointer variable, then you again need a pointer to store it. Thus, when one pointer variable stores the address of another pointer variable, it is known as **Pointer to Pointer** variable or **Double Pointer**.

Syntax:

```
int **p1;
```

Here, we have used two indirection operator(`*`) which stores and points to the address of a pointer variable i.e, `int *` . If we want to store the address of this (double pointer) variable `p1` , then the syntax would become:

```
int ***p2
```

Simple program to represent Pointer to a Pointer

```c
#include <stdio.h>

int main() {

    int  a = 10;
    int  *p1;       //this can store the address of variable a
    int  **p2;
    /*
        this can store the address of pointer variable p1 only.
        It cannot store the address of variable 'a'
    */

    p1 = &a;
    p2 = &p1;

    printf("Address of a = %u\n", &a);
    printf("Address of p1 = %u\n", &p1);
    printf("Address of p2 = %u\n\n", &p2);

    // below print statement will give the address of 'a'
    printf("Value at the address stored by p2 = %u\n", *p2);

    printf("Value at the address stored by p1 = %d\n\n", *p1);

    printf("Value of **p2 = %d\n", **p2); //read this *(*p2)

    /*
        This is not allowed, it will give a compile time error-
        p2 = &a;
        printf("%u", p2);
    */
    return 0;
}
```

Address of a = 2686724 Address of p1 = 2686728 Address of p2 = 2686732 Value at the address stored by p2 = 2686724 Value at the address stored by p1 = 10 Value of **p2 = 10

Explanation of the above program

- `p1` pointer variable can only hold the address of the variable `a` (i.e Number of indirection operator(*)-1 variable). Similarly, `p2` variable can only hold the address of variable `p1` . It cannot hold the address of variable `a` .



- `*p2` gives us the value at an address stored by the `p2` pointer. `p2` stores the address of `p1` pointer and value at the address of `p1` is the address of variable `a`. Thus, `*p2` prints address of `a`.
- `**p2` can be read as `*(*p2)`. Hence, it gives us the value stored at the address `*p2`. From above statement, you know `*p2` means the address of variable a. Hence, the value at the address `*p2` is 10. Thus, `**p2` prints `10`.

Pointer and Arrays in C

studytonight.com/c/pointers-with-array.php

Before you start with Pointer and Arrays in C, learn about these topics in prior:

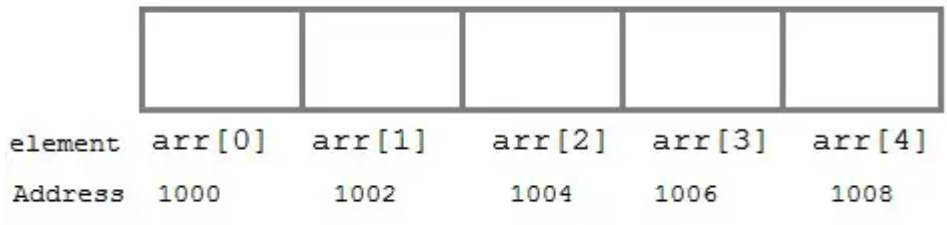
- Array in C
- Pointer in C

When an array in C language is declared, compiler allocates sufficient memory to contain all its elements. Its base address is also allocated by the compiler.

Declare an array `arr`,

```
int arr[5] = { 1, 2, 3, 4, 5 };
```

Suppose the base address of `arr` is 1000 and each integer requires two bytes, the five elements will be stored as follows:



Variable `arr` will give the base address, which is a constant pointer pointing to `arr[0]`. Hence `arr` contains the address of `arr[0]` i.e `1000`.

`arr` has two purpose -

- It is the name of the array
- It acts as a pointer pointing towards the first element in the array.

> `arr` is equal to `&arr[0]` by default

For better understanding of the declaration and initialization of the pointer - click here. and refer to the program for its implementation.

NOTE:

> You cannot decrement a pointer once incremented. `p--` won't work.

Pointer to Array

Use a pointer to an array, and then use that pointer to access the array elements. For example,

```
#include<stdio.h>

void main()
{
   int a[3] = {1, 2, 3};
   int *p = a;
   for (int i = 0; i < 3; i++)
   {
      printf("%d", *p);
      p++;
   }
  return 0;
}
```

1 2 3

Replacing the **printf("%d", *p);** statement of above example, with below mentioned statements. Lets see what will be the result.

printf("%d", a[i]); ⟶ **prints the array, by incrementing index**

printf("%d", i[a]); ⟶ **this will also print elements of array**

printf("%d", a+i); ⟶ **This will print address of all the array elements**

printf("%d", *(a+i)); ⟶ **Will print value of array element.**

printf("%d", *a); ⟶ **will print value of a[0] only**

a++; ⟶ **Compile time error, we cannot change base address of the array.**

Syntax:

```
*(a+i)  //pointer with an array
```

is same as:

```
a[i]
```

Pointer to Multidimensional Array

Let's see how to make a pointer point to a multidimensional array. In `a[i][j]`, `a` will give the base address of this array, even `a + 0 + 0` will also give the base address, that is the address of `a[0][0]` element.

Syntax:

```
*(*(a + i) + j)
```

Pointer and Character strings

Pointer is used to create strings. Pointer variables of `char` type are treated as string.

```
char *str = "Hello";
```

The above code creates a string and stores its address in the pointer variable `str`. The pointer `str` now points to the first character of the string "Hello".

> The string created using `char` pointer can be assigned a value at **runtime**.

```
char *str;
str = "hello";
```

> The content of the string can be printed using `printf()` and `puts()`.

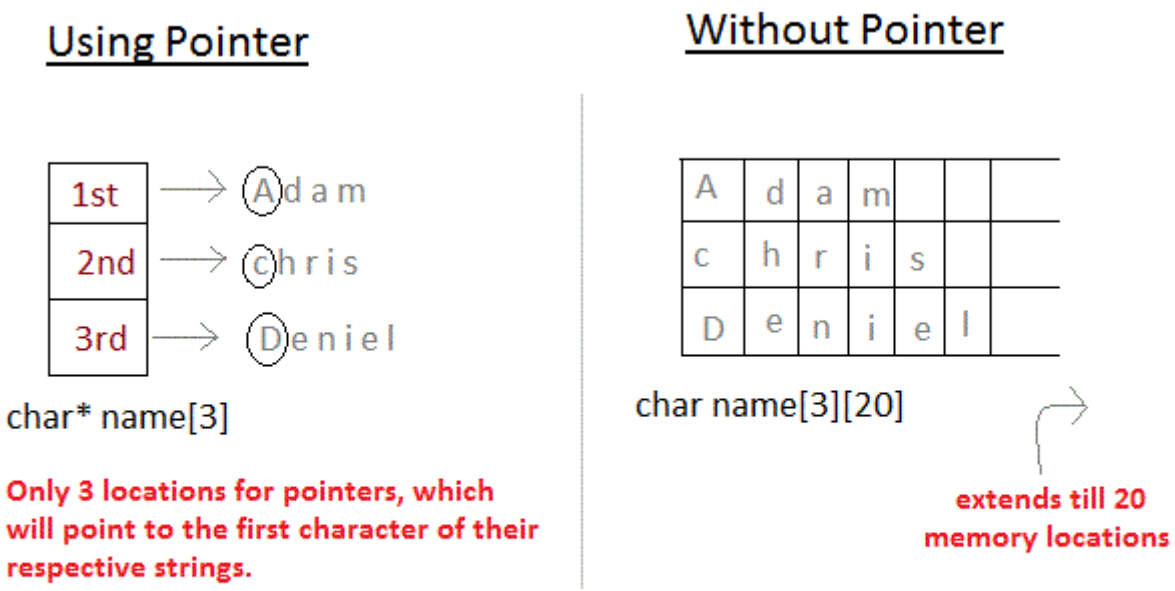
```
printf("%s", str);
puts(str);
```

> `str` is a pointer to the string and also name of the string. Therefore we do not need to use indirection operator `*`.

Array of Pointers

Pointers are very helpful in handling character arrays with rows of varying lengths.

```
char *name[3] = {
    "Adam",
    "chris",
    "Deniel"
};
//without pointer
char name[3][20] = {
    "Adam",
    "chris",
    "Deniel"
};
```



In the second approach memory wastage is more, hence it is preferred to use pointer in such cases.

Pointer to Array of Structures in C

studytonight.com/c/pointers-to-structure-in-c.php

Like we have array of integers, array of pointers etc, we can also have array of structure variables. And to use the array of structure variables efficiently, we use **pointers of structure type**. We can also have pointer to a single structure variable, but it is mostly used when we are dealing with array of structure variables.

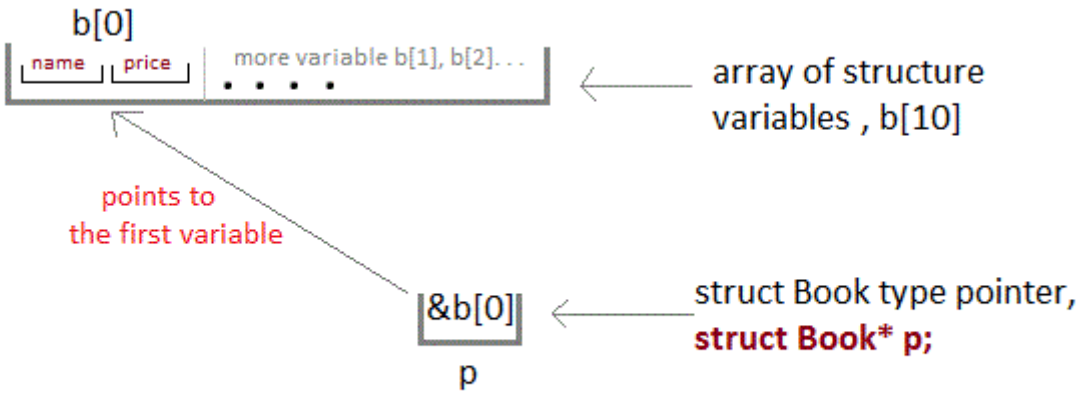
```
#include <stdio.h>

struct Book
{
    char name[10];
    int price;
}

int main()
{
    struct Book a;      //Single structure variable
    struct Book* ptr;   //Pointer of Structure type
    ptr = &a;

    struct Book b[10];  //Array of structure variables
    struct Book* p;     //Pointer of Structure type
    p = &b;

    return 0;
}
```



Accessing Structure Members with Pointer

To access members of structure using the structure variable, we used the dot `.` operator. But when we have a pointer of structure type, we use arrow `->` to access structure members.

```
#include <stdio.h>

struct my_structure {
    char name[20];
    int number;
    int rank;
};

int main()
{
    struct my_structure variable = {"StudyTonight", 35, 1};

    struct my_structure *ptr;
    ptr = &variable;

    printf("NAME: %s\n", ptr->name);
    printf("NUMBER: %d\n", ptr->number);
    printf("RANK: %d", ptr->rank);

    return 0;
}
```

NAME: StudyTonight NUMBER: 35 RANK: 1

Pointer Arithmetic in C

studytonight.com/c/pointer-arithmetic-in-c.php

If you want to have complete knowledge of pointers, pointer arithmetic is very important to understand. In this topic we will study how the memory addresses change when you increment a pointer.

16 bit Machine (Turbo C)

In a 16 bit machine, size of all types of pointer, be it `int*`, `float*`, `char*` or `double*` is always **2 bytes**. But when we perform any arithmetic function like increment on a pointer, changes occur as per the size of their primitive data type.

Size of datatypes on 16-bit Machine:

Type	Size (in bytes)
int or signed int	2
char	1
long	4
float	4
double	8
long double	10

Examples for Pointer Arithmetic

Now lets take a few examples and understand this more clearly.

```
int* i;
i++;
```

In the above case, pointer will be of 2 bytes. And when we increment it, it will increment by 2 bytes because `int` is also of 2 bytes.

```
float* i;
i++;
```

In this case, size of pointer is still 2 bytes initially. But now, when we increment it, it will increment by 4 bytes because `float` datatype is of 4 bytes.

```
double* i;
i++;
```

Similarly, in this case, size of pointer is still 2 bytes. But now, when we increment it, it will increment by 8 bytes because its data type is `double`.

---

32 bit Machine (Visual Basic C++)

The concept of pointer arithmetic remains exact same, but the size of pointer and various datatypes is different in a 32 bit machine. Pointer in 32 bit machine is of **4 bytes**.

And, following is a table for **Size of datatypes on 32-bit Machine :**

Type	Size (in bytes)
int or signed int	4
char	2
long	8
float	8
double	16

Note: We cannot add two pointers. This is because pointers contain addresses, adding two addresses makes no sense, because you have no idea what it would point to.

But we can subtract two pointers. This is because difference between two pointers gives the number of elements of its data type that can be stored between the two pointers.

Program for pointer arithmetic(32-bit machine)

```
#include <stdio.h>

int main()
{
    int m = 5, n = 10, o = 0;

    int *p1;
    int *p2;
    int *p3;

    p1 = &m;    //printing the address of m
    p2 = &n;    //printing the address of n

    printf("p1 = %d\n", p1);
    printf("p2 = %d\n", p2);

    o = *p1+*p2;
    printf("*p1+*p2 = %d\n", o);//point 1

    p3 = p1-p2;
    printf("p1 - p2 = %d\n", p3); //point 2

    p1++;
    printf("p1++ = %d\n", p1); //point 3

    p2--;
    printf("p2-- = %d\n", p2); //point 4

    //Below line will give ERROR
    printf("p1+p2 = %d\n", p1+p2); //point 5

    return 0;
}
```

p1 = 2680016 p2 = 2680012 *p1+*p2 = 15 p1-p2 = 1 p1++ = 2680020 p2-- = 2680008

Explanation of the above program:

1. **Point 1:** Here, `*` means 'value at the given address'. Thus, it adds the value of m and n which is 15.
2. **Point 2:** It subtracts the addresses of the two variables and then divides it by the size of the pointer datatype (here integer, which has size of 4 bytes) which gives us the number of elements of integer data type that can be stored within it.
3. **Point 3:** It increments the address stored by the pointer by the size of its datatype(here 4).
4. **Point 4:** It decrements the address stored by the pointer by the size of its datatype(here 4).
5. **Point 5:** Addition of two pointers is not allowed.

Pointers as Function Argument in C

studytonight.com/c/pointer-with-function-in-c.php

Pointer as a function parameter is used to hold addresses of arguments passed during function call. This is also known as **call by reference**. When a function is called by reference any change made to the reference variable will effect the original variable.

Example Time: Swapping two numbers using Pointer

```
#include <stdio.h>

void swap(int *a, int *b);

int main()
{
    int m = 10, n = 20;
    printf("m = %d\n", m);
    printf("n = %d\n\n", n);

    swap(&m, &n);    //passing address of m and n to the swap function
    printf("After Swapping:\n\n");
    printf("m = %d\n", m);
    printf("n = %d", n);
    return 0;
}

/*
    pointer 'a' and 'b' holds and
    points to the address of 'm' and 'n'
*/
void swap(int *a, int *b)
{
    int temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
```

m = 10 n = 20 After Swapping: m = 20 n = 10

Functions returning Pointer variables

A function can also `return` a pointer to the calling function. In this case you must be careful, because local variables of function doesn't live outside the function. They have scope only inside the function. Hence if you return a pointer connected to a local variable, that pointer will be pointing to nothing when the function ends.

```
#include <stdio.h>

int* larger(int*, int*);

void main()
{
    int a = 15;
    int b = 92;
    int *p;
    p = larger(&a, &b);
    printf("%d is larger",*p);
}

int* larger(int *x, int *y)
{
    if(*x > *y)
        return x;
    else
        return y;
}
```

92 is larger

Safe ways to return a valid Pointer.

1. Either use **argument with functions**. Because argument passed to the functions are declared inside the calling function, hence they will live outside the function as well.
2. Or, use `static` **local variables** inside the function and return them. As static variables have a lifetime until the `main()` function exits, therefore they will be available througout the program.

Pointer to functions

It is possible to declare a pointer pointing to a function which can then be used as an argument in another function. A pointer to a function is declared as follows,

```
type (*pointer-name)(parameter);
```

Here is an example :

```
int (*sum)();   //legal declaration of pointer to function
int *sum();     //This is not a declaration of pointer to function
```

A function pointer can point to a specific function when it is assigned the name of that function.

```
int sum(int, int);
int (*s)(int, int);
s = sum;
```

Here `s` is a pointer to a function `sum`. Now `sum` can be called using function pointer `s` along with providing the required argument values.

```
s (10, 20);
```

Example of Pointer to Function

```
#include <stdio.h>

int sum(int x, int y)
{
    return x+y;
}

int main( )
{
    int (*fp)(int, int);
    fp = sum;
    int s = fp(10, 15);
    printf("Sum is %d", s);

    return 0;
}
```

25

Complicated Function Pointer example

You will find a lot of complex function pointer examples around, lets see one such example and try to understand it.

```
void *(*foo) (int*);
```

It appears complex but it is very simple. In this case `(*foo)` is a pointer to the function, whose argument is of `int*` type and return type is `void*`.

File Input/Output in C

studytonight.com/c/file-input-output.php

A **file** represents a sequence of bytes on the disk where a group of related data is stored. File is created for permanent storage of data. It is a ready made structure.

In C language, we use a structure **pointer of file type** to declare a file.

```
FILE *fp;
```

C provides a number of functions that helps to perform basic file operations. Following are the functions,

Function	description
fopen()	create a new file or open a existing file
fclose()	closes a file
getc()	reads a character from a file
putc()	writes a character to a file
fscanf()	reads a set of data from a file
fprintf()	writes a set of data to a file
getw()	reads a integer from a file
putw()	writes a integer to a file
fseek()	set the position to desire point
ftell()	gives current position in the file
rewind()	set the position to the begining point

Opening a File or Creating a File

The `fopen()` function is used to create a new file or to open an existing file.

General Syntax:

```
*fp = FILE *fopen(const char *filename, const char *mode);
```

Here, `*fp` is the FILE pointer (`FILE *fp`), which will hold the reference to the opened(or created) file.

filename is the name of the file to be opened and **mode** specifies the purpose of opening the file. Mode can be of following types,

mode	description
r	opens a text file in reading mode
w	opens or create a text file in writing mode.
a	opens a text file in append mode
r+	opens a text file in both reading and writing mode
w+	opens a text file in both reading and writing mode
a+	opens a text file in both reading and writing mode
rb	opens a binary file in reading mode
wb	opens or create a binary file in writing mode
ab	opens a binary file in append mode
rb+	opens a binary file in both reading and writing mode
wb+	opens a binary file in both reading and writing mode
ab+	opens a binary file in both reading and writing mode

Closing a File

The `fclose()` function is used to close an already opened file.

General Syntax :

```
int fclose( FILE *fp);
```

Here `fclose()` function closes the file and returns **zero** on success, or **EOF** if there is an error in closing the file. This **EOF** is a constant defined in the header file **stdio.h**.

Input/Output operation on File

In the above table we have discussed about various file I/O functions to perform reading and writing on file. `getc()` and `putc()` are the simplest functions which can be used to read and write individual characters to a file.

```
#include<stdio.h>

int main()
{
    FILE *fp;
    char ch;
    fp = fopen("one.txt", "w");
    printf("Enter data...");
    while( (ch = getchar()) != EOF) {
        putc(ch, fp);
    }
    fclose(fp);
    fp = fopen("one.txt", "r");

    while( (ch = getc(fp)! = EOF)
    printf("%c",ch);

    // closing the file pointer
    fclose(fp);

    return 0;
}
```

Reading and Writing to File using `fprintf()` and `fscanf()`

```
#include<stdio.h>

struct emp
{
    char name[10];
    int age;
};

void main()
{
    struct emp e;
    FILE *p,*q;
    p = fopen("one.txt", "a");
    q = fopen("one.txt", "r");
    printf("Enter Name and Age:");
    scanf("%s %d", e.name, &e.age);
    fprintf(p,"%s %d", e.name, e.age);
    fclose(p);
    do
    {
        fscanf(q,"%s %d", e.name, e.age);
        printf("%s %d", e.name, e.age);
    }
    while(!feof(q));
}
```

In this program, we have created two FILE pointers and both are refering to the same file but in different modes.

`fprintf()` function directly writes into the file, while `fscanf()` reads from the file, which can then be printed on the console using standard `printf()` function.

Difference between Append and Write Mode

Write (w) mode and Append (a) mode, while opening a file are almost the same. Both are used to write in a file. In both the modes, new file is created if it doesn't exists already.

The only difference they have is, when you **open** a file in the **write** mode, the file is reset, resulting in deletion of any data already present in the file. While in **append** mode this will not happen. Append mode is used to append or add data to the existing data of file(if any). Hence, when you open a file in Append(a) mode, the cursor is positioned at the end of the present data in the file.

Reading and Writing in a Binary File

A Binary file is similar to a text file, but it contains only large numerical data. The Opening modes are mentioned in the table for opening modes above.

`fread()` and `fwrite()` functions are used to read and write is a binary file.

```
fwrite(data-element-to-be-written, size_of_elements, number_of_elements,
pointer-to-file);
```

`fread()` is also used in the same way, with the same arguments like `fwrite()` function. Below mentioned is a simple example of writing into a binary file

```
const char *mytext = "The quick brown fox jumps over the lazy dog";
FILE *bfp= fopen("test.txt", "wb");
if (bfp)
{
    fwrite(mytext, sizeof(char), strlen(mytext), bfp);
    fclose(bfp);
}
```

`fseek()`, `ftell()` and `rewind()` functions

- `fseek()` : It is used to move the reading control to different positions using fseek function.
- `ftell()` : It tells the byte location of current position of cursor in file pointer.
- `rewind()` : It moves the control to beginning of the file.

Error Handling in C

studytonight.com/c/error-handling-in-c.php

C language does not provide any direct support for error handling. However a few methods and variables defined in **error.h** header file can be used to point out error using the return statement in a function. In C language, a function returns `-1` or `NULL` value in case of any error and a global variable **errno** is set with the error code. So the return value can be used to check error while programming.

What is errno?

Whenever a function call is made in C language, a variable named `errno` is associated with it. It is a global variable, which can be used to identify which type of error was encountered while function execution, based on its value. Below we have the list of Error numbers and what does they mean.

errno value	Error
1	Operation not permitted
2	No such file or directory
3	No such process
4	Interrupted system call
5	I/O error
6	No such device or address
7	Argument list too long
8	Exec format error
9	Bad file number
10	No child processes
11	Try again
12	Out of memory
13	Permission denied

C language uses the following functions to represent error messages associated with **errno**:

- `perror()` : returns the string passed to it along with the textual represention of the current errno value.
- `strerror()` is defined in **string.h** library. This method returns a pointer to the string representation of the current errno value.

Time for an Example

```
#include <stdio.h>
#include <errno.h>
#include <string.h>

int main ()
{
    FILE *fp;

    /*
        If a file, which does not exists, is opened,
        we will get an error
    */
    fp = fopen("IWillReturnError.txt", "r");

    printf("Value of errno: %d\n ", errno);
    printf("The error message is : %s\n", strerror(errno));
    perror("Message from perror");

    return 0;
}
```

Value of errno: 2 The error message is: No such file or directory Message from perror: No such file or directory

---

Other ways of Error Handling

We can also use **Exit Status** constants in the `exit()` function to inform the calling function about the error. The two constant values available for use are `EXIT_SUCCESS` and `EXIT_FAILURE`. These are nothing but macros defined **stdlib.h** header file.

```
#include <stdio.h>
#include <errno.h>
#include <stdlib.h>
#include <string.h>

extern int errno;

void main()
{
    char *ptr = malloc( 1000000000UL);  //requesting to allocate 1gb memory
space
    if (ptr == NULL)    //if memory not available, it will return null
    {
        puts("malloc failed");
        puts(strerror(errno));
        exit(EXIT_FAILURE);     //exit status failure
    }
    else
    {
        free( ptr);
        exit(EXIT_SUCCESS);     //exit status Success
    }
}
```

Here exit function is used to indicate exit status. Its always a good practice to exit a program with a exit status. **EXIT_SUCCESS** and **EXIT_FAILURE** are two macro used to show exit status. In case of program coming out after a successful operation **EXIT_SUCCESS** is used to show successful exit. It is defined as 0. **EXIT_Failure** is used in case of any failure in the program. It is defined as -1.

Division by Zero

There are some situation where nothing can be done to handle the error. In C language one such situation is division by zero. All you can do is avoid doing this, becasue if you do so, C language is not able to understand what happened, and gives a runtime error.

Best way to avoid this is, to check the value of the divisor before using it in the division operations. You can use `if` condition, and if it is found to be zero, just display a message and return from the function.

Dynamic Memory Allocation in C

studytonight.com/c/dynamic-memory-allocation-in-c.php

In C language, the process of allocating memory at runtime is known as **dynamic memory allocation**. Library functions known as **memory management functions** are used for assigning (allocating) and freeing memory, during execution of a program. These functions are defined in the **stdlib.h** header file.

These functions allocate memory from a memory area known as **heap** and when the memory is not in use, it is freed to be used for some other purpose. We access the dynamically allocated memory using pointers

We have done static memory allocation till now. The memory requirement was fixed. It could not be increased or decreased during execution of the program. This type of allocation has a drawback. We cannot predict how much memory will be required. For example, if we declare an array, like this:

```
int marks[50];
```

We might only store data for **25 students**. As a result, memory will be wasted. Or we may need to store data for **60 students**, in that case we will have less memory. The solution to these problems is dynamic memory allocation.

Difference between Static and Dynamic Memory Allocation

Before we move on to study dynamic memory allocation in detail, let's take a look at the differences between static and dynamic memory allocation.

Static Memory Allocation	Dynamic Memory Allocation
Allocation is done before execution of the program(at compile time).	Allocation is done during execution of the program(at runtime).
Here, variables are allocated memory permanently.	Here, memory is allocated only when the program is active.
It uses **stack** for managing memory allocation.	It uses **heap** for managing memory allocation.
We cannot reuse the memory.	We can reuse the memory and free it after usage.

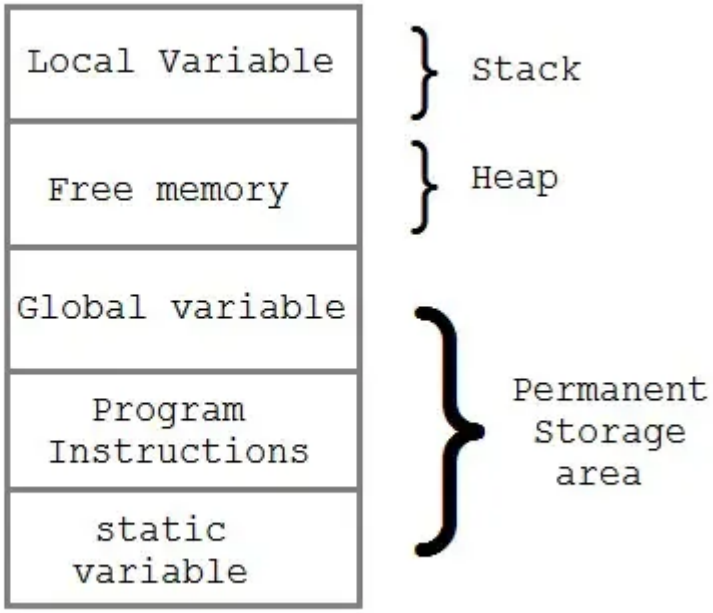
Execution is faster than dynamic memory allocation.	Execution is slower than static memory allocation.
It is less efficient.	It is more efficient.
It is simple to use.	It can be complicated to use.

Function	Description
`malloc()`	allocates requested size of bytes and returns a void pointer pointing to the first byte of the allocated space
`calloc()`	allocates space for an array of elements, initialize them to zero and then returns a void pointer to the memory
`free`	releases previously allocated memory
`realloc`	modify the size of previously allocated space

Memory Allocation Process

Global variables, `static` variables and program instructions get their memory in **permanent** storage area whereas **local** variables are stored in a memory area called **Stack**.

The memory space between these two region is known as **Heap** area. This region is used for dynamic memory allocation during execution of the program. The size of heap keeps changing.



Allocating block of Memory

Now let's see how the different functions for dynamic memory allocation works.

1. `malloc()` Function

`malloc()` function is used for allocating block of memory at runtime. This function reserves a block of memory of the given size and returns a **pointer** of type `void`. This means that we can assign it to any type of pointer using typecasting. If it fails to allocate enough space as specified, it returns a `NULL` pointer.

Syntax:

Here is the syntax of this function,

```
void* malloc(size_t size)
```

In **stdlib.h**, `size_t` is defined as `unsigned int`.

The memory allocated by `malloc()` contains garbage value.

We use this function like this,

```
data_type *ptr;
ptr = (data_type *)malloc(specified size);
```

Here, we typecast the pointer returned by `malloc()` function using (**data_type** *).

For example:

```
void main(){
    int *x;
    float *y;
    x = (int*)malloc(50);     //50 contiguous bytes of memory are allocated and
the address of the first byte is stored in x.

    y = (float*)malloc(5 * sizeof(float);    // This allocates memory to store 5
decimal numbers.
    // we should always check the value returned.
    if(x == NULL)
        printf("Memory unavailable\n");
    if(y == NULL)
        printf("Memory unavailable\n");
}
```

Let's take another example:

```
#include<stdio.h>
#include <stdlib.h>
int main(){
    int *ptr;
    ptr = malloc(5 * sizeof(int)); /* a block of 5 integers */
    if (ptr != NULL) {
        *(ptr + 1) = 10; /* assign 10 to second integer */
        printf("Value of the 2nd integer is %d",*(ptr + 1));
    }
    return 0;
}
```

Value of the 2nd integer is 10

---

2. `calloc()` Function

`calloc()` is another memory allocation function that is used for allocating memory at runtime. `calloc` function is normally used for allocating memory to derived data types such as **arrays** and **structures**. If it fails to allocate enough space as specified, it returns a `NULL` pointer.

The memory allocated by calloc() is initialized to zero.

Syntax:

Let's see the syntax for this function,

```
void *calloc(number of items, size_t size);
```

Time for an Example: `calloc()` function

Here is a simple code example,

```
char * ptr;
ptr = (char*)calloc(5, sizeof(char);
```

Let's look at another example,

```
struct employee
{
    char *name;
    int salary;
};
typedef struct employee emp;
emp *e1;
e1 = (emp*)calloc(30, sizeof(emp));
```

---

3. `realloc()` Function

The `realloc()` function is used to change the memory size that is already allocated dynamically to a variable.

If we want to change the size of memory allocated by `malloc()` or `calloc()` function, we use `realloc()` function. Without losing the old data, it changes the size of the memory block. The **first argument of this function is a pointer** pointing to the block of memory we allocated before and the **second argument is the updated size** of the memory block. On failure, it returns **NULL**.

If the **new size**(of memory required) is larger than the old size, it retains the data and newly allocated bytes are uninitialized. It might move the old data to a new block of memory with increased size if there's not enough space at the old address.

Syntax:

Let's see the syntax for this function,

```
void* realloc(void *pointer, size_t new-size);
```

Time for an Example: `realloc()` function

Let's see, how we can use this function.

```
int *x;
x = (int*)malloc(50 * sizeof(int));
x = (int*)realloc(x, 100);   //allocated a new memory to variable x
```

Another example depicting the use of `realloc()` function.

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int x,*ptr;
    ptr = (int*)malloc(5 * sizeof(int));
    if(ptr == NULL){
        printf("Memory not available");
        exit (1);
    }
    for(int i = 0; i < 5; i++)
        scanf("%d", ptr + i);
    ptr = (int*)realloc(ptr, 7 * sizeof(int));
    if(ptr == NULL){
        printf("Memory not available");
        exit (1);
    }
    for(int i = 5; i < 7; i++)
        scanf("%d", ptr + i);
    for(int i = 0; i < 7; i++)
        printf("%d : %d\n",i + 1, *(ptr + i));
    free(ptr);
    return 0;
}
```

5 4 6 7 8 3 2
1 : 5
2 : 4
3 : 6
4 : 7
5 : 8
6 : 3
7 : 2

Diffrence between `malloc()` and `calloc()`

`calloc()` Function	`malloc()` Function
`calloc()` function initializes the allocated memory with **0** value.	`malloc()` function initializes the allocated memory with **garbage** values.
Number of arguments is 2	Number of argument is 1
Syntax: `(cast_type *)calloc(blocks, size_of_block);`	**Syntax:** `(cast_type *)malloc(size_in_bytes);`

---

4. `free()` Function

Let's start by knowing the syntax of this function:

```
void free(void *p);
```

The memory we allocate dynamically exists till the end of the program. As a programmer, it is our duty to **release that memory** so that it can be used for other purposes and does not cause memory leaks. We do it as follows:

```
int *x;
x = (int*)malloc(50 * sizeof(int));
free(x);
```

When we allocate dynamic memory in functions, if the function does not return address of the memory dynamically allocated, the memory should be free within that function itself. But if the function returns the dynamically allocated memory's address, then the momory has to be freed in the calling function.

Program to represent Dynamic Memory Allocation(using calloc())

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i, n;
    int *element;

    printf("Enter total number of elements: ");
    scanf("%d", &n);

    /*
        returns a void pointer(which is type-casted to int*)
        pointing to the first block of the allocated space
    */
    element = (int*) calloc(n,sizeof(int));

    /*
        If it fails to allocate enough space as specified,
        it returns a NULL pointer.
    */
    if(element == NULL)
    {
        printf("Error.Not enough space available");
        exit(0);
    }

    for(i = 0; i < n; i++)
    {
        /*
            storing elements from the user
            in the allocated space
        */
       scanf("%d", element+i);
    }
    for(i = 1; i < n; i++)
    {
       if(*element > *(element+i))
       {
            *element = *(element+i);
        }
    }

    printf("Smallest element is %d", *element);

    return 0;
}
```

Enter total number of elements: 5 4 2 1 5 3 Smallest element is 1

In this way, we can make use of dynamic memory allocation in our program.

So this was all about dynamic memory allocation in C language where we used `malloc()` function, `calloc()` function, `realloc()` function, and `free()` function.

Command Line Argument in C

studytonight.com/c/command-line-argument.php

So far we have seen how to input values in C code during compile-time and runtime. To do that, we declared variables in the main() and then worked on them but there is a way to input values without declaring it in the main().

C offers us a feature called **"command line argument"** using which we can enter values from the command line at the time of execution. Command line argument is a parameter supplied to the program when it is invoked or run.

Use of Command Line arguments in C

- They are used when we need to control our program from outside instead of hard-coding it.
- They make installation of programs easier.

Command line argument is an important concept in C programming. Command line arguments are passed to the `main()` method.

Syntax:

```
int main(int argc, char *argv[])
```

Here, `argc` counts the **number of arguments** on the command line and `argv[ ]` is a pointer array which holds pointers of type `char` which points to the arguments passed to the program.

Let's see a simple code example to check whether any command line arguments is provided to the code or not.

```
#include<stdio.h>
int main(int argc, char *argv[])
{
    if(argc < 2)
        printf("No argument supplied. The only argument here is %s", argv[0]);

    return 0;
}
```

Compile the above code using: `gcc filename.c -o filename`

Then run it using: **./filename**

Or you can just use our Online C compiler, but you need to Login to try it out.

Run Code →

No argument supplied. The only argument here is ./a.out

From the above example, we can infer that the **first command line argument is the program file name**, which is always added by default by the compiler.

Hence, **argv[0] = name of our file** and **argc = 1**.

We compiled the above code using command, `gcc filename.c -o filename`. We provided the `-o` flag so that the output file is given the name that we provided.)

Example for Command Line Argument

If we want to print all the arguments in our program, we can do it as follows.

```
#include <stdio.h>
#include <conio.h>

int main(int argc, char *argv[])
{
    int i;
    if( argc >= 2 )
    {
        printf("The arguments supplied are:\n");
        for(i = 1; i < argc; i++)
        {
            printf("%s\t", argv[i]);
        }
    }
    else
    {
        printf("argument list is empty.\n");
    }
    return 0;
}
```

Compile the above code using command: `gcc name_of_file.c`, then run it using: `./a.out Welcome to Studytonight`, we have provided command line argument while running the compiled code.

1 : Welcome
2 : to
3 : Studytonight

We provided 3 words seperated by space as arguments, while running the code. So, they will be considered as three separate arguments.

Remember that `argv[0]` holds the name of the program and `argv[1]` points to the first command line argument and `argv[argc-1]` gives the last argument. If no argument is supplied, `argc` will be 1.

Some more examples...

Let's see how inclusion of **quotes** changes the output of the same program.

Compile the above code again: `gcc name_of_file.c`

Run it using: `./a.out "welcome to studytonight"`

1 : welcome to studytonight

Here, we have provided all the 3 words enclosed in a **double quote**. So, it is considered as a **single argument**. Same thing happens with single quotes.

Compile the same code again: `gcc name_of_file.c`

Then, run it using: `./a.out 'welcome to studytonight'`

1 : welcome to studytonight

Similarly, we can give `int` and `float` type arguments but they will be treated as `strings`.

Again compile the above code: `gcc name_of_file.c`

And run it using: `./a.out 1 0 a 5.07`

1 : 1
2 : 0
3 : a
4 : 5.07

Conclusion

In this way, we can use command line arguments in C. Command Line arguments in C is a great way to provide input to the program at runtime making the program resuable with different input values.